

LAYER: 1_Core

This file is a merged representation of a subset of the codebase, containing files not matching ignore patterns, combined into a single document by Repomix.

<file_summary>

This section contains a summary of this file.

<purpose>

This file contains a packed representation of a subset of the repository's contents that is considered the most important context.

It is designed to be easily consumable by AI systems for analysis, code review, or other automated processes.

</purpose>

<file_format>

The content is organized as follows:

1. This summary section
2. Repository information
3. Directory structure
4. Repository files (if enabled)
5. Multiple file entries, each consisting of:
 - File path as an attribute
 - Full contents of the file

</file_format>

<usage_guidelines>

- This file should be treated as read-only. Any changes should be made to the original repository files, not this packed version.
- When processing this file, use the file path to distinguish between different files in the repository.
- Be aware that this file may contain sensitive information. Handle it with the same level of security as you would the original repository.

</usage_guidelines>

<notes>

- Some files may have been excluded based on .gitignore rules and Repomix's configuration
- Binary files are not included in this packed representation. Please refer to the Repository Structure section for a complete list of file paths, including binary files
- Files matching these patterns are excluded: **/obj/**, **/bin/**, **/*.png, **/*.jpg, **/node_modules/**, **/*.Designer.cs
- Files matching patterns in .gitignore are excluded
- Files matching default ignore patterns are excluded
- Files are sorted by Git change count (files with more changes are at the bottom)

</notes>

</file_summary>

<directory_structure>

Configuration.Web/Components/_Imports.razor
Configuration.Web/Components/Layout/MainLayout.razor
Configuration.Web/Components/Layout/ReconnectModal.razor
Configuration.Web/Components/NavMenu.razor
Configuration.Web/Components/NotFound.razor
Configuration.Web/Components/Routes.razor
Configuration.Web/Configurator/IWebAppConfigurator.cs
Configuration.Web/Configurator/WebAppConfigurator.cs
Configuration.Web/Configurator/WebScanningExtensions.cs
Configuration.Web/Promatis.Net.Configuration.Web.csproj
Configuration.Web/Properties/launchSettings.json
Configuration.Web/Services/TabNavigationService/ComponentRegistry.cs
Configuration.Web/Services/TabNavigationService/TabItem.cs
Configuration.Web/Services/TabNavigationService/TabNavigationService.cs
Configuration.Web/Services/UiModuleService.cs
Configuration.Web/UiCommandBus/IUiCommandBus.cs
Configuration.Web/UiCommandBus/UiCommandBus.cs
Configuration.Web/WebAppBootstrapper.cs
Configuration/AppBootstrapper.cs
Configuration/AppConfigurator.cs
Configuration/IAppConfigurator.cs
Configuration/Promatis.Net.Configuration.csproj
Configuration/ServiceScanningExtensions.cs
Data/Configurations/AuditLogConfiguration.cs
Data/Configurations/ModelBuilderConventions.cs
Data/DBContext/ApplicationDbContext.cs
Data/DBContext/ModelBuilderExtensions.cs

Data/Interceptors/ChangeEntryModel.cs
Data/Interceptors/DatabaseTriggerInterceptor.cs
Data/Interceptors/IUserProvider.cs
Data/Promatis.Net.Data.csproj
Data/Triggers/Castom/AuditTrigger.cs
Data/Triggers/Global/FluentValidationTrigger.cs
Data/Triggers/Global/ReferenceTreeParentTrigger.cs
Data/TriggerService/DatabaseTriggerService.cs
Data/TriggerService/EntityStateChangeEnum.cs
Data/TriggerService/IDatabaseTriggerService.cs
Data/TriggerService/PropertyChangeInfo.cs
Data/TriggerService/TriggerEvents/EntityCancelArgsBase.cs
Data/TriggerService/TriggerEvents/EntityCancelEventArgs.cs
Data/TriggerService/TriggerEvents/EntityChangedEventArgsBase.cs
Data/TriggerService/TriggerEvents/EntityChangedEventArgs.cs
Data/TriggerService/TriggerEvents/EntityEventArgsBase.cs
Data/TriggerService/TriggerInterfaces/IAfterSaveTrigger.cs
Data/TriggerService/TriggerInterfaces/IBeforeSaveTrigger.cs
Data/Validation/GlobalPolymorphicValidator.cs
Domain.Interface/Enums/EnumExtensions.cs
Domain.Interface/Interfaces/IAudit.cs
Domain.Interface/Interfaces/IDomainObject.cs
Domain.Interface/Interfaces/IDomainObjectHasKey.cs
Domain.Interface/Interfaces/ISoftDeletable.cs
Domain.Interface/Interfaces/ITreeNode.cs
Domain.Interface/Promatis.Net.Domain.Interface.csproj
Domain/AuditLog/AuditLog.cs
Domain/DomainObject/DomainObject.cs
Domain/DomainObject/DomainObjectBase.cs
Domain/DomainObject/DomainObjectValidator.cs
Domain/Promatis.Net.Domain.csproj
Domain/ReferenceBase/ReferenceBase.cs
Domain/ReferenceBase/ReferenceBaseValidator.cs
Domain/ReferenceTreeBase/ReferenceTreeBase.cs
Domain/ReferenceTreeBase/ReferenceTreeBaseValidator.cs
Promatis.Net.UI/_Imports.razor
Promatis.Net.UI/Components/EditDialog/DialogTab.razor
Promatis.Net.UI/Components/EditDialog/DialogTab.razor.cs
Promatis.Net.UI/Components/EditDialog/EditDialog.razor
Promatis.Net.UI/Components/EditDialog/EditDialog.razor.cs
Promatis.Net.UI/Components/Grid/GridActionContext.cs
Promatis.Net.UI/Components/Grid/GridPage.razor
Promatis.Net.UI/Components/Grid/GridPage.razor.cs
Promatis.Net.UI/Components/Toolbar/IHasSelectedData.cs
Promatis.Net.UI/Components/Toolbar/IHasToolbar.cs
Promatis.Net.UI/Components/Toolbar/ToolbarActionContext.cs
Promatis.Net.UI/Components/Toolbar/ToolbarCustomAction.cs
Promatis.Net.UI/Components/Toolbar/ToolbarPosition.cs
Promatis.Net.UI/Components/Toolbar/UiToolbar.razor
Promatis.Net.UI/Components/Toolbar/UiToolbar.razor.cs
Promatis.Net.UI/Components/Tree/TreeActionContext.cs
Promatis.Net.UI/Components/Tree/TreePage.razor
Promatis.Net.UI/Components/Tree/TreePage.razor.cs
Promatis.Net.UI/Components/Workspace/IWorkspaceActionContext.cs
Promatis.Net.UI/Components/Workspace/WorkspaceActionContext.cs
Promatis.Net.UI/Components/Workspace/WorkspacePage.razor
Promatis.Net.UI/Components/Workspace/WorkspacePage.razor.cs
Promatis.Net.UI/Dashboard.razor
Promatis.Net.UI/Dashboard.razor.css
Promatis.Net.UI/IUiModule.cs
Promatis.Net.UI/Pages/AuditLog/AuditLogPage.razor
Promatis.Net.UI/Pages/AuditLog/AuditLogPageContext.cs
Promatis.Net.UI/Promatis.Net.UI.csproj
Promatis.Net.UI/Theme/PromatisTheme.cs
Promatis.Net.UI/UiDataBroker.cs
Promatis.Net.UI/UiModule.cs
Promatis.Net.UI/wwwroot/App.css
Service/Contracts/AuditLog/AuditLogSearchRequest.cs
Service/Contracts/AuditLog/PagedResult.cs
Service/Promatis.Net.Service.csproj
Service/Services/AuditLogService/AuditLogService.cs
Service/Services/AuditLogService/IAuditLogService.cs
Service/Services/BaseService/BaseService.cs
Service/Services/BaseService/IBaseService.cs
Service/Services/ReferenceService/IReferenceService.cs
Service/Services/ReferenceService/ReferenceService.cs

```
Service/Services/ReferenceTreeService/IReferenceTreeService.cs
Service/Services/ReferenceTreeService/ReferenceTreeService.cs
Service/Services/TreeService/ITreeService.cs
Service/Services/TreeService/TreeService.cs
Tests/AssemblyInfo.cs
Tests/Domain/DomainLogicTests.cs
Tests/Domain/DomainObjectBaseLogicTests.cs
Tests/Domain/DomainObjectBaseTests.cs
Tests/Domain/DomainObjectExtendedTests.cs
Tests/Domain/ReferenceBaseTests.cs
Tests/Domain/ReferenceTreeTests.cs
Tests/Domain/SoftDeletableTests.cs
Tests/Interceptor/DatabaseTriggerInterceptorTests.cs
Tests/Promatis.Net.ApplicationModel.Tests.csproj
Tests/Service/BaseServiceTests_Crud.cs
Tests/Service/BaseServiceTests.cs
Tests/Service/ReferenceServiceTests_Search.cs
Tests/Service/ReferenceTreeServiceTests.cs
Tests/Service/ServiceIntegrationTests.cs
Tests/Trigger/AuditTriggerTests.cs
Tests/Trigger/DatabaseTriggerServiceTests.cs
Tests/Trigger/FluentValidationTriggerTests.cs
Tests/Trigger/FullTriggerChainTests.cs
Tests/Trigger/ReferenceTreeParentTriggerTests.cs
Tests/Trigger/TreeTriggerChainTests.cs
Tests/Validator/DomainObjectValidatorTests.cs
Tests/Validator/ReferenceBaseValidatorTests.cs
Tests/Validator/ReferenceTreeBaseValidatorTests.cs
</directory_structure>
```

<files>

This section contains the contents of the repository's files.

<file path="Configuration.Web/Components/_Imports.razor">

```
@using System.Net.Http
@using System.Net.Http.Json
@using Microsoft.AspNetCore.Components.Forms
@using Microsoft.AspNetCore.Components.Routing
@using Microsoft.AspNetCore.Components.Web
@using Microsoft.JSInterop
@using MudBlazor
@using Promatis.Net.Ui
@using Promatis.Net.Configuration.Web
@using Promatis.Net.Configuration.Web.Components
@using Promatis.Net.Configuration.Web.Components.Layout
@using static Microsoft.AspNetCore.Components.Web.RenderMode
</file>
```

<file path="Configuration.Web/Components/Layout/MainLayout.razor">

```
@inherits LayoutComponentBase
@using Promatis.Net.Ui.Theme
@
@inject UiModuleService UiService
@inject TabNavigationService TabService
@implements IDisposable
@
@* A,,!Aô A A´ Aô : B 2Dô7D´2C 5CÂ ?D >C$0C"4CT@ D$5CÄK D DC´0C4>CÂ BE <Cô>C4> D 5Cd8CÄ0 *@
<MudThemeProvider Theme="PromatisTheme.DefaultTheme" @bind-IsDarkMode="_isDarkMode" />
<MudPopoverProvider />
@
<MudDialogProvider FullWidth="false"
    MaxWidth="MaxWidth.False"
    CloseButton="false"
    BackdropClick="false"
    NoHeader="false"
    Position="DialogPosition.Center"
    CloseOnEscapeKey="true" />
@
<MudSnackbarProvider />
@
<CascadingValue Value="this">
    <MudLayout>
        @* A$5D ECô0Dò ?C =CT;DÂ ?D 8C´>Cd5Cô8Dò ¢
        <MudAppBar Color="Color.Default" Elevation="1">
            @* A,,!Aô A A´ Aô : At0CÄ5C05C0> Color.Primary C00 Color.Default *@
            <MudIconButton Icon="@Icons.Material.Filled.Menu"

```

```

                Color="Color.Inherit"
                Edge="Edge.Start"
                Size="Size.Large"
                OnClick="ToggleDrawer" />
<MudStack Spacing="-1">
    @* AD5DD>C'BC0>CR =C 7C$0C08CR
    <MudText Typo="Typo.h6" Class="ml-2">
        B$5D BCä2D'9 C0@Cä5C=B Promatis Net
    </MudText>
</MudStack>

<MudSpacer />

<MudIconButton Icon="@(_isDarkMode ? Icons.Material.Filled.LightMode :
Icons.Material.Filled.DarkMode)"
                Color="Color.Inherit"
                OnClick="ToggleDarkMode" />

</MudAppBar>
@* A00C05C'L CÄ5C0N *
<MudDrawer @bind-Open="_drawerOpen" ClipMode="DrawerClipMode.Always"
Variant="DrawerVariant.Temporary" Overlay="true" OverlayAutoClose="true" Width="220px"
Elevation="1">
    <NavMenu />
</MudDrawer>

<MudMainContent Class="pt-16 px-4 pb-4 d-flex flex-column"
                Style="height: calc(100vh - 8px); min-height: calc(100vh - 8px); margin-
top: 10px; display: flex; flex-direction: column; flex-grow: 1; overflow: hidden;">
    @* A,,!A0 A A' A0 : A4;Cä1C ;DÄ=D'9 ErrorBoundary D41D 0C0 A C$5D EC05C4> D4@Cä2C00. B BC,,;C, fÆW
    @if (TabService.OpenTabs.Count == 0)
    {
        <div class="flex-grow-1 overflow-auto">
            <ErrorBoundary>
                <ChildContent>
                    @Body
                </ChildContent>
                <ErrorContent Context="exception">
                    <MudAlert Severity="Severity.Error" Class="my-4">
                        A0@Cä8Ct>D,,;C :D 8D$8Dt5D :C 0 CäHC,,1C=0 C,,=D$5D DCT9D 0: @exception.Message
                    </MudAlert>
                </ErrorContent>
            </ErrorBoundary>
        </div>
    }
    else
    {
        <MudTabs @bind-ActivePanelIndex="TabService.ActiveTabIndex"
                Position="Position.Top"
                Color="Color.Default"
                SliderColor="Color.Primary"
                Class="mdi-tabs-container w-100 flex-grow-1 d-flex flex-column"
                Style="height:100%; max-height: 100%; min-height: 0; display: flex; flex-
direction: column; flex-grow: 1;">
            TabPanelsClass="pa-0 flex-grow-1 d-flex flex-column overflow-hidden"
            ApplyEffectsToContainer="true"
            TabButtonsClass="rounded-t-lg shadow-sm"
            Outlined="true">

                @for (int i = 0; i < TabService.OpenTabs.Count; i++)
                {
                    int localIndex = i;
                    TabItem tab = TabService.OpenTabs[i];

                    @* A,,!A0 A A' A0 : AD>C 0C$;CT=C fÆW,0AD$@D4:D$CD 0 CD;D0 AC <Cä3Cä BC 1-C00C05C'0,
                    <MudTabPanel @key="tab" ID="@localIndex" Class="h-100 d-flex flex-column">
                        <TabContent>
                            <div class="d-flex align-center gap-2" style="cursor: pointer;">
                                @if (!string.IsNullOrEmpty(tab.Icon))
                                {
                                    <MudIcon Icon="@tab.Icon" Size="Size.Small" />
                                }
                                <MudText Typo="Typo.body2" Class="font-weight-
medium">@tab.Title</MudText>
                            </div>
                        </TabContent>
                    </MudTabPanel>
                }
            </MudTabs>
        </div>
    }
}

```

```

<MudIconButton Icon="@Icons.Material.Filled.Close"
                Size="Size.Small"
                Color="Color.Default"
                Class="pa-0 ms-1"
                Style="width: 16px; height: 16px; min-width:
16px;"
                OnClick="@((e) =>
TabService.CloseTabByIndex(localIndex))" />
            </div>
        </TabContent>
        <ChildContent>
            @* B0BCäB C=C0BCT9C05D 6CTAD$:Cä 7C =C,,<C 5D" R 2D'ACäBD² 2C;C 4C=8 C,
            <div class="pa-2 rounded-b-lg w-100 d-flex flex-column flex-grow-1"
                Style="height: 100%; max-height: 100%; min-height: 0;
overflow: hidden;">
                <
                <ErrorBoundary>
                    <ChildContent>
                        @* A,,!Aö A A' Aö : AÄK Cä1Cä@C GC,,2C 5CÄ G-æ Ö-46ö× öæVÇB 2 DD8C
                        @* C=C>D$>D KC' AC,,;Cä9 C$KD$0C48C$0CTB D BD 0C08DdC C00D,,5C4> CD5
                        <div class="d-flex flex-column flex-grow-1 w-100"
                            <DynamicComponent Type="@tab.ComponentType" />
                            </div>
                        </ChildContent>
                        <ErrorContent Context="exception">
                            <MudAlert Severity="Severity.Error" Class="my-4">
                                Aö@Cä8Ct>D,,;C :D 8D$8Dt5D :C 0 CäHC,,1C=0 C,,=D$5D DCT9D 0 C$:
                                </MudAlert>
                            </ErrorContent>
                        </ErrorBoundary>
                    </div>
                </ChildContent>
            </MudTabPanel>
        }
    </MudTabs>
}
</MudMainContent>
</MudLayout>
</CascadingValue>
@code {
    private bool _drawerOpen = true;
    private bool _isDarkMode; // Aö> D4<Cä;Dt0C08Dä ?D 8C'>Cd5C08CR 7C ?D4AC=0CTBD 0 C" AC$5D$;Cä9 D$5CÄ5D
    protected override void OnInitialized()
    {
        base.OnInitialized();
        TabService.OnTabsChanged += HandleTabsChanged;
        _drawerOpen = false;
    }

    private void ToggleDrawer() => _drawerOpen = !_drawerOpen;

    /// <summary>
    /// Aö5D 5C;DäGC 5D" DC'0C2 BE <C0>C4> D 5Cd8CÄ0 C, 7C AD$0C$;Dö5D" xVEF†VÖU &÷f-FW" <C4=Cä2CT=C0> C05D
    /// </summary>
    private void ToggleDarkMode() => _isDarkMode = !_isDarkMode;

    private void HandleTabsChanged() => InvokeAsync(StateHasChanged);

    public void CloseDrawer()
    {
        if (_drawerOpen)
        {
            _drawerOpen = false;
            StateHasChanged();
        }
    }

    public void Dispose() => TabService.OnTabsChanged -= HandleTabsChanged;
}
</file>

<file path="Configuration.Web/Components/Layout/ReconnectModal.razor">

```

```

<script type="module" src="@Assets[\"Components/Layout/ReconnectModal.razor.js\"]"></script>
<dialog id="components-reconnect-modal" data-nosnippet>
  <div class="components-reconnect-container">
    <div class="components-rejoining-animation" aria-hidden="true">
      <div></div>
      <div></div>
    </div>
    <p class="components-reconnect-first-attempt-visible">
      Rejoining the server...
    </p>
    <p class="components-reconnect-repeated-attempt-visible">
      Rejoin failed... trying again in <span id="components-seconds-to-next-attempt">
seconds.
    </p>
    <p class="components-reconnect-failed-visible">
      Failed to rejoin.<br />Please retry or reload the page.
    </p>
    <button id="components-reconnect-button" class="components-reconnect-failed-visible">
      Retry
    </button>
    <p class="components-pause-visible">
      The session has been paused by the server.
    </p>
    <p class="components-resume-failed-visible">
      Failed to resume the session.<br />Please retry or reload the page.
    </p>
    <button id="components-resume-button" class="components-pause-visible components-resume-
failed-visible">
      Resume
    </button>
  </div>
</dialog>
</file>

<file path="Configuration.Web/Components/NavMenu.razor">
@using System.Reflection
@inject UiModuleService UiService
@inject TabNavigationService TabService
@inject ComponentRegistry Registry
<MudNavMenu>
  @{}
  List<(string Title, string Href, string Icon, string? Group)> allItems = UiService.Modules
    .SelectMany(m => m.GetMenuItems())
    .ToList();

  // AäBD 8D >C$K$0CT< D0;CT<CT=D$K C$5D EC05C4> D4@Cä2C00
  IEnumerable<(string Title, string Href, string Icon, string? Group)> topLevelItems =
allItems.Where(i => string.IsNullOrEmpty(i.Group));
  foreach ((string Title, string Href, string Icon, string? Group) item in topLevelItems)
  {
    <!-- A,,!Aô A A´ Aô : A$<CTAD$> Href C,,ACô>C´LCtCCT< OnClick CD;Dò >D$:D KD$8Dò 2Cª;C 4Cä: C 5Cr
    <MudNavLink Icon="@item.Icon" OnClick="@(() => OpenMenuAsTab(item.Title, item.Href,
item.Icon))">
      @item.Title
    </MudNavLink>
  }
  // A4@D4?C08D CCT< D0;CT<CT=D$K C0> Cª0D$5C4>D 8D0<D
  IOrderedEnumerable<IGrouping<string, (string Title, string Href, string Icon, string?
Group)>> pooledGroups = allItems
    .Where(i => !string.IsNullOrEmpty(i.Group))
    .GroupBy(i => i.Group!)
    .OrderBy(g => g.Key);

  foreach (IGrouping<string, (string Title, string Href, string Icon, string? Group)> group
in pooledGroups)
  {
    <MudNavGroup Title="@group.Key" Icon="@Icons.Material.Filled.Folder" Expanded="false">
      @foreach ((string Title, string Href, string Icon, string? Group) item in group)
      {
        <!-- A,,!Aô A A´ Aô : B41D 0C0 ‡&VbÂ 4Cä1C 2C´5C0 öä6Æ-6² òÓí
        <MudNavLink Icon="@item.Icon" OnClick="@(() => OpenMenuAsTab(item.Title,
item.Href, item.Icon))">
          @item.Title

```

```

        </MudNavLink>@
    }@
    </MudNavGroup>@
}
</MudNavMenu>@
@
@code {
    /// <summary>@
    /// A@5D 5DT2C BD´2C 5CÂ AC@2Cä7C0>C´ MC@7CT<C0;Dô@ D >CD8D$5C´LD :Cä3Câ <C :CTBC
    /// </summary>@
    [CascadingParameter]@
    protected MainLayout? MainLayoutInstance { get; set; }@
}
    protected override void OnInitialized()@
    {
        base.OnInitialized();@
        IEnumerable<Assembly> assemblies = UiService.Modules.Select(m =>
m.GetType().Assembly).Distinct();@
        Registry.RegisterModules(assemblies);@
    }
}
    private void OpenMenuAsTab(string title, string href, string icon)@
    {
        Type? componentType = Registry.GetComponentTypeByRoute(href);@
}
        if (componentType != null)@
        {
            // AäBC@D´2C 5CÂ 2C@;C 4C@C@
            TabService.OpenTab(title, href, icon, componentType);@
}
            // A 2D$>CÄ0D$8Dt5D :C, 7C :D KC$0CT< C >C@>C$>CR <CT=Dâ ?Cä2CT@DR >C@=C ?CäAC´5 C@;C,, :C
            MainLayoutInstance?.CloseDrawer();@
        }@
        else@
        {
            System.Diagnostics.Debug.WriteLine($"AôCD$L {href} C05 Ct0D 5C48D BD 8D >C$0C0 2 C@>CÄ?Cä=CT=D$0D
        }@
    }@
}
</file>

<file path="Configuration.Web/Components/NotFound.razor">
@page "/not-found"@
@layout MainLayout@
@
<PageTitle>404 - B BD 0C08Dd0 C05 C00C"4CT=C Âö vUF-FÆSí
@
<MudContainer MaxWidth="MaxWidth.Small" Class="d-flex align-center justify-center" Style="height:
70vh;">@
    <MudPaper Elevation="3" Class="pa-8 text-center" Style="width: 100%;">@
        <MudIcon Icon="@Icons.Material.Filled.SearchOff" Color="Color.Error" Size="Size.Large"
Class="mb-4" />@
        <MudText Typo="Typo.h5" Class="mb-2">AäHC,,1C@0 404</MudText>@
        <MudText Typo="Typo.body2" Class="mud-text-secondary mb-6">@
            A,,7C$8C08D$5, Ct0C0@C HC,,2C 5CÄKC´ 0CD@CTA C05 D CD"5D BC$CCTB, C´8C > D >CäBC$5D$AD$2D4ND"8C´ <C
        </MudText>@
        <MudButton Variant="Variant.Filled" Color="Color.Primary" href="/">@
            A$5D =D4BDÄADò =C 3C´0C$=D4ND
        </MudButton>@
    </MudPaper>@
</MudContainer>
</file>

<file path="Configuration.Web/Components/Routes.razor">
@using System.Reflection@
@inject UiModuleService UiService@
@
<Router AppAssembly="@typeof(Routes).Assembly"@
    AdditionalAssemblies="@_moduleAssemblies"@
    NotFoundPage="@typeof(NotFound)">@
    <Found Context="routeData">@
        <RouteView RouteData="@routeData" DefaultLayout="@typeof(MainLayout)" />@
        <FocusOnNavigate RouteData="@routeData" Selector="h1" />@
    </Found>@
</Router>@

```

```

Ð
@code {Ð
    private Assembly[] _moduleAssemblies = [];Ð
Ð
    protected override void OnInitialized()Ð
    {Ð
        _moduleAssemblies = UiService.ModulesÐ
            .Select(m => m.GetType().Assembly)Ð
            .Distinct()Ð
            .ToArray();Ð
        Ð
        base.OnInitialized();Ð
    }Ð
}
</file>

<file path="Configuration.Web/Configurator/IWebAppConfigurator.cs">
namespace Promatis.Net.Configuration.Web;Ð
Ð
public interface IWebAppConfigurator : IAppConfiguratorÐ
{Ð
    void ConfigureMiddleware(WebApplication app);Ð
}
</file>

<file path="Configuration.Web/Configurator/WebAppConfigurator.cs">
using System.Reflection;Ð
using Microsoft.AspNetCore.Components;Ð
using MudBlazor.Services;Ð
Ð
namespace Promatis.Net.Configuration.Web;Ð
Ð
/// <summary>Ð
/// A 0Ct>C$KC' :Cä=DD8C4CD 0D$>D 4C`O Web-Cô@C,,;Cä6CT=C,,9 C00 C 0Ct5 Blazor C, xVD&Æ |÷"í
/// </summary>Ð
public class WebAppConfigurator<TRootComponent> : AppConfigurator, IWebAppConfiguratorÐ
    where TRootComponent : IComponentÐ
{Ð
    public override void ConfigureServices(IServiceCollection services, IConfiguration
configuration)Ð
    {Ð
        // 1. A 0Ct>C$0Dò 8C0DD 0D BD CC=BD4@C ,, ABÂ !CT@C$8D K, A$0C`8CD0D$>D K, B$@C,,3C45D K, B :C =CT@ DL
        base.ConfigureServices(services, configuration);Ð
Ð
        // 2. UI C,,=DD@C AD$@D4:D$CD 0 (Aô>C,,ACf •V"öGVÆR 2 D 1Cä@C=0DRÂ =C 2C,,3C FC,,0)Ð
        services.AddWebInfrastructure(projectPrefix: "Promatis.");Ð
Ð
        // B B "AT A A A Aä (MDI): B 5C48D BD 8D CCT< C,,=DD@C AD$@D4:D$CD C CÄ=Cä3Cä2C;C 4CäGC0>C4> C,,
        services.AddSingleton<ComponentRegistry>();Ð
        services.AddScoped<TabNavigationService>();Ð
Ð
        // 3. B ?CTFC,,DC,,:C &Æ |÷" ,,~çFW& 7F~fR 6W'fW"•
        services.AddRazorComponents()Ð
            .AddInteractiveServerComponents();Ð
Ð
        // 4. B 5C48D BD 0Dd8Dò 8 C00D BD >C":C xVD&Æ |÷-
        services.AddMudServices(config =>Ð
        {Ð
            config.SnackbarConfiguration.PositionClass =
MudBlazor.Defaults.Classes.Position.BottomRight;Ð
            config.SnackbarConfiguration.PreventDuplicates = false;Ð
            config.SnackbarConfiguration.NewestOnTop = true;Ð
            config.SnackbarConfiguration.ShowCloseIcon = true;Ð
            config.SnackbarConfiguration.VisibleStateDuration = 5000;Ð
        });Ð
Ð
        // 5. B,,8C00 UI-D >C KD$8C•
        services.AddScoped<IUiCommandBus, UiCommandBus>();Ð
    }Ð
}
Ð
/// <summary>Ð
/// A00D BD >C":C :Cä=C$5C"5D 0 Cä1D 0C >D$:C, ...EE 07C ?D >D >C" ,,ö~FFÆWv &R'í
/// </summary>Ð
public virtual void ConfigureMiddleware(WebApplication app)Ð
{Ð
    // A 0Ct>C$KCR ö~FFÆWv &W2 4C`O D 0C >D$K Web-Cô@C,,;Cä6CT=C,,0D

```



```

app.UseStaticFiles();
app.UseAntiforgery();

// 1. A0>C'CDt0CT< Ct0D 5C48D BD 8D >C$0C0=D'5 UI-D 1Cä@C8 C,,7 DI-C8>C0BCT9C05D 0 DT>D BC
using IServiceScope scope = app.Services.CreateScope();
var uiService = scope.ServiceProvider.GetRequiredService<UiModuleService>();

Assembly[] moduleAssemblies = uiService.Modules
    .Select(m => m.GetType().Assembly)
    .Distinct()
    .ToArray();

// A,, A,,&A,, A' At Bd B0 A B$+ A$ A' AD A£¢ C ?Cä;C00CT< D 5CTAD$@ D >D4BCä2 C8>CÄ?Cä=CT=D$0CÄ8 C,,7
var componentRegistry = scope.ServiceProvider.GetRequiredService<ComponentRegistry>();
componentRegistry.RegisterModules(moduleAssemblies);

// 2. B 5C48D BD 8D CCT< C8>D =CT2Cä9 C8>CÄ?Cä=CT=D" 8 D :C =C,,@D45CÄ AD$@C =C,,FD² <Cä4D4;CT9 C00 D 5
app.MapRazorComponents<TRootComponent>()
    .AddInteractiveServerRenderMode()
    .AddAdditionalAssemblies(moduleAssemblies);
}

public override void ConfigureApp(IHost app)
{
    // A$Kct>C" 1C 7Cä2Cä9 C,,=C,,FC,,0C'8Ct0Dd8C, ,,0C$BCä@CT3C,,AD$@C FC,,0 D$@C,,3C45D >C" AB GCT@CT7 D :C =
    base.ConfigureApp(app);

    // At4CTADÄ <Cä6C0> CD>C 0C$8D$L C0@Cä2CT@C8C Ct4Cä@Cä2DÄ0 A C,,;C, =C GC ;DÄ=D'9 Seed CD0C0=D'E CD;
}
}
</file>

<file path="Configuration.Web/Configurator/WebScanningExtensions.cs">
using Promatis.Net.UI;
using Promatis.Net.UI.Components.Workspace;
using System.Reflection;
namespace Promatis.Net.Configuration.Web;
public static class WebInfrastructureExtensions
{
    public static void AddWebInfrastructure(this IServiceCollection services, string projectPrefix
= "Promatis.")
    {
        Console.WriteLine("[SCANNER] At0C0CD : C0@Cä3D 5C$0 D 1Cä@Cä: CÄ>C0>C'8D$0 (B$>C'LC8> UI-CÄ>CDCC'8)..
        string binPath = AppContext.BaseDirectory;

        // A,,!A0 A A' A0 : A00 D4@Cä2C05 DD0C";Cä2Cä9 D 8D BCT<D² 8D"5CÄ BCä;DÄ:Cä BCR FÆÄÄ :CäBCä@D'5 C00Dt
        string[] assemblyFiles = Directory.GetFiles(binPath, $"{projectPrefix}*.UI.dll",
SearchOption.TopDirectoryOnly);

        foreach (string file in assemblyFiles)
        {
            try
            {
                // A0@C,,=D44C,,BCT;DÄ=Cä 7C 3D CCd0CT< C" 76VÖ&Ç"/Æ0 D6öçFW†BäFVf VÇM
                Assembly.LoadFrom(file);
            }
            catch (Exception ex)
            {
                Console.WriteLine($"[SCANNER] A05 D44C ;CäADÄ 7C 3D CCT8D$L DD0C"; D 1Cä@C8 {Path.GetFileName
}
}

// A,,!A0 A A' A0 : BD8C'LD$@D45CÄ FöÖ -âÄ 1CT@D0 AC >D :C, AD$@Cä3Cä A C0@CTDC,,:D >CÄ 8 Cä:Cä=Dt0
Assembly[] assemblies = AppDomain.CurrentDomain.GetAssemblies()
    .Where(a => a.FullName != null
        && a.FullName.StartsWith(projectPrefix)
        && a.GetName().Name!.EndsWith(".UI"))
    .Distinct()
    .ToArray();

Console.WriteLine($"[SCANNER] B :C =C,,@Cä2C =C,,5 Ct0C$5D HCT=Cää C 9CD5C0> {assemblies.Length} Dd5C'
// A B$ ÄÄ B$ Bt B A / B A4 B "B Bd B0 T'Ô Ää B$ A8!B$ A" A' B$Ää ÄÄ+ (AD A A$ AT Ää•

```

```

Console.WriteLine("[SCANNER] B 5C48D BD 0Dd8Dò 22Ô:Cä=D$5CðAD$>C" AD$@C =C,,F Cò> CÄ0D :CT@D2 •v÷&·7
var contextTypes = assembliesð
    .SelectMany(s => s.GetTypes())ð
    .Where(t => typeof(IWorkspaceActionContext).IsAssignableFrom(t)ð
        && !t.IsAbstractð
        && !t.IsInterfaceð
        // A,,!Aô A A' Aô : Aô@Cä?D4ACð0CT< CäBCð@D'BD'5 generic-D,,0C ;Cä=D² AC <Cä3Cä OCD@C
        && !t.IsGenericTypeDefinition);ð
ð
int registeredContextsCount = 0;ð
foreach (Type type in contextTypes)ð
{ð
    // 1. B 5C48D BD 8D CCT< D 0CÄ :Cä=Cð@CTBCôKC' ?D 8Cð;C 4CÔ>C' :Cä=D$5CðAD" ,,=C ?D 8CÄ5D Â VF-DÄ
    services.AddTransient(type);ð
    registeredContextsCount++;ð
ð
    Console.WriteLine($"          % % [Að Aô"AT B "] {type.Name}");ð
ð
    // 2. A 5D 5CÄ AD$@Cä3Cä 1C'8Cd0C"HC,,9 C 0Ct>C$KC' BC,,? (CÔ0Cô@C,,<CT@, GridActionContext<AuditLog
    Type? baseType = type.BaseType;ð
    if (baseType != null && baseType.IsGenericType)ð
    {ð
        services.AddTransient(baseType, type);ð
ð
        string genericArgs = string.Join(", ", baseType.GetGenericArguments().Select(a =>
a.Name));ð
        string baseTypeName = baseType.Name.Split('.')[0];ð
        Console.WriteLine($"          % % % AÄ0D :C ¢ ¶& 6UG- Tæ ÖWÓÇ¶vVæW&-4 &w7Óâ""½
    }ð
}ð
Console.WriteLine($"[SCANNER] B4ACô5D,,=Cä @C 7C$5D =D4BCä ·&Vv-7FW&VD6öçFW‡G46÷VçGò :Cä=D$5CðAD$>C" C
ð
Console.WriteLine();ð
ð
Console.WriteLine("[SCANNER] B 5C48D BD 0Dd8Dò T'ô:Cä<Cô>Cô5CôBCä2 C, <Cä4D4;CT9 Cô0C$8C40Dd8C, GCT@C
ð
// A 2D$>CÄ0D$8Dt5D :Cä5 D :C =C,,@Cä2C =C,,5 C, @CT3C,,AD$@C FC,,0 C,,=D$5D DCT9D >C" •V"öôGVÆ]
services.Scan(scan => scanð
    .FromAssemblies(assemblies)ð
    .AddClasses(c => c.AssignableTo<IUiModule>()).Where(t => !t.IsAbstract))ð
    .AsImplementedInterfaces()ð
    .WithScopedLifetime()ð
);ð
ð
// B 5C48D BD 0Dd8Dò 2C HCT3Cä 0C4@CT3C BCä@C <Cä4D4;CT9ð
services.AddScoped<UiModuleService>();ð
ð
// A'>C48D >C$0Cô8CR >C =C @D46CT=CôKDR ?D4=CðBCä2 CÄ5CÔNð
IEnumerable<Type> uiModuleTypes = assembliesð
    .SelectMany(a => a.GetTypes())ð
    .Where(t => typeof(IUiModule).IsAssignableFrom(t) && !t.IsInterface && !t.IsAbstract);ð
ð
foreach (Type type in uiModuleTypes)ð
{ð
    tryð
    {ð
        if (Activator.CreateInstance(type) is IUiModule module)ð
        {ð
            Console.WriteLine($"[SCANNER] UI AÄ>CDCC'L: {module.Name}");ð
            List<(string Title, string Href, string Icon, string? Group)> menuItems =
module.GetMenuItems()?.ToList() ?? new();ð
ð
            if (!menuItems.Any())ð
            {ð
                Console.WriteLine($"          % % [AôCD BCä5 CÄ5CÔN]");ð
                continue;ð
            }ð
ð
            IEnumerable<(string Title, string Href, string Icon, string? Group)> rootItems
= menuItems.Where(i => string.IsNullOrEmpty(i.Group));ð
            IEnumerable<IGrouping<string?, (string Title, string Href, string Icon, string?
Group)>> groupedItems = menuItems.Where(i => !string.IsNullOrEmpty(i.Group)).GroupBy(i => i.Group);ð
ð
            foreach ((string Title, string Href, string Icon, string? Group) item in
rootItems)ð
            {ð

```

```

        Console.WriteLine($"          % % {item.Title} -> {item.Href}");
    }
    foreach (IGrouping<string?, (string Title, string Href, string Icon, string?
Group)> group in groupedItems)
    {
        Console.WriteLine($"          % % [{group.Key}]");
        List<(string Title, string Href, string Icon, string? Group)> itemsInGroup
= group.ToList();
        for (int i = 0; i < itemsInGroup.Count; i++)
        {
            string prefix = (i == itemsInGroup.Count - 1) ? "          % % % " :
            Console.WriteLine($" {itemsInGroup[i].Title} ->
{itemsInGroup[i].Href}");
        }
    }
}
catch (Exception ex)
{
    Console.WriteLine($"[SCANNER] AäHC,,1Cð0 DtBCT=C,,O CÄ5CÔN CD;Dò BC,,?C .G— Rää ÖWÓ¢ ¶W,äÖW76 v
}
}
Console.WriteLine("[SCANNER] B 5C48D BD 0Dd8Dò T'Ô:Cä<Cô>CÔ5CÔBCä2 D4ACô5D,,=Cä 7C 2CT@D,,5CÔ0");
Console.WriteLine();
}
}
</file>

<file path="Configuration.Web/Promatis.Net.Configuration.Web.csproj">
<Project Sdk="Microsoft.NET.Sdk.Web">
    <PropertyGroup>
        <TargetFramework>net10.0</TargetFramework>
        <ImplicitUsings>enable</ImplicitUsings>
        <Nullable>enable</Nullable>
        "Ä÷WG WEG— SäÆ-'& '"Âô÷WG WEG— Sí
    </PropertyGroup>
    <ItemGroup>
        <PackageReference Include="MudBlazor" Version="9.4.0" />
    </ItemGroup>
    <ItemGroup>
        <ProjectReference Include="..\Configuration\Promatis.Net.Configuration.csproj" />
        <ProjectReference Include="..\Promatis.Net.UI\Promatis.Net.UI.csproj" />
    </ItemGroup>
</Project>
</file>

<file path="Configuration.Web/Properties/launchSettings.json">
{
    "profiles": {
        "Promatis.Net.Configuration.Web": {
            "commandName": "Project",
            "launchBrowser": true,
            "environmentVariables": {
                "ASPNETCORE_ENVIRONMENT": "Development"
            },
            "applicationUrl": "https://localhost:56819;http://localhost:56820"
        }
    }
}
</file>

<file path="Configuration.Web/Services/TabNavigationService/ComponentRegistry.cs">
using System.Reflection;
using Microsoft.AspNetCore.Components;
namespace Promatis.Net.Configuration.Web;
public class ComponentRegistry
{
    private readonly Dictionary<string, Type> _routeMap = new(StringComparer.OrdinalIgnoreCase);

```

```

D
/// <summary>D
/// B :C =C,,@D45D" AC >D :C, <Cä4D4;CT9 C, AD$@Cä8D" :C @D$C: URL -> B Æ 72 "C,,? C¤>CÄ?Cä=CT=D$0D
/// </summary>D
public void RegisterModules(IEnumerable<Assembly> assemblies)D
{D
    foreach (Assembly assembly in assemblies)D
    {D
        IEnumerable<Type> componentTypes = assembly.GetTypes()D
            .Where(t => typeof(IComponent).IsAssignableFrom(t) && !t.IsAbstract);D
D
        foreach (Type type in componentTypes)D
        {D
            // A,,ICT< C BD 8C CD" µ&÷WFT GG&- 'WfU0Â 2 C¤>D$>D KC' :Cä<C08C´8D CCTBD 0 CD8D 5C¤BC,,2C
            IEnumerable<RouteAttribute> routeAttributes =
type.GetCustomAttributes<RouteAttribute>();D
            foreach (RouteAttribute attr in routeAttributes)D
            {D
                _routeMap[attr.Template] = type;D
            }D
        }D
    }D
}D
D
public Type? GetComponentTypeByRoute(string route)D
{D
    return _routeMap.TryGetValue(route, out Type? type) ? type : null;D
}D
}
</file>

<file path="Configuration.Web/Services/TabNavigationService/TabItem.cs">
namespace Promatis.Net.Configuration.Web;D
D
public class TabItemD
{D
    public string Id { get; init; } = string.Empty; // A00D, ‡&Vm
    public string Title { get; init; } = string.Empty;D
    public string Icon { get; init; } = string.Empty;D
    public Type ComponentType { get; init; } = null!;D
}
</file>

<file path="Configuration.Web/Services/TabNavigationService/TabNavigationService.cs">
namespace Promatis.Net.Configuration.Web;D
D
public class TabNavigationServiceD
{D
    public List<TabItem> OpenTabs { get; } = new();D
    public int ActiveTabIndex { get; set; }D
    public event Action? OnTabsChanged;D
D
    public void OpenTab(string title, string href, string icon, Type componentType)D
    {D
        TabItem? existingTab = OpenTabs.FirstOrDefault(t => t.Id.Equals(href,
StringComparison.OrdinalIgnoreCase));D
D
        if (existingTab != null)D
        {D
            ActiveTabIndex = OpenTabs.IndexOf(existingTab);D
        }D
        elseD
        {D
            var newTab = new TabItemD
            {D
                Id = href,D
                Title = title,D
                Icon = icon,D
                ComponentType = componentTypeD
            };D
D
            OpenTabs.Add(newTab);D
            ActiveTabIndex = OpenTabs.Count - 1;D
        }D
D
        NotifyStateChanged();D
    }
}

```

```

    }
}

/// <summary>
/// A 5Ct>C00D =Cä5 Ct0C@D'BC,,5 C$:C'0CD:C, ?Cä 5E GC,,AC'>C$>CÄC C,,=CD5C@AD=
/// </summary>
public void CloseTabByIndex(int index)
{
    if (index < 0 || index >= OpenTabs.Count) return;

    OpenTabs.RemoveAt(index);

    // A,,=D$5C';CT:D$CC ;DÄ=D'9 D 0D GCTB DD>C@CD 0 C00 CäAD$0C$HC,,5D 0 C$:C'0CD:C•
    if (ActiveTabIndex >= OpenTabs.Count)
    {
        ActiveTabIndex = OpenTabs.Count - 1;
    }

    if (ActiveTabIndex < 0 && OpenTabs.Count > 0)
    {
        ActiveTabIndex = 0;
    }

    NotifyStateChanged();
}

private void NotifyStateChanged() => OnTabsChanged?.Invoke();
}
</file>

<file path="Configuration.Web/Services/UiModuleService.cs">
using Promatis.Net.UI;
namespace Promatis.Net.Configuration.Web;
{
    // B 5D 2C,,A, C@>D$>D KC' ACä1C,,@C 5D" 2D Q C$>CT4C,,=Cí
    public class UiModuleService(IEnumerable<IUiModule> modules)
    {
        public IEnumerable<IUiModule> Modules => modules;
    }
}
</file>

<file path="Configuration.Web/UiCommandBus/IUiCommandBus.cs">
using MudBlazor;
namespace Promatis.Net.Configuration.Web;
{
    /// <summary>
    /// A,,=D$5D DCT9D HC,,=D² T'ÔACä1D'BC,,9D
    /// </summary>
    public interface IUiCommandBus
    {
        /// <summary>
        /// B 5C48D BD 0Dd8Dò >C @C 1CäBDt8C@0 C$Kct>C$0 DD>D <D² ,,2D'?Cä;C00CTBD 0 C0@C,,=C,,<C ND"8CÄ <Cä4D4;CT<,
        /// </summary>
        void RegisterHandler(string commandName, Func<Dictionary<string, object>, Task<DialogResult?>> handler);
    }

    /// <summary>
    /// A$7C 8CÄ>CD5C"AD$2C,,5 CÄ5Cd4D2 <Cä4D4;Dô<C, 2D ;CT?D4N (C$KctKC$0CTBD 0 C,,7 MDM)
    /// </summary>
    Task<DialogResult?> ExecuteAsync(string commandName, Dictionary<string, object> parameters);
}
</file>

<file path="Configuration.Web/UiCommandBus/UiCommandBus.cs">
using MudBlazor;
namespace Promatis.Net.Configuration.Web;
{
    /// <summary>
    /// B,,8C00 UI-D >C KD$8C•
    /// </summary>
    public class UiCommandBus : IUiCommandBus
    {
        // A0>D$>C@>C 5Ct>C00D =D'9 D ;Cä2C @DÄ 4C'0 D 5C48D BD 0Dd8C, :Cä<C =CB 2 D 0C0BC 9CÄ5 SignalR
        private readonly System.Collections.Concurrent.ConcurrentDictionary<string,

```

```

Func<Dictionary<string, object>, Task<DialogResult?>>> _handlers = new();
{
    public void RegisterHandler(string commandName, Func<Dictionary<string, object>,
Task<DialogResult?>> handler)
    {
        _handlers[commandName] = handler ?? throw new ArgumentNullException(nameof(handler));
    }
}

public async Task<DialogResult?> ExecuteAsync(string commandName, Dictionary<string, object>
parameters)
{
    if (_handlers.TryGetValue(commandName, out Func<Dictionary<string, object>,
Task<DialogResult?>>? handler))
    {
        return await handler(parameters);
    }

    // ATAC'8 CÄ>CDCC'L (C00C0@C,,<CT@, DCA) C05 C0>CD:C'NDt5C0 2 C'0D4=Dt5D 5, D 8D BCT<C =CR CC00CD5D"Ä
    return null;
}
}
</file>

<file path="Configuration.Web/WebAppBootstrapper.cs">
using Microsoft.AspNetCore.Components;
{
    namespace Promatis.Net.Configuration.Web;
    {
        // A00D ;CT4C08C¢ &ö÷G7G& W"Ä :CäBCä@D'9 D4<CT5D" 7C ?D4AC¤0D$L Web-D 5D 2CT@D
        public abstract class WebAppBootstrapper<TRootComponent> : AppBootstrapper
        where TRootComponent : IComponent
        {
            public override void Run(string[] args)
            {
                // B =Cä2C 7C 3D CCd0CT< DLL (C00 D ;D4GC 9 C0@D0<Cä3Cä 7C ?D4AC¤0)
                // LoadProjectAssemblies ("Promatis.") D46CR 2D'7C$0C00 C" & 6RÄ'VâÄ
                // C0> Ct4CTADÄ <D² 8D ?Cä;DÄ7D45CÄ vV$ Ä-6 F-öä'V-ÆFW"i

                // Ää Bô A "AT BÄ Ää¢ CT7 D0BCä3Cä F0Ö -â =CR CC$8CD8D" <Cä4D4;C, ?D 8 Type.GetType
                LoadProjectAssemblies("Promatis.");

                WebApplicationBuilder builder = WebApplication.CreateBuilder(args);
                List<IAppConfigurator> configs = GetConfigurators(builder.Configuration).ToList();

                foreach (IAppConfigurator cfg in configs)
                {
                    cfg.ConfigureServices(builder.Services, builder.Configuration);
                }

                WebApplication app = builder.Build();

                // A00D BD >C":C Ö-FFÆWv &R <Cä4D4;CT9D
                foreach (IWebAppConfigurator cfg in configs.OfType<IWebAppConfigurator>())
                {
                    cfg.ConfigureMiddleware(app);
                }

                foreach (IAppConfigurator cfg in configs)
                {
                    cfg.ConfigureApp(app);
                }

                app.Run();
            }
        }
    }
}
</file>

<file path="Configuration/AppBootstrapper.cs">
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.Hosting;
using System.Reflection;
using System.Runtime.Loader;
{
    namespace Promatis.Net.Configuration;
    {
        public abstract class AppBootstrapper
        {
            public virtual void Run(string[] args)
            {
                // 1. A0 A,, B4 A,, "AT BÄ A / At A4 B4 A¤ DLL (DtBCä1D² G- RävWEG- R 8DR 2C,,4CT;);
            }
        }
    }
}

```

```

        LoadProjectAssemblies("Promatis.");
    }
    HostApplicationBuilder builder = Host.CreateApplicationBuilder(args);

    // 2. B4 A0+A' Aä B A A0$A,, B4 A "Aä Aä A,, JSON
    List<IAppConfigurator> configurators = GetConfigurators(builder.Configuration).ToList();

    foreach (IAppConfigurator config in configurators)
    {
        config.ConfigureServices(builder.Services, builder.Configuration);
    }

    IHost host = builder.Build();

    foreach (IAppConfigurator config in configurators)
    {
        config.ConfigureApp(host);
    }

    host.Run();
}

protected virtual IEnumerable<IAppConfigurator> GetConfigurators(IConfiguration configuration)
{
    string[] typeNames = configuration.GetSection("LauncherSettings:Modules").Get<string[]>()
        ?? [];

    foreach (string name in typeNames)
    {
        // A,,ICT< D$8C0 2Cä 2D 5DR 7C 3D CCd5C0=D'E D 1Cä@C0D]
        Type? type = AppDomain.CurrentDomain.GetAssemblies()
            .Select(a => a.GetType(name))
            .FirstOrDefault(t => t != null);

        if (type != null && typeof(IAppConfigurator).IsAssignableFrom(type))
        {
            if (Activator.CreateInstance(type) is IAppConfigurator instance)
            {
                yield return instance;
            }
        }
        else
        {
            Console.WriteLine($"[BOOTSTRAPPER][WARN] B$8C0 =CR =C 9CD5C0 8C'8 C05 C$0C'8CD5C0¢ ¶æ ÖW0""½
        }
    }
}

protected void LoadProjectAssemblies(string prefix)
{
    string? path = Path.GetDirectoryName(Assembly.GetExecutingAssembly().Location);
    if (path == null) return;

    foreach (string file in Directory.GetFiles(path, $"{prefix}*.dll"))
    {
        try
        {
            AssemblyLoadContext.Default.LoadFromAssemblyPath(file);
        }
        catch { /* A,,3C0>D 8D CCT< CäHC,,1C08 Ct0C4@D47C08 */ }
    }
}
}
</file>

<file path="Configuration/AppConfigurator.cs">
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Hosting;
using Promatis.Net.Data;
using System.Text.Json;
using System.Text.Json.Serialization;

namespace Promatis.Net.Configuration;

public class AppConfigurator : IAppConfigurator

```

```

{
    public virtual void ConfigureServices(IServiceCollection services, IConfiguration configuration)
    {
        // B :C =C,,@D45CÂ 2D Q (C$:C'NDt0D0 VF-EG&-vvW" 2 C0@Că5C▯BCR F F •
        services.AddDomainInfrastructure(projectPrefix: "Promatis.");
    }

    // B 5C48D BD 8D CCT< C 0Ct>C$KCR ACT@C$8D K0
    services.AddScoped<DatabaseTriggerService>();
    services.AddScoped<IDatabaseTriggerService>(sp =>
    sp.GetRequiredService<DatabaseTriggerService>());
    services.AddScoped<DatabaseTriggerInterceptor>();

    // Aă Bd Af¢ ¥4ôâ =C AD$@Că9C▯8D
    services.AddSingleton(new JsonSerializerOptions
    {
        PropertyNamingPolicy = JsonNamingPolicy.CamelCase,
        DefaultIgnoreCondition = JsonIgnoreCondition.WhenWritingNull,
        WriteIndented = false
    });
}

public virtual void ConfigureApp(IHost app)
{
    using IServiceScope scope = app.Services.CreateScope();
    scope.ServiceProvider.AutoRegisterTriggers();
}
}
</file>

<file path="Configuration/IAppConfigurator.cs">
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Hosting;
namespace Promatis.Net.Configuration;
public interface IAppConfigurator
{
    void ConfigureServices(IServiceCollection services, IConfiguration configuration);
    void ConfigureApp(IHost app);
}
</file>

<file path="Configuration/Promatis.Net.Configuration.csproj">
<Project Sdk="Microsoft.NET.Sdk">
    <PropertyGroup>
        <TargetFramework>net10.0</TargetFramework>
        <ImplicitUsings>enable</ImplicitUsings>
        <Nullable>enable</Nullable>
    </PropertyGroup>
    <ItemGroup>
        <None Include="Assets\FV0w&÷W 1
        ' Å 6¶ vU&VfW&Væ6R -æ6ÇVFS0$0-7&÷6ögBäW†FVÇ6-öç2ä†÷7F-ær" fW'6-öä0# ä ä," óí
        Version="10.0.8" />
        ' Å 6¶ vU&VfW&Væ6R -æ6ÇVFS0$0-7&÷6ögBäW†FVÇ6-öç2ä†÷7F-ær" fW'6-öä0# ä ä," óí
        ' Å 6¶ vU&VfW&Væ6R -æ6ÇVFS0$0-7&÷6ögBäW†FVÇ6-öç2ä†÷7F-ærä '7G& 7F-öç2" fW'6-öä0# ä ä," óí
        ' Å 6¶ vU&VfW&Væ6R -æ6ÇVFS0$0-7&÷6ögBäW†FVÇ6-öç2ä†÷7F-ærä '7G& 7F-öç2" fW'6-öä0# ä ä," óí
        ' Å 6¶ vU&VfW&Væ6R -æ6ÇVFS0%67'WF÷"" fW'6-öä0#rä ä " óí
        "Ä0-FV0w&÷W 1
        </ItemGroup>
    </Project>
</file>

<file path="Configuration/ServiceScanningExtensions.cs">
using FluentValidation;

```



```

using Microsoft.Extensions.DependencyInjection;
using Promatis.Net.Data;
using Promatis.Net.Domain;
using Promatis.Net.Service;
using System.Reflection;
using System.Runtime.Loader;

namespace Promatis.Net.Configuration;

public static class ServiceScanningExtensions
{
    public static void AddDomainInfrastructure(this IServiceCollection services, string
projectPrefix = "Promatis.")
    {
        Console.WriteLine();
        Console.WriteLine("[SCANNER] At0C0CD : C 2D$>CÄ0D$8Dt5D :Cä9 D 5C48D BD 0Dd8C, 2 DI...");

        // 1. A0 A,, B4 A,, "AT BÄ A / At A4 B4 A B Aä Aä
        string? path = Path.GetDirectoryName(Assembly.GetExecutingAssembly().Location);
        if (path != null)
        {
            foreach (string file in Directory.GetFiles(path, $"{projectPrefix}*.dll"))
            {
                try
                {
                    AssemblyLoadContext.Default.LoadFromAssemblyPath(file);
                }
                catch (Exception ex)
                {
                    string fileName = Path.GetFileName(file);
                    Console.ForegroundColor = ConsoleColor.Red;
                    Console.WriteLine($"[SCANNER][ERROR] A05 D44C ;CäADÂ 7C 3D CCT8D$L D 1Cä@C=C {fileName}");
                    Console.ResetColor();
                }
            }
        }

        // A 5D 5CÂ 2D 5 Ct0C4@D46CT=C0KCR AC >D :C, A C$0D,,8CÂ AC,,AD$5CÄ=D´< C0@CTDC,,:D >Cí
        Assembly[] assemblies = AppDomain.CurrentDomain.GetAssemblies()
            .Where(a => a.FullName != null && a.FullName.StartsWith(projectPrefix))
            .Distinct()
            .ToArray();

        int countBefore = services.Count;

        // 2. A B$ AÄ B$ Bt B Aä B A A,, Aä A A,, A, AT A,,!B$ A &A,,/ Bt B Ar 45%UDö-
        services.Scan(scan =>
        {
            // A´>C=C0C´LC00D0 DD4=C=FC,,0 CD;D0 @CT:D4@D 8C$=Cä9 C0@Cä2CT@C=8 C$ACT9 Dd5C0>Dt:C, =C AC´5CD>C$0
            bool IsSubclassOfRawGeneric(Type generic, Type? toCheck)
            {
                while (toCheck != null && toCheck != typeof(object))
                {
                    Type cur = toCheck.IsGenericType ? toCheck.GetGenericTypeDefinition() : toCheck;
                    if (generic == cur) return true;
                    toCheck = toCheck.BaseType;
                }
                return false;
            }

            scan.FromAssemblies(assemblies)
                // A,,!A0 A A´ A0 : B 5C48D BD 0Dd8D0 !AT A$ B A" A C0>CD4CT@Cd:Cä9 C4;D41Cä:Cä3Cä =C AC´5CD
                .AddClasses(classes => classes.Where(type =>
                    !type.IsAbstract &&
                    !type.IsGenericTypeDefinition &&
                    IsSubclassOfRawGeneric(typeof(BaseService<,,>), type)))
                .AsSelf()
                .AsImplementedInterfaces()
                .WithScopedLifetime()

                // B 5C48D BD 0Dd8D0 A A,, A "Aä Aä (FluentValidation)
                .AddClasses(c => c.AssignableTo(typeof(IValidator<>)))
                .Where(t => !t.IsAbstract && !t.IsGenericTypeDefinition)
                .AsImplementedInterfaces()
                .WithTransientLifetime()
        });
    }
}

```

```

        // B 5C48D BD 0Dd8D0 "B A4 AT Aä (Before/After Save)D
        .AddClasses(c => c.AssignableToAny(typeof(IBeforeSaveTrigger<>),
typeof(IAfterSaveTrigger<>)).Where(t => !t.IsAbstract))D
        .AsSelfWithInterfaces()D
        .WithScopedLifetime();D
    });D
D
    // --- B A4 B "B Bd B0 A´ A A´,A0 A4 A0 A´ AÄ B $A0 A4 A$ A´ AD B$ B B0 B ---D
    // B 5C48D BD 8D CCT< C00C¢ >D$:D KD$K¢´ 4Cd5C05D 8C¢â "CT?CT@DÄ ?D 8 Ct0C0@CäACR •f Æ-F F÷#Ä Dä1Cä9A
    // DI-C0>C0BCT9C05D ääUB 3C @C =D$8D >C$0C0=Cä >D$4C AD" =C H C0>C´8CÄ>D DC0KC´ 4C,,AC05D$GCT@D
    services.AddScoped(typeof(IValidator<>), typeof(GlobalPolymorphicValidator<>));D
D
    // 3. A,,!A0 A A´ A0 Aä A, A AT A0 AR Aä A,, Aä A A,, B At#A´,B$ B$ A-
    List<ServiceDescriptor> newRegistrations = services.Skip(countBefore).ToList();D
    var loggedTypes = new HashSet<string>();D
D
    foreach (ServiceDescriptor reg in newRegistrations)D
    {D
        // A 5D 5CÄ @CT0C´LC0KC´ BC,,? D 5C ;C,,7C FC,,8 (C" ?D 8Cä@C,,BCTBCR 8Cr -x ÆVÖVçF F-öâG- RÄ 8C00Dt5
        Type? implType = reg.ImplementationType ?? reg.ImplementationInstance?.GetType();D
D
        if (implType == null) continue;D
D
        // --- A A A,, A$ A´ AD B$ B A" 00Ý
        bool isValidator = reg.ServiceType.IsGenericType &&
            reg.ServiceType.GetGenericTypeDefinition() == typeof(IValidator<>);D
D
        if (isValidator && loggedTypes.Add($"Val:{implType.FullName}"))D
        {D
            var inheritanceChain = new List<string>();D
            Type? currentType = implType;D
D
            while (currentType != null && currentType != typeof(object))D
            {D
                string typeName = currentType.Name.Contains('.') ? currentType.Name.Split('.')[0] : currentType.Name;D
                inheritanceChain.Add(typeName);D
                currentType = currentType.BaseType;D
                if (typeName == "AbstractValidator") break;D
            }D
D
            string chainDisplay = string.Join(" -> ", inheritanceChain);D
            Console.WriteLine($"[SCANNER] A$0C´8CD0D$>D ¢ ¶6† -äF-7 Æ -0 ¢ •f Æ-F F÷""½
            continue; // A´>C2 4C´0 D0BCä9 Ct0C08D 8 C$K¢$5CD5C0Ä 8CD5CÄ : D ;CT4D4ND"5C•
        }D
D
        // --- A A A,, B B A,,!Aä (A00D ;CT4C08C0>C" & 6U6W'f-6R´ 00Ý
        // A,,!A0 A A´ A0 : B41D 0C0> Cd5D BC0>CR CD ;Cä2C,,5 reg.ServiceType == reg.ImplementationType, D
        // D$0C¢ :C : D 5D 2C,,AD² BCT?CT@DÄ @CT3C,,AD$@C,,@D4ND$AD0 ?C @C <C, C´0D A + A,,=D$5D DCT9D
        bool isService = false;D
        var serviceChain = new List<string>();D
        Type? currentServiceType = implType;D
D
        while (currentServiceType != null && currentServiceType != typeof(object))D
        {D
            string typeName = currentServiceType.Name.Contains('.') ?
currentServiceType.Name.Split('.')[0] : currentServiceType.Name;D
            serviceChain.Add(typeName);D
D
            if (currentServiceType.IsGenericType &&
currentServiceType.GetGenericTypeDefinition() == typeof(BaseService<,,>))D
            {D
                isService = true;D
                break;D
            }D
            currentServiceType = currentServiceType.BaseType;D
        }D
D
        if (isService && loggedTypes.Add($"Srv:{implType.FullName}"))D
        {D
            string chainDisplay = string.Join(" -> ", serviceChain);D
            Console.WriteLine($"[SCANNER] B 5D 2C,,A: {chainDisplay}");D
            continue;D
        }D
D
        // --- A A A,, B$ A,, A4 B A" 00Ý

```

```

        bool isTrigger = implType.GetInterfaces().Any(i => i.IsGenericType &&
            (i.GetGenericTypeDefinition() == typeof(IBeforeSaveTrigger<>) ||
            i.GetGenericTypeDefinition() == typeof(IAfterSaveTrigger<>)));

        if (isTrigger && loggedTypes.Add($"Tri:{implType.FullName}"))
        {
            Console.WriteLine($"[SCANNER] B${C,,3C45D} ¢ ¤-× ÅG- Ræë ÖW0""½
        }

        Console.WriteLine("[SCANNER] A 2D$>CÄ0D$8Dt5D :C 0 D 5C48D BD 0Dd8Dò CD ?CTHCÔ> Ct0C$5D HCT=C â""½
        Console.WriteLine();

        public static void AutoRegisterTriggers(this IServiceProvider serviceProvider)
        {
            Console.WriteLine("[AUTOREG] At0C0CD : C 2D$>CÄ0D$8Dt5D :Cä9 D 5C48D BD 0Dd8C, BD 8C43CT@Cä2 C" F F &

            var triggerService = serviceProvider.GetRequiredService<IDatabaseTriggerService>();

            // 1. A0>C,,AC¢ 2D 5DR :C´0D ACä2 D$@C,,3C45D >C" 2 D 1Cä@C¢0DR &öÖ F-2â-
            IEnumerable<Type> triggerTypes = AppDomain.CurrentDomain.GetAssemblies()
                .Where(a => a.FullName?.StartsWith("Promatis.") == true)
                .SelectMany(s => s.GetTypes())
                .Where(t => t is { IsClass: true, IsAbstract: false } &&
                    t.GetInterfaces().Any(i => i.IsGenericType &&
                        (i.GetGenericTypeDefinition() ==
                            typeof(IBeforeSaveTrigger<>) ||
                            typeof(IAfterSaveTrigger<>))))
                .ToList();

            MethodInfo? registerMethod =
                typeof(IDatabaseTriggerService).GetMethod(nameof(IDatabaseTriggerService.Register));

            int totalBindings = 0;

            foreach (Type triggerType in triggerTypes)
            {
                // A00DT>CD8CÄ 2D 5 C,,=D$5D DCT9D K D$@C,,3C45D >C"Â :CäBCä@D´5 D 5C ;C,,7D45D" 4C =C0KC´ :C´0D A0
                IEnumerable<Type> interfaces = triggerType.GetInterfaces()
                    .Where(i => i.IsGenericType &&
                        (i.GetGenericTypeDefinition() == typeof(IBeforeSaveTrigger<>) ||
                        i.GetGenericTypeDefinition() == typeof(IAfterSaveTrigger<>)));

                foreach (Type @interface in interfaces)
                {
                    // A0>C´CDt0CT< D$8Cò AD4IC0>D BC, ...B 8Cr "&Vf÷&U6 fUG&-vvW#ÂCâ•
                    Type entityType = @interface.GetGenericArguments()[0];

                    // Aä?D 5CD5C´0CT< D$8Cò BD 8C43CT@C 4C´O C¢@C AC,,2Cä3Cä ;Cä3C
                    string triggerKind = @interface.GetGenericTypeDefinition() ==
                        typeof(IBeforeSaveTrigger<>)
                        ? "Before"
                        : "After ";

                    try
                    {
                        // A$KctKC$0CT< Register<TEntity, TTrigger>()
                        MethodInfo genericRegister = registerMethod!.MakeGenericMethod(entityType,
                            triggerType);

                        genericRegister.Invoke(triggerService, null);

                        // A´ A4 B A$ A0 AR #B AT(A0 A´ B A$/At A•
                        Console.WriteLine($"[AUTOREG] [{triggerKind}] {entityType.Name} <---
                        {triggerType.Name}");
                        totalBindings++;
                    }
                    catch (Exception ex)
                    {
                        Console.ForegroundColor = ConsoleColor.Red;
                        Console.WriteLine($"[AUTOREG][ERROR] AäHC,,1C¢0 C0@C,,2D07C¢8 {triggerType.Name} C¢ ¤VçF-G•
                        {ex.InnerException?.Message ?? ex.Message}");
                        Console.ResetColor();
                    }
                }
            }
        }

```

```

    }
    Console.WriteLine($"[AUTOREG] At0C$5D HCT=Cââ !Că7CD0C0> C0@C,,2D07Că:: {totalBindings}");
    Console.WriteLine();
}
}
</file>

<file path="Data/Configurations/AuditLogConfiguration.cs">
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata.Builders;
using Promatis.Net.Domain;
namespace Promatis.Net.Data;
public class AuditLogConfiguration : IEntityTypeConfiguration<AuditLog>
{
    public void Configure(EntityTypeBuilder<AuditLog> builder)
    {
        builder.Property(e => e.EntityName).HasMaxLength(100);
        builder.Property(e => e.Action).HasMaxLength(50);
        builder.Property(e => e.ChangedBy).HasMaxLength(100);

        // ChangesJson CăAD$0C$;D05CĂ 1CT7 C`8CĂ8D$0 (text/jsonb), D$0C$ :C : D ?C,,ACă: C,,7CĂ5C05C08C' <Că6CT
        builder.Property(e => e.ChangesJson).HasColumnType("jsonb");
    }
}
</file>

<file path="Data/Configurations/ModelBuilderConventions.cs">
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata;
using System.Linq.Expressions;
using Promatis.Net.Domain.Interface;
namespace Promatis.Net.Data;
public static class ModelBuilderConventions
{
    public static void ApplyGlobalConventions(this ModelBuilder modelBuilder)
    {
        foreach (IMutableEntityType entityType in modelBuilder.Model.GetEntityTypes())
        {
            Type type = entityType.ClrType;

            // A`8CĂ8D$K CD;D0 AD$@Că: (255 C0> D4<Că;Dt0C08Dă•
            foreach (IMutableProperty property in entityType.GetProperties())
            {
                if (property.ClrType == typeof(string))
                {
                    if (property.GetMaxLength() == null)
                        property.SetMaxLength(255);
                }
            }

            // A 2D$>-Că;DăGC, 4C`O Guid (ValueGeneratedNever, D$0C$ :C : Id D >Ct4C 5D$AD0 2 DomainObject)
            if (typeof(IDomainObjectHasKey<Guid>).IsAssignableFrom(type))
            {
                modelBuilder.Entity(type).Property("Id").ValueGeneratedNever();
            }

            // A B$ -BD A`,B$ C, A$"Aăô Aô AT B 4C`O SoftDelete Că1D=5C=BCă2D
            if (typeof(ISoftDeletable).IsAssignableFrom(type))
            {
                IMutableEntityType? baseType = entityType.BaseType;

                // B BC 2C,,< DD8C`LD$@ D$>C`LCă> C00 Că>D 5C0L C,,5D 0D EC,,8 (C$:C`NDt0D0 0C AD$@C :D$=D`9 Uni
                if (baseType == null || !typeof(ISoftDeletable).IsAssignableFrom(baseType.ClrType))
                {
                    // B4AD$0C0>C$:C VW`"f-ÇFW-
                    modelBuilder.SetSoftDeleteFilter(type);

                    // B4AD$0C0>C$:C 8C04CT:D 0 C00 DeletedAt CD;D0 CD :Că@CT=C,,0 Ct0C0@CăACă2D
                    modelBuilder.Entity(type).HasIndex("DeletedAt");
                }
            }
        }
    }
}

```

```

    }
}

private static void SetSoftDeleteFilter(this ModelBuilder modelBuilder, Type entityType)
{
    // AD;Dò E BÖE , 2C 6CÔ> D BC 2C„BDÂ DC„;DÄBD =C :Cä@CT=DÂÂ 4C 6CR 5D ;C, >Cò 0C AD$@C :D$=D´9.D
    // B4AC´>C$8CR -46Æ 72 4CäAD$0D$>Dt=Câí
    if (entityType.IsClass)
    {
        modelBuilder.Entity(entityType).HasQueryFilter(GenerateFilter(entityType));
    }
}

private static LambdaExpression GenerateFilter(Type type)
{
    ParameterExpression parameter = Expression.Parameter(type, "e");
    // A„ACô>C´LctCCT< Property CD;Dò 4CäAD$CCô0 C¢ FVÆWFVD B GCT@CT7 C„=D$5D DCT9D 8C´8 D 2Cä9D BC$>D
    MemberExpression property = Expression.Property(parameter,
nameof(ISoftDeletable.DeletedAt));
    ConstantExpression nullConstant = Expression.Constant(null, typeof(DateTime?));
    BinaryExpression body = Expression.Equal(property, nullConstant);

    return Expression.Lambda(body, parameter);
}
}
</file>

<file path="Data/DbContext/ApplicationDbContext.cs">
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Promatis.Net.Domain;

namespace Promatis.Net.Data;

public class ApplicationDbContext(
    DbContextOptions options,
    IConfiguration configuration,
    IServiceProvider? serviceProvider = null)
    : DbContext(options)
{
    protected virtual string Schema => "public";
    public DbSet<AuditLog> AuditLogs => Set<AuditLog>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.HasDefaultSchema(Schema);
        modelBuilder.ApplyModuleConfigurations(this);
        modelBuilder.ApplyGlobalConventions();
        base.OnModelCreating(modelBuilder);
    }

    // A„!Aô A A´ Aô B´ A, A AT Aô+A´ A A„ Aô"D
    protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
    {
        base.OnConfiguring(optionsBuilder);

        // 1. Aô@Cä2CT@Dô5CÂÂ GD$> CÄK C" @C =D$0C"<CRÂ 0 Cô5 C" ?D >Dd5D ACR <C„3D 0Dd8C, 4Ä•
        if (!EF.IsDesignTime && serviceProvider != null)
        {
            DatabaseTriggerInterceptor? interceptor = null;

            try
            {
                // Aô>CôKDS:C ¢ D >C CCT< D 0Ct@CTHC„BDÂ =C ?D OCÄCdâ „5D ;C, ?D >C$0C"4CT@ Cä:C 7C ;D O S
                interceptor = serviceProvider.GetService(typeof(DatabaseTriggerInterceptor)) as
DatabaseTriggerInterceptor;
            }
            catch (InvalidOperationException)
            {
                // Aô>CôKDS:C #¢ D ;C, ?D >C$0C"4CT@ C¤>D =CT2Cä9 (Root), D >Ct4C 5CÂ 66÷ R 2D Cdt=D4N!D
                // BôBCä 3C @C =D$8D CCTB, DtBCä 66÷ VBô8CôBCT@Dd5CôBCä@ D 0Ct@CTHC„BD O C 5Cr >D„8C >C¢ > Ro
                var scopeFactory = serviceProvider.GetService(typeof(IServiceScopeFactory)) as
IServiceScopeFactory;

                if (scopeFactory != null)

```

```

        {
            // B >Ct4C 5CÂ 2D 5CÂ5CÔ=D4N Că1C'0D BDÂ 2C,,4C,,<CăAD$8
            using IServiceScope scope = scopeFactory.CreateScope();
            interceptor =
scope.ServiceProvider.GetService(typeof(DatabaseTriggerInterceptor)) as DatabaseTriggerInterceptor;
        }
    }
}

// ATAC'8 C,,=D$5D FCT?D$>D CD ?CTHCÔ> CÔ0C"4CT= Că4CÔ8CÂ 8Cr ACô>D >C >C" r ?Că4C;DăGC 5CÂ 5C4>
if (interceptor != null)
{
    optionsBuilder.AddInterceptors(interceptor);
}
}
}
}
</file>

<file path="Data/DbContext/ModelBuilderExtensions.cs">
using Microsoft.EntityFrameworkCore;
using Promatis.Net.Domain.Interface;
using System.Reflection;
using Microsoft.EntityFrameworkCore.Metadata;
using Microsoft.EntityFrameworkCore.Metadata.Builders;
namespace Promatis.Net.Data;
public static class ModelBuilderExtensions
{
    public static void ApplyModuleConfigurations(this ModelBuilder modelBuilder, DbContext context)
    {
        // 1. B :C =C,,@D45CÂ BCă;DĂ:Că AC >D :C, 4C =CÔKDR ?D >CT:D$0 Promatis.*.Data
        List<Assembly> dataAssemblies = AppDomain.CurrentDomain.GetAssemblies()
            .Where(a => {
                string? name = a.GetName().Name;
                return name != null &&
                    name.StartsWith("Promatis.") &&
                    name.EndsWith(".Data");
            }).ToList();

        // 2. Aô>C'CDt0CT< D$8CôK D CD"=CăAD$5C'Â :CăBCă@D'5 Dô2CÔ> Că1D=0C$;CT=D² 2 D$5C=CD"5CÂ :Că=D$5C=AD$
        HashSet<Type> contextDbSetTypes = context.GetType()
            .GetProperties(BindingFlags.Public | BindingFlags.Instance)
            .Where(p => p.PropertyType.IsGenericType && p.PropertyType.GetGenericTypeDefinition()
== typeof(DbSet<>))
            .Select(p => p.PropertyType.GetGenericArguments()[0])
            .ToHashSet();

        foreach (Assembly assembly in dataAssemblies)
        {
            // Aô@C,,<CT=Dô5CÂ :Că=DD8C4CD 0Dd8C, BCă;DĂ:Că 4C'0 D$5DR BC,,?Că2, C= >D$>D KCR <C BCT@C,,0C'LCÔK (
            modelBuilder.ApplyConfigurationsFromAssembly(assembly, type =>
            {
                if (type is not { IsClass: true, IsAbstract: false, IsGenericTypeDefinition:
false })
                return false;

                // Aô0DT>CD8CÂ 8CÔBCT@DD5C"A C= >CÔDC,,3D4@C FC,,8 C, BC,,? D CD"=CăAD$8, C= >D$>D CDâ >CÔ0 CÔ0D B
                Type? configInterface = type.GetInterfaces()
                    .FirstOrDefault(i => i.IsGenericType && i.GetGenericTypeDefinition() ==
typeof(IEntityTypeConfiguration<>));

                if (configInterface == null) return false;

                Type entityType = configInterface.GetGenericArguments()[0];

                // A= A,, "A,, 'AT!A= A' $A,, BĂ"B   D ;C, =C AD$@C 8C$0CT<C 0 D CD"=CăAD$L C 1D BD 0C=BCÔ0,
                // Cô@Că2CT@Dô5CÂ 5D BDÂ ;C, C CÔ5E ECăBDÂ >CD8CÔ 6C,,2Că9 CÔ0D ;CT4CÔ8C 2 D$5C=CD"5CÂ F%6W
                if (entityType.IsAbstract)
                {
                    bool hasConcreteDerivedInDbSet = contextDbSetTypes.Any(t =>
t.IsSubclassOf(entityType));
                    if (!hasConcreteDerivedInDbSet) return false; // Aô@Că?D4AC=0CT< C= >CÔDC,,3D4@C FC,,N C 1D
                }

                return type.GetConstructor(Type.EmptyTypes) != null;
            }
        }
    }
}

```

```

        });
    }
}

// 3. A B$ -A,, AÔ B B A$ AÔ AR ´ B B B$ A$(A,,%» A B "B A»&A,, Ð
IEnumerable<Type> allAbstractEntities = dataAssemblies
    .SelectMany(a => a.GetTypes())
    .Where(t => t is { IsClass: true, IsAbstract: true } && !t.IsGenericTypeDefinition);

foreach (Type abstractType in allAbstractEntities)
{
    bool isUsedInCurrentContext = contextDbSetTypes.Any(t => t == abstractType ||
t.IsSubclassOf(abstractType));

    if (!isUsedInCurrentContext)
    {
        modelBuilder.Ignore(abstractType);
    }
}

// 4. A B$ AÄ B$ Bt B A / AÔ B "B A" A At A´ B A$ AÔ B´% AD B A$,AT A, AÔ AT B A-
foreach (IMutableEntityType entityType in modelBuilder.Model.GetEntityTypes())
{
    Type? clrType = entityType.ClrType;
    if (clrType == null) continue;

    // A,,ICT< D 5C ;C,,7C FC,,N C,,=D$5D DCT9D 0 ITreeNode<>Ð
    Type? treeInterface = clrType.GetInterfaces()
        .FirstOrDefault(i => i.IsGenericType && i.GetGenericTypeDefinition() ==
typeof(INode));

    if (treeInterface != null)
    {
        Type targetTreeType = treeInterface.GetGenericArguments()[0];

        // AÔ0D BD 0C,,2C 5CÂ BCä;DÄ:Cä =C 1C 7Cä2Cä< D4@Cä2CÔ5 Cä1D=0C$;CT=C,,O D 2Cä9D BC" „GD$>C K
        if (clrType == targetTreeType)
        {
            EntityTypeBuilder builder = modelBuilder.Entity(clrType);

            // 1. AÔ0D BD >C":C AC$0Ct8 B >CD8D$5C´L-Aô>D$>CÄ>C-
            builder.HasOne("Parent")
                .WithMany("Children")
                .HasForeignKey("ParentId")
                .OnDelete(DeleteBehavior.Restrict);

            // 2. A B$ -A,, AD A»!: B >Ct4C 5CÂ 8CÔ4CT:D 4C´O C$=CTHCÔ5C4> C»;DäGC &VçD-M
            // BÔBCâ CD :Cä@C,,B Cô>C,,AC¢ MC´5CÄ5CÔBCä2 Cô> ParentId Cô@C, @C 1CäBCR "Æöö·W 2 D 5D 2C
            builder.HasIndex("ParentId")
                .HasDatabaseName($"IX_{clrType.Name}_ParentId");
        }
    }
}
}
</file>

<file path="Data/Interceptors/ChangeEntryModel.cs">
namespace Promatis.Net.Data;
{
    /// <summary>Ð
    /// AÄ>CD5C´L DD8C=AC FC,,8 C,,7CÄ5CÔ5CÔ8C´ „BD 0CÔACô>D BCÔKC´ >C JCT:D"´Â ?D 5CDAD$0C$;DôND"0Dò ACÔ8CÄ>C¢ ACä
    /// CD>CÄ5CÔ=Cä9 D CD"=CäAD$8 C" <Cä<CT=D" 5E 4Cä1C 2C´5CÔ8DòÄ <Cä4C,,DC,,:C FC,,8 C,,;C, CCD0C´5CÔ8Dòí
    /// </summary>Ð
    internal class ChangeEntryModelÐ
    {
        /// <summary>Ð
        /// A,,7CÄ5CÔ0CT<D´9 DÔ:Ct5CÄ?C´OD 4Cä<CT=CÔ>C4> Cä1D=5C=BC „AD4ICÔ>D BC,´i
        /// Aô@C,,2Cä4C,,BD O C¢ :Cä=C»@CTBCÔ>CÄ D$8CôC C$=D4BD 8 D$@C,,3C45D >C"í
        /// </summary>Ð
        public object Entity { get; init; } = null!;

        /// <summary>Ð
        /// B$5C=CD"8C´ AD$0D$CD 8Ct<CT=CT=C,,O D CD"=CäAD$8 C" AC,,AD$5CÄ5 (Added, Modified, Deleted, SoftDeleted)
        /// </summary>Ð
        public EntityStateChangeEnum State { get; init; }
    }
}

```

```

    /// <summary>ð
    /// Að>C´;CT:Dd8Dð BCäGCTGCÔKDR 8Ct<CT=CT=C,,9 D 2Cä9D BC" „?Cä;CT9) D CD"=CäAD$8.ð
    /// At0C0>C´=Dð5D$ADð 4CT;DÄBCä9 Ct=C GCT=C,,9 "B BC @Cä5 -> A0>C$>CR"ä C´O C0>C$KDR AD4IC0>D BCT9 D >CD5
    /// </summary>ð
    public List<PropertyChangeInfo> Changes { get; init; } = [];ð
ð
    /// <summary>ð
    /// A,,4CT=D$8DD8Cð0D$>D „8CÄO C,,;C, ;Cä3C,,=) Cð>C´Lct>C$0D$5C´O, D >C$5D HC,,2D,,5C4> CD0C0=Cä5 CD5C"AD$2C
    /// </summary>ð
    public string? ChangedBy { get; init; }ð
ð
    /// <summary>ð
    /// AD0D$0 C, 2D 5CÄO DD8CðAC FC,,8 C,,7CÄ5C05C08Dð 2 DD>D <C BCR UD2i
    /// </summary>ð
    public DateTime ChangedAt { get; init; }ð
}
</file>

<file path="Data/Interceptors/DatabaseTriggerInterceptor.cs">
using System.Collections.Concurrent;ð
using Microsoft.EntityFrameworkCore;ð
using Microsoft.EntityFrameworkCore.ChangeTracking;ð
using Microsoft.EntityFrameworkCore.Diagnostics;ð
using Microsoft.Extensions.DependencyInjection;ð
using Promatis.Net.Domain.Interface;ð
ð
namespace Promatis.Net.Data;ð
ð
    /// <summary>ð
    /// A05D 5DT2C Bdt8Cð >C05D 0Dd8C´ F$6öçFW†BÄ >C 5D ?CTGC,,2C ND"8C´ @C 1CäBD2 $<D03Cð>C4> D44C ;CT=C,,0" (Soft
    /// C0@CT4C$0D 8D$5C´LC0CDä 2C ;C,,4C FC,,N C,,7CÄ5C05C08C´ 8 D :C$>Ct=Cä5 D$@C,,3C45D =Cä5 D42CT4Cä<C´5C08CR AC,,
    /// </summary>ð
    /// <param name="scopeFactory">BD0C @C,,:C 4C´O D >Ct4C =C,,0 C,,7Cä;C,,@Cä2C =C0KDR >C ;C AD$5C´ 2C,,4C,,<CäAD$8
    public class DatabaseTriggerInterceptor(IServiceScopeFactory scopeFactory) : SaveChangesInterceptorð
    {ð
        /// <summary>ð
        /// A0>D$>Cð>C 5Ct>C00D =Cä5 DT@C =C,,;C,,ICR 7C DC,,:D 8D >C$0C0=D´E C,,7CÄ5C05C08C´ 2 D 0CÄ:C E D$5CðCD"5C´
        /// Að;DäG – D4=C,,:C ;DÄ=D´9 C,,4CT=D$8DD8Cð0D$>D MCð7CT<C0;D0@C F$6öçFW†Bi
        /// </summary>ð
        private readonly ConcurrentDictionary<Guid, List<ChangeEntryModel>> _capturedChanges = new();ð
ð
        /// <summary>ð
        /// A AC,,=DT@Cä=C0KC´ ?CT@CTEC$0D$GC,,:, C$KC0>C´=D05CÄKC´ Aä DC :D$8Dt5D :Cä3Cä ACäED 0C05C08Dð 8Ct<CT=C
        /// AäBC$5Dt0CTB Ct0 C´>C48CðC Soft Delete, DD8CðAC FC,,N D =C,,<Cð0 C,,7CÄ5C05C08C´ 8 Ct0C0CD : D$@C,,3C45D
        /// </summary>ð
        public override async ValueTask<InterceptionResult<int>> SavingChangesAsync(ð
            DbContextEventData eventData, InterceptionResult<int> result, CancellationToken ct =
default)ð
        {ð
            if (eventData.Context == null) return result;ð
ð
            // B >Ct4C 5CÄ ;Cä:C ;DÄ=D´9 C40D 0C0BC,,@Cä2C =C0> Cd8C$>C´ 66÷ R 4C´O C0@CT4CäBC$@C ICT=C,,0 C0@Cä1C´
            using IServiceScope scope = scopeFactory.CreateScope();ð
            var triggerService = scope.ServiceProvider.GetRequiredService<IDatabaseTriggerService>();ð
ð
            string? userName = await GetUserNameInternalAsync(scope.ServiceProvider);ð
ð
            // 1. A0@CT4C$0D 8D$5C´LC00Dð >C @C 1CäBCð0 Soft Delete (AÄ0C4:Cä5 D44C ;CT=C,,5)ð
            // A00DT>CD8CÄ 2D 5 D CD"=CäAD$8 D 8C0BCT@DD5C"ACä< ISoftDeletable, D2 :CäBCä@D´E D BC BD4A "B44C ;C
            IEnumerable<EntityEntry<ISoftDeletable>> entries =
            eventData.Context.ChangeTracker.Entries<ISoftDeletable>()ð
                .Where(e => e.State == EntityState.Deleted);ð
ð
            foreach (EntityEntry<ISoftDeletable> entry in entries)ð
            {ð
                // A0>CD<CT=D05CÄ DC,,7C,,GCTACð>CR CCD0C´5C08CR =C >C =Cä2C´5C08CR 4C =C0KDR 2 A ð
                entry.State = EntityState.Modified;ð
                entry.Entity.DeletedAt = DateTime.UtcNow;ð
                entry.Entity.DeletedBy = userName;ð
            }ð
ð
            // A0@C,,=D44C,,BCT;DÄ=Cä 7C AD$0C$;D05CÄ Tb 6÷&R ?CT@CTAdt8D$0D$L CD5D 5C$> C,,7CÄ5C05C08C´ ?CäAC´5 C0>
            eventData.Context.ChangeTracker.DetectChanges();ð
ð
            // 2. At0DT2C B C,,7CÄ5C05C08C´ „ACäED 0C00CTB C00D$8C$=D´5 D$8C0K CD0C0=D´E object? CD;D0 BCTAD$>C"•
            List<ChangeEntryModel> captured = CaptureChanges(eventData.Context, userName);ð

```



```

D
    // A0C,,2D07D'2C 5CÂ :Cä;C'5C=FC,,N C,,7CÄ5C05C08C' : C=CO:D 5D$=Cä<D2 MC=7CT<C0;D0@D2 :Cä=D$5C=AD$0 C
    _capturedChanges[eventData.Context.ContextId.InstanceId] = captured;D
D
    // 3. A$0C'8CD0Dd8D0 „&Vf÷&U6 fR' GCT@CT7 Cd8C$>C' G&-vvW%6W'f-6]
    // At0C0CD :C 5D" 4Cä<CT=C0KCR ?D 0C$8C'0 C, 2C ;C,,4C BCä@D² ?CT@CT4 D$5CÂÂ :C : CD0C0=D'5 C0>C00CDCD
    foreach (ChangeEntryModel item in captured)D
        await triggerService.ValidateAsync(item.Entity, item.State, item.Changes,
eventData.Context);D
D
    return result;D
}D
D
    /// <summary>D
    /// A AC,,=DT@Cä=C0KC' ?CT@CTEC$0D$GC,,:, C$KC0>C'=D05CÄKC' Aä!A' D4AC05D,,=Cä3Cä ACäED 0C05C08D0 8Ct<CT=C
    /// AäBC$5Dt0CTB Ct0 D42CT4Cä<C'5C08CR ?Cä4C08D GC,,:Cä2 C, BD 8C43CT@Cä2 C0>D B-Cä1D 0C >D$:C, „DC 7C 6
    /// </summary>D
    public override async ValueTask<int> SavedChangesAsync(D
        SaveChangesCompletedEventData eventData, int result, CancellationToken ct = default)D
    {D
        // A,,7C$;CT:C 5CÂ 7C EC$0Dt5C0=D'5 C,,7CÄ5C05C08D0 8 D @C 7D2 0D$>CÄ0D =Cä CCD0C'0CT< C,,E C,,7 D ;Cä2C
        if (eventData.Context != null &&
        _capturedChanges.TryRemove(eventData.Context.ContextId.InstanceId, out List<ChangeEntryModel>?
entries))D
        {D
            // B >Ct4C 5CÂ ;Cä:C ;DÄ=D'9 C40D 0C0BC,,@Cä2C =C0> Cd8C$>C' 66÷ R 4C'0 DD0CtK Saved (C$KC0>C'=D05
            using IServiceScope scope = scopeFactory.CreateScope();D
            var triggerService =
scope.ServiceProvider.GetRequiredService<IDatabaseTriggerService>();D
D
            // A AC,,=DT@Cä=C0> D42CT4Cä<C'0CT< D 8D BCT<D2 >C 8Ct<CT=CT=C,,0DR ,,=C ?D 8CÄ5D Ä 4C'0 Cä1C0>C$;C
            foreach (ChangeEntryModel item in entries)D
                await triggerService.NotifyAsync(item.Entity, item.State, item.Changes,
item.ChangedBy, item.ChangedAt);D
            }D
            return result;D
        }D
    }D
D
    /// <summary>D
    /// A05D 5DT2C BDt8C0 AC >D0 ?D 8 D >DT@C =CT=C,,8 C,,7CÄ5C05C08C'â C @C =D$8D CCTB CäGC,,AD$:D2 :D0HC 7C
    /// C0@CT4CäBC$@C IC 0 D4BCTGC=8 C00CÄ0D$8 C0@C, ?C 4CT=C,,8 D$@C =Ct0C=FC,,9.D
    /// </summary>D
    public override Task SaveChangesFailedAsync(DbContextErrorEventData eventData,
CancellationToken ct = default)D
    {D
        if (eventData.Context != null)D
            _capturedChanges.TryRemove(eventData.Context.ContextId.InstanceId, out _);D
D
        return base.SaveChangesFailedAsync(eventData, ct);D
    }D
D
    /// <summary>D
    /// A$=D4BD 5C0=C,,9 CÄ5D$>CB 0C00C'8Ct0 D$@CT:CT@C 8Ct<CT=CT=C,,9 EF Core. D
    /// A$KDt8D ;D05D" 4CT;DÄBD2 „AD$0D KCR0=Cä2D'5 Ct=C GCT=C,,0 C0>C'5C'' 8 DD>D <C,,@D45D" <Cä4CT;C, 8Ct<CT=
    /// </summary>D
    /// <param name="context">B$5C=CD"8C' :Cä=D$5C=AD" 1C 7D² 4C =C0KDRäÄ÷ & Óí
    /// <param name="userName">A,,<D0 ?Cä;DÄ7Cä2C BCT;D0Ä ACä2CT@D,,8C$HCT3Cä >C05D 0Dd8DäãÄ÷ & Óí
    /// <returns>B ?C,,ACä: D BD CC=BD4@C,,@Cä2C =C0KDR <Cä4CT;CT9 C,,7CÄ5C05C08C' Ç6VR 7&Vc0$6† ævTVÇG'"ÖöFVÄ"ó
    private List<ChangeEntryModel> CaptureChanges(DbContext context, string? userName)D
    {D
        // BD8C'LD$@D45CÄ BCä;DÄ:Cä AD4IC0>D BC, 2 D >D BCä0C08D0E AD>C 0C$;CT=C,,0, A,,7CÄ5C05C08D0 8C'8 B44C
        List<EntityEntry> entries = context.ChangeTracker.Entries()D
            .Where(e => e.State is EntityState.Added or EntityState.Modified or EntityState.Deleted)D
            .Where(e => !e.Entity.GetType().Name.Contains("AuditLog"))D
            .ToList();D
D
        var result = new List<ChangeEntryModel>();D
D
        foreach (EntityEntry e in entries)D
        {D
            EntityStateChangeEnum state = MapState(e.State);D
D
            // B ?CTFC,,DC,,GC0KC' <C ?C08C03 D >D BCä0C08D0 4C'0 "CÄ0C4:Cä CCD0C'5C0=D'E" Ct0C08D 5C•
            if (e.Entity is ISoftDeletable soft)D
            {D
                PropertyEntry prop = e.Property(nameof(ISoftDeletable.DeletedAt));D

```

```

        if (prop.IsModified && soft.DeletedAt != null) state =
EntityStateChangeEnum.SoftDeleted;
    }
    List<PropertyChangeInfo> changes = new();

    // B FCT=C @C,,9 1: B CD"=CäAD$L CD>C 0C$;CT=C â D 5 D$5C=CD"8CR 7C00Dt5C08D0 AC$>C"AD$2 D GC,,BC
    if (state == EntityStateChangeEnum.Added)
    {
        changes = e.Properties
            .Select(p => new PropertyChangeInfo
            {
                PropertyName = p.Metadata.Name,
                OriginalValue = null,
                CurrentValue = p.CurrentValue
            }).ToList();
    }
    // B FCT=C @C,,9 2: B CD"=CäAD$L C,,7CÄ5C05C00 C,,;C, <D03C= D44C ;CT=C â D´GC,,AC´OCT< D 0Ct=C,,FD2
    else if (state is EntityStateChangeEnum.Modified or EntityStateChangeEnum.SoftDeleted)
    {
        PropertyValues originalValues = e.OriginalValues;

        foreach (PropertyEntry p in e.Properties.Where(p => p.IsModified))
        {
            string propertyName = p.Metadata.Name;
            object? originalValue = originalValues[propertyName];
            object? currentValue = p.CurrentValue;

            // BD8C=AC,,@D45CÄ 8Ct<CT=CT=C,,5 D$>C´LC= CTAC´8 Ct=C GCT=C,,0 CD5C"AD$2C,,BCT;DÄ=Cä @C 7C´
            if (!Equals(originalValue, currentValue))
            {
                changes.Add(new PropertyChangeInfo
                {
                    PropertyName = propertyName,
                    OriginalValue = originalValue,
                    CurrentValue = currentValue
                });
            }
        }
    }

    // AD0Cd5 CTAC´8 C= >C´;CT:Dd8D0 BCäGCTGC0KDR 8Ct<CT=CT=C,,9 C0>C´5C´ „6† ævW2´ ?D4AD$0 D
    // C,,7-Ct0 D 0C0BC 9CÄ01C,,=CD8C03C &Æ |÷"Ä <D² B B A$ Aä 4Cä1C 2C´OCT< D CD"=CäAD$L C" AC08D
    // CTAC´8 DD0C @C,,:C 7C DC,,:D 8D >C$0C´0 D >D BCäOC08CR 0öF-f-VB -D$> C40D 0C0BC,,@D45D" R 4C
    if (state == EntityStateChangeEnum.Modified && changes.Count == 0)
    {
        // B >Ct4C 5CÄ DC,,:D$8C$=D4N Ct0C08D L C,,7CÄ5C05C08D0 4C´O CD>CÄ5C0=D´E D$@C,,3C45D >C"Ä
        // DtBCä1D² =CR ;Cä<C BDÄ 2C ;C,,4C FC,,N, C0> D >DT@C =C,,BDÄ 8CÄ?D4;DÄA CD;D0 T•
        changes.Add(new PropertyChangeInfo
        {
            PropertyName = "Id",
            OriginalValue = e.Property("Id").OriginalValue,
            CurrentValue = e.Property("Id").CurrentValue
        });
    }

    result.Add(new ChangeEntryModel
    {
        Entity = e.Entity,
        State = state,
        ChangedBy = userName,
        ChangedAt = DateTime.UtcNow,
        Changes = changes
    });
}

return result;
}

/// <summary>
/// AÄ0C0?C,,=C2 2C0CD$@CT=C08DR ACäAD$>D0=C,,9 EntityFramework State C$> C$=D4BD 5C0=C,,9 CD>CÄ5C0=D´9 C05D
/// </summary>
private EntityStateChangeEnum MapState(EntityState state) => state switch
{
    EntityState.Added => EntityStateChangeEnum.Added,
    EntityState.Deleted => EntityStateChangeEnum.Deleted,
}

```

```

        _ => EntityStateChangeEnum.Modified;
    };
}

/// <summary>
/// A 5Ct>C00D =Cä5 C0>C´CDt5C08CR 8CÄ5C08 D$5C=C"5C4> C0>C´LCt>C$0D$5C´O C,,7 C=C0BCT:D BC 0C$BCä@C,,7C
/// </summary>
/// <param name="provider">A´>C=C0C´LC0KC´ ?D >C$0C"4CT@ D ;D46C „D´ 6öçF -æW"´äÄ÷ & Óí
private async Task<string?> GetUserNameInternalAsync(IServiceProvider provider)
{
    try
    {
        var userProvider = provider.GetService<IUserProvider>();
        if (userProvider != null)
            return await userProvider.GetCurrentUserNameAsync();
    }
    catch
    {
        // ignored
    }
}

return "System";
}
}
</file>

<file path="Data/Interceptors/IUserProvider.cs">
namespace Promatis.Net.Data;
{
    public interface IUserProvider
    {
        // AÄ5D$>CB <Cä6CTB C KD$L C AC,,=DT@Cä=C0KCÄÄ 5D ;C, ?Cä;D4GCT=C,,5 C,,<CT=C, BD 5C CCTB Ct0C0@CäAC
        Task<string?> GetCurrentUserNameAsync();
    }
}
</file>

<file path="Data/Promatis.Net.Data.csproj">
<Project Sdk="Microsoft.NET.Sdk">
{
    <PropertyGroup>
        <TargetFramework>net10.0</TargetFramework>
        <ImplicitUsings>enable</ImplicitUsings>
        <Nullable>enable</Nullable>
    </PropertyGroup>
{
    <ItemGroup>
        <PackageReference Include="Microsoft.EntityFrameworkCore" Version="10.0.8" />
        <PackageReference Include="Microsoft.EntityFrameworkCore.Relational" Version="10.0.8" />
        <PackageReference Include="Npgsql.EntityFrameworkCore.PostgreSQL" Version="10.0.1" />
    </ItemGroup>
{
    <ItemGroup>
        <ProjectReference Include="..\Domain\Promatis.Net.Domain.csproj" />
    </ItemGroup>
}
"Ä-FV0w&÷W í
<AssemblyAttribute Include="System.Runtime.CompilerServices.InternalsVisibleTo">
    Ä & 0WFW# ä &0 F-2äæWBä Ä-6 F-öä0FVÄâFW7G3Ä0ö & 0WFW# ä Ä 00 CÄO D$5D BCä2Cä3Cä ?D >CT:D$0 -->
</AssemblyAttribute>
"Ä0-FV0w&÷W í
}
}
</Project>
</file>

<file path="Data/Triggers/Castom/AuditTrigger.cs">
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.DependencyInjection;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using System.Text.Json;
{
    namespace Promatis.Net.Data;
{
    public class AuditTrigger(
        IServiceScopeFactory scopeFactory,
        JsonSerializerOptions jsonOptions)

```

```

: IAfterSaveTrigger<DomainObject>()
{
    public async Task HandleAsync(EntityChangedEventArgs<DomainObject> args)
    {
        // AäGC,,IC 5CÂ @C =D$0C"<-D$8Cð >D" 2Cä7CÄ>Cd=Cä9 CD8C00CÄ8Dt5D :Cä9 Cö@Cä:D 8-Cä1Cä;CäGC=8 EF Core (
        Type entityType = args.Entity.GetType();
        if (entityType.Namespace == "Castle.Proxies" && entityType.BaseType != null)
        {
            entityType = entityType.BaseType;
        }

        // 1. BD8C´LD$: C´>C48D CCT< D$>C´LC= D$5 D CD"=CäAD$8, C= D$>D KCR ?Cä<CTGCT=D² " VF-M
        if (entityType.GetInterfaces().All(i => i.Name != nameof(IAudit)))
            return;

        // 2. BD>D <C,,@D45CÄ 4C =C0KCR 4C´O JSON
        object changesToSerialize = args.State switch
        {
            EntityStateChangeEnum.Added => args.Changes.Select(c => new
            {
                c.PropertyName,
                NewValue = c.CurrentValue
            }),
            EntityStateChangeEnum.Modified or EntityStateChangeEnum.SoftDeleted =>
args.Changes.Select(c => new
            {
                c.PropertyName,
                OldValue = c.OriginalValue,
                NewValue = c.CurrentValue
            }),
            EntityStateChangeEnum.Deleted => new { Message = "Aä1D=5C=B D44C ;CT= Cö>C´=CäAD$LDâ" Öí
        } _ => args.Changes;
    }

    // 3. B >Ct4C 5CÄ 7C ?C,,ADÄ ;Cä3C
    var auditLog = new AuditLog
    {
        Id = Guid.NewGuid(),
        EntityName = entityType.Name, // A,,!Aö A A´ Aö : Aö8D,,5CÄ GC,,AD$>CR 8CÄO C=;C AD 0 Cä@C4AD$@D4:D
        EntityId = args.Entity.Id,
        Action = args.State.ToString(),
        ChangedAt = args.ChangedAt,
        ChangedBy = args.ChangedBy,
        ChangesJson = JsonSerializer.Serialize(changesToSerialize, jsonOptions)
    };

    // 4. A,,!Aö A A´ Aö : A 5Ct>Cö0D =Cä5 D >DT@C =CT=C,,5 C" AB GCT@CT7 C$KCD5C´5Cö=D´9, C40D 0CöBC,,@Cä
    using IServiceScope scope = scopeFactory.CreateScope();

    // A,,ICT< DD0C @C,,D2 1C 7Cä2Cä3Cä :Cä=D$5C=AD$0 Dt5D 5Cr ?D >C$0C"4CT@ C´>C=0C´LCö>C4> Scope
    var factory = scope.ServiceProvider.GetService<IDbContextFactory<ApplicationDbContext>>();

    if (factory != null)
    {
        await using ApplicationDbContext context = await factory.CreateDbContextAsync();
        context.Set<AuditLog>().Add(auditLog);
        await context.SaveChangesAsync();
    }
    else
    {
        Console.WriteLine("[AuditTrigger][Error] Aö5 D44C ;CäADÄ =C 9D$8 IDbContextFactory<ApplicationDbC
    }
}
}
</file>

```

```

<file path="Data/Triggers/Global/FluentValidationTrigger.cs">
using FluentValidation;
using FluentValidation.Results;
using Microsoft.Extensions.DependencyInjection;
using Promatis.Net.Domain.Interface;

namespace Promatis.Net.Data;

```

```

    }
    public class FluentValidationTrigger(IServiceScopeFactory scopeFactory) // A,,!Aô A A´ Aô : A,,=Cd5C=BC,,@D45CÂ
    : IBeforeSaveTrigger<IDomainObjectHasKey<Guid>>
    {
        public async Task HandleAsync(EntityCancelEventArgs<IDomainObjectHasKey<Guid>> args)
        {
            Type entityType = args.Entity.GetType();
            Type validatorType = typeof(IValidator<>).MakeGenericType(entityType);

            // A,,!Aô A A´ Aô : B >Ct4C 5CÂ 8Ct>C´8D >C$0C0=D4N, C 5Ct>C00D =D4N Cä1C´0D BDÂ 2C,,4C,,<CäAD$8 (Scope
            // B0BCâ 3C @C =D$8D CCTB, DtBCâ •6W'f-6U &÷f-FW" 1D44CTB "Cd8C$KCÂ" 8 D4AC05D,,=Câ @C 7D 5D,,8D" =C H
            using IServiceScope scope = scopeFactory.CreateScope();

            // At0C0@C HC,,2C 5CÂ 2C ;C,,4C BCä@ C,,7 D 2CT6CT3CâÂ 3C @C =D$8D >C$0C0=Câ 6C,,2Cä3Câ 66÷ R0?D >C$0C"4C
            if (scope.ServiceProvider.GetService(validatorType) is IValidator validator)
            {
                var context = new ValidationContext<object>(args.Entity);
                ValidationResult result = await validator.ValidateAsync(context);

                if (!result.IsValid)
                {
                    args.Cancel = true;
                    // BD>D <C,,@D45CÂ :D 0D 8C$>CRÂ GC,,AD$>CR ACä>C ICT=C,,5 Cä1 CäHC,,1C=0DR 4C´O UI
                    args.ErrorMessage = string.Join(" | ", result.Errors.Select(e => e.ErrorMessage));
                }
            }
        }
    }
}
</file>

<file path="Data/Triggers/Global/ReferenceTreeParentTrigger.cs">
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Internal;
using Microsoft.EntityFrameworkCore.Metadata;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using System.Reflection;

namespace Promatis.Net.Data;

/// <summary>
/// B4=C,,2CT@D 0C´LC0KC´ 8C0DD 0D BD CC=BD4@C0KC´ BD 8C43CT@ CD;Dò ?D >C$5D :C, FCT;CäAD$=CäAD$8 C,,5D 0D EC,,G
/// C, AC=2Cä7C0>C´ 7C IC,,BD² >D" FC,,:C´8Dt5D :C,,E Ct0C$8D 8CÄ>D BCT9 C05D 5CB ACäED 0C05C08CT< C" !B4 ABí
/// </summary>
public class ReferenceTreeParentTrigger : IBeforeSaveTrigger<IDomainObjectHasKey<Guid>>
{
    private static readonly MethodInfo DbContextSetMethod = typeof(DbContext)
        .GetMethods(BindingFlags.Public | BindingFlags.Instance)
        .First(m => m.Name == nameof(DbContext.Set) && m.IsGenericMethod &&
            m.GetParameters().Length == 0);

    private static readonly MethodInfo AsNoTrackingMethod =
        typeof(EntityFrameworkQueryableExtensions)
        .GetMethods(BindingFlags.Public | BindingFlags.Static)
        .First(m => m.Name == nameof(EntityFrameworkQueryableExtensions.AsNoTracking) &&
            m.IsGenericMethod);

    private static readonly MethodInfo IgnoreQueryFiltersMethod =
        typeof(EntityFrameworkQueryableExtensions)
        .GetMethods(BindingFlags.Public | BindingFlags.Static)
        .First(m => m.Name == nameof(EntityFrameworkQueryableExtensions.IgnoreQueryFilters) &&
            m.IsGenericMethod);

    public async Task HandleAsync(EntityCancelEventArgs<IDomainObjectHasKey<Guid>> args)
    {
        // Aô Aä!B$ AR AT(AT A,, : Bt5D 5Cr @CTDC´5C=AC,,N C0@Cä2CT@D05CÄÂ 5D BDÂ ;C, C Cä1D=5C=BC AC$>C"AD$2
        // ATAC´8 D 2Cä9D BC$0 C05D" r MD$>D" >C JCT:D" =CR 4CT@CT2CâÂ <D² ?D >D BCâ 2D´ECä4C,,<!
        PropertyInfo? parentIdProp = args.Entity.GetType().GetProperty("ParentId");
        if (parentIdProp == null) return;

        // Bt8D$0CT< Ct=C GCT=C,,0 D 2Cä9D BC" 4C,,=C <C,,GCTAC=8D
        Guid entityId = args.Entity.Id;
        Guid? parentId = (Guid?)parentIdProp.GetValue(args.Entity);

        PropertyInfo? deletedAtProp = args.Entity.GetType().GetProperty("DeletedAt");
    }
}

```

```

Ð
// 1. At0D"8D$0 CăB Cō@Dô<Că3Că AC <CăFC,,BC,,@Că2C =C,,0Ð
if (parentId.HasValue && parentId == entityId)Ð
{Ð
    args.Cancel = true;Ð
    args.ErrorMessage = "Aă1D=5C=B Cō5 CĂ>Cd5D" 1D`BDÂ @Că4C,,BCT;CT< D 0CĂ>CĂC D 5C 5.";Ð
    return;Ð
}Ð
Ð
if (parentId.HasValue && parentId.Value != Guid.Empty)Ð
{Ð
    // A,,ICT< D >CD8D$5C`O C" 6† ævUG& 6¶W" AD 5CD8 C`NC KDR >C JCT:D$>C"Â C C= >D$>D KDR ACă2Cō0CD0CT
    object? localParent = args.Context.ChangeTracker.Entries()Ð
        .FirstOrDefault(e => ((IDomainObjectHasKey<Guid>)e.Entity).Id ==
parentId.Value)?.Entity;Ð
    Ð
    bool exists;Ð
    bool isDeleted;Ð
    Ð
    if (localParent != null)Ð
    {Ð
        exists = true;Ð
        var localDeletedAt = (DateTime?)deletedAtProp?.GetValue(localParent);Ð
        isDeleted = localDeletedAt != null;Ð
    }Ð
    elseÐ
    {Ð
        // A$0D,,0 Că@C,,3C,,=C ;DĂ=C O D 5DD;CT:D 8Dò 4C`O C$KDt8D ;CT=C,,0 C= >D =Dò 8 Query C" Tb 6÷&]
        IEntity? entityTypeInModel =
args.Context.Model.FindEntityType(args.Entity.GetType());Ð
        Type rootClrType = entityTypeInModel?.GetRootType()?.ClrType ??
args.Entity.GetType();Ð
    Ð
        object? rawSet =
DbContextSetMethod.MakeGenericMethod(rootClrType).Invoke(args.Context, null);Ð
        object? noTrackingQuery =
AsNoTrackingMethod.MakeGenericMethod(rootClrType).Invoke(null, [rawSet]);Ð
        object? ignoreFiltersQuery =
IgnoreQueryFiltersMethod.MakeGenericMethod(rootClrType).Invoke(null, [noTrackingQuery]);Ð
    Ð
        IQueryable<object> query = ((IQueryable)ignoreFiltersQuery!).Cast<object>();Ð
    Ð
        // A,,ICT< Că1D=5C=B Cō> CD8Cō0CĂ8Dt5D :Că<D2 AC$>C"AD$2D2 $-B" !B4 AM
        object? dbParent = await query.FirstOrDefaultAsync(x => EF.Property<Guid>(x, "Id")
== parentId.Value);Ð
    Ð
        exists = dbParent != null;Ð
        var dbDeletedAt = dbParent != null ? (DateTime?)deletedAtProp?.GetValue(dbParent) :
null;Ð
        isDeleted = dbDeletedAt != null;Ð
    }Ð
    Ð
    // 2. Aô@Că2CT@C=0 DD8Ct8Dt5D :Că3Că AD4ICTAD$2Că2C =C,,0 D >CD8D$5C`LD :Că3Că Cct;C
    if (!exists)Ð
    {Ð
        args.Cancel = true;Ð
        args.ErrorMessage = $"B4:C 7C =CōKC' @Că4C,,BCT;DĂAC=8C' >C JCT:D" ,, "C¢ . &VçD-Gò' =CR =C 9CD
        return;Ð
    }Ð
    Ð
    // 3. Aô@Că2CT@C=0 D BC BD4AC <Dô3C= >C4> D44C ;CT=C,,0Ð
    if (isDeleted)Ð
    {Ð
        args.Cancel = true;Ð
        args.ErrorMessage = "Aô5C`Lct0 Cō0Ct=C GC,,BDÂ @Că4C,,BCT;CT< D44C ;CT=CōKC' >C JCT:D"â#½
        return;Ð
    }Ð
    Ð
    // 4. A4 B4 Aă A / At B" B$ Aă" Bd A= Aă (A -> B -> C -> A)Ð
    Guid? currentCheckId = parentId;Ð
    var visitedIds = new HashSet<Guid>();Ð
    Ð
    while (currentCheckId.HasValue && currentCheckId.Value != Guid.Empty)Ð
    {Ð
        if (currentCheckId == entityId)Ð
        {Ð

```

```

        args.Cancel = true;
        args.ErrorMessage = "Bd8C;C,,GCTAC0D0 7C 2C,,AC,,<CäAD$!: C05C'Lct0 C05D 5CÄ5D BC,,BDÄ @Cä4
        return;
    }
}

if (!visitedIds.Add(currentCheckId.Value))
    break;

// A,,ICT< D0;CT<CT=D" FCT?CäGC08 D =C GC ;C 2 ChangeTracker
object? nextLocalElement = args.Context.ChangeTracker.Entries()
    .FirstOrDefault(e => ((IDomainObjectHasKey<Guid>)e.Entity).Id ==
currentCheckId.Value)?.Entity;
}

if (nextLocalElement != null)
{
    currentCheckId = (Guid?)parentIdProp.GetValue(nextLocalElement);
}
else
{
    Guid? id = currentCheckId;

    IEntityType? loopEntityTypeInModel =
args.Context.Model.FindEntityType(args.Entity.GetType());
    Type loopRootClrType = loopEntityTypeInModel?.GetRootType()?.ClrType ??
args.Entity.GetType();
}

    object? loopRawSet =
DbContextSetMethod.MakeGenericMethod(loopRootClrType).Invoke(args.Context, null);
    object? loopNoTrackingQuery =
AsNoTrackingMethod.MakeGenericMethod(loopRootClrType).Invoke(null, [loopRawSet]);
    object? loopIgnoreFiltersQuery =
IgnoreQueryFiltersMethod.MakeGenericMethod(loopRootClrType).Invoke(null, [loopNoTrackingQuery]);
}

    IQueryable<object> loopQuery =
((IQueryable)loopIgnoreFiltersQuery!).Cast<object>();
}

    object? parentNodeObj = await loopQuery.FirstOrDefaultAsync(x =>
EF.Property<Guid>(x, "Id") == id.Value);
}

    currentCheckId = parentNodeObj != null ?
(Guid?)parentIdProp.GetValue(parentNodeObj) : null;
}
}
}
}
}
</file>

<file path="Data/TriggerService/DatabaseTriggerService.cs">
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.DependencyInjection;
using System.Collections.Concurrent;
}
namespace Promatis.Net.Data;
}
/// <summary>
/// B 5C ;C,,7C FC,,0 Dd5C0BD 0C'Lc0>C4> CD8D ?CTBDt5D 0 D$@C,,3C45D >C"â #C0@C 2C'0CTB D 5C48D BD 0Dd8CT9 CD>CÄ
/// C, :Cä>D 4C,,=C,,@D45D" DC 7D² 2C ;C,,4C FC,,8 (CD> D >DT@C =CT=C,,0) C, CC$5CD>CÄ;CT=C,,0 (C0>D ;CR ACäED 0C05
/// </summary>
/// <param name="serviceProvider">A :D$CC ;DÄ=D'9 C0>C0BCT:D B Ct0C$8D 8CÄ>D BCT9 (DI Container) D$5C0CD"5C4>
public class DatabaseTriggerService(IServiceProvider serviceProvider) : IDatabaseTriggerService
{
    /// <summary>
    /// A4;Cä1C ;DÄ=Cä5 D >C KD$8CRÄ CC$5CD>CÄ;D0ND"5CR AC,,AD$5CÄC Cä1 D4AC05D,,=Cä< C0>CÄ<C,,BCR AD4IC0>D BC,
    /// A,,AC0>C'LctCCTBD 0 CD;D0 @CT0C0BC,,2C0>C4> Cä1C0>C$;CT=C,,0 C,,=D$5D DCT9D 0 (MudBlazor) C,,;C, AC02Cä7C0
    /// </summary>
    public static event Action<EntityStateChangeEnum, object>? OnEntityCommitted;

    /// <summary>
    /// B 5CTAD$@ C0>CD?C,,Adt8C0>C"Ä 2D'7Cä;C00DäIC,,ED 0 AD D >DT@C =CT=C,,0 C,,7CÄ5C05C08C' ,, $C 7C C ;C,,4C
    /// AD5C'5C40D" ?D 8C08CÄ0CTB C @C4CCÄ5C0BD² ACä1D'BC,,0 C, 0C0BD40C'Lc0KC' 66÷ VB 6W'f-6U &÷f-FW"1
    /// </summary>
    private static readonly Dictionary<Type, List<Func<EntityCancelArgsBase, IServiceProvider,
Task>>> BeforeSubscribers = new();
}

    /// <summary>

```

```

/// B 5CTAD$@ Cð>CD?C,,Adt8C¤>C"Â 2D´?Cä;C00DäIC,,ED 0 Að B AR CD ?CTHC0>C4> D >DT@C =CT=C,,0 C,,7CÄ5C05C08C
/// </summary>ð
private static readonly Dictionary<Type, List<Func<EntityChangedArgsBase, IServiceProvider,
Task>>> AfterSubscribers = new();ð
ð
/// <summary>ð
/// Að>D$>C¤>C 5Ct>C0ðD =D´9 C¤MD, 8CT@C @DT8C, BC,,?Cä2. A,,AC¤;DäGC 5D" ?Cä2D$>D =Cä5 C,,AC0>C´Lct>C$0C08C
/// C0@C, >C0@CT4CT;CT=C,,8 C 0Ct>C$KDR :C´0D ACä2 C, 8C0BCT@DD5C"ACä2 Cä1D 0C 0D$KC$0CT<D´E D CD"=CäAD$5C
/// </summary>ð
private static readonly ConcurrentDictionary<Type, List<Type>>> HierarchyCache = new();ð
ð
/// <summary>ð
/// B 5C48D BD 8D CCTB CD>CÄ5C0=D´9 D$@C,,3C45D 4C´0 C¤>C0:D 5D$=Cä3Cä BC,,?C AD4IC0>D BC,1
/// A$KctKC$0CTBD 0 C 2D$>CÄ0D$8Dt5D :C, ?D 8 D :C =C,,@Cä2C =C,,8 D 1Cä@Cä: C$> C$@CT<D0 8C08Dd8C ;C,,7C FC
/// </summary>ð
/// <typeparam name="TEntity">B$8C0 4Cä<CT=C0>C´ AD4IC0>D BC,ãÄ÷G– W & Óí
/// <typeparam name="TTrigger">B$8C0 :C´0D AC 0BD 8C43CT@C Ä @CT0C´8CtCDäICT3Cä ;Cä3C,,:D2 >C @C 1CäBC¤8.<
public void Register<TEntity, TTrigger>()ð
    where TEntity : classð
    where TTrigger : classð
{ð
    Type entityType = typeof(TEntity);ð
    Type triggerType = typeof(TTrigger);ð
ð
    // A0@Cä2CT@D05CÄÄ :C :C,,5 C,,=D$5D DCT9D K Cd8Ct=CT=C0>C4> Dd8C¤;C @CT0C´8CtCCTB CD0C0=D´9 D$@C,,3C45
    bool isBefore = typeof(IBeforeSaveTrigger<TEntity>).IsAssignableFrom(triggerType);ð
    bool isAfter = typeof(IAfterSaveTrigger<TEntity>).IsAssignableFrom(triggerType);ð
ð
    if (isBefore)ð
    {ð
        // BD>D <C,,@D45CÄ >D$;Cä6CT=C0CDä ;D0<C 4D2â Cä=D$5C"=CT@ 'sp' C05D 5CD0CTBD 0 C" <Cä<CT=D" 2D´7
        // DtBCä ?D 5CD>D$2D 0D"0CTB D4BCTGC¤C CD>C´3Cä6C,,2D4IC,,E Cä1D¤5C¤BCä2 (Captive Dependency).ð
        AddSubscriber(BeforeSubscribers, entityType, async (argsBase, sp) =>ð
        {ð
            // B 0Ct@CTHC 5CÄ MC¤7CT<C0;D0@ D$@C,,3C45D 0 C,,7 C :D$CC ;DÄ=Cä3Cä 66÷ R BCT:D4ICT3Cä 7C ?D >
            if (sp.GetRequiredService<TTrigger>() is IBeforeSaveTrigger<TEntity> handler)ð
            {ð
                // Aä1Cä@C GC,,2C 5CÄ 1C 7Cä2D´5 C @C4CCÄ5C0BD² 2 D BD >C4> D$8C08Ct8D >C$0C0=D´9 C¤>C0BCT
                var typedArgs = new EntityCancelEventArgs<TEntity>(ð
                    (TEntity)argsBase.Entity,ð
                    argsBase.State,ð
                    argsBase.Changes,ð
                    argsBase.Context);ð
ð
                await handler.HandleAsync(typedArgs);ð
ð
                // A$>Ct2D 0D"0CT< D 5CtCC´LD$0D$K C$KC0>C´=CT=C,,0 D$@C,,3C45D 0 Cä1D 0D$=Cä 2 C 0Ct>C$KC´
                argsBase.Cancel = typedArgs.Cancel;ð
                argsBase.ErrorMessage = typedArgs.ErrorMessage;ð
                argsBase.Handled = typedArgs.Handled;ð
            }ð
        });ð
    }ð
ð
    if (isAfter)ð
    {ð
        AddSubscriber(AfterSubscribers, entityType, async (argsBase, sp) =>ð
        {ð
            if (sp.GetRequiredService<TTrigger>() is IAfterSaveTrigger<TEntity> handler)ð
            {ð
                var typedArgs = new EntityChangedEventArgs<TEntity>(ð
                    (TEntity)argsBase.Entity,ð
                    argsBase.State,ð
                    argsBase.Changes,ð
                    argsBase.ChangedBy,ð
                    argsBase.ChangedAt);ð
ð
                await handler.HandleAsync(typedArgs);ð
                argsBase.Handled = typedArgs.Handled;ð
            }ð
        });ð
    }ð
ð
    // ATAC´8 C¤;C AD =CR @CT0C´8CtCCTB C08 Cä4C,,= C¤>C0BD 0C¤B – D0BCä :D 8D$8Dt5D :C 0 CäHC,,1C¤0 C¤>C0
    if (!isBefore && !isAfter)ð
    {ð

```



```

        throw new InvalidOperationException(
            $"A;C AD .G&-vvw%G- Ræ ÖWÖ =CR @CT0C'8CtCCTB IBeforeSaveTrigger C,,;C, " gFW%6 fUG&-vvw" 4C
        )
    }
}
D
    /// <summary>
    /// A$KCô>C'=Dô5D" AC=2Cä7CÔCDâ 2C ;C,,4C FC,,N C,,7CÄ5CÔ0CT<Cä9 D CD"=CäAD$8 Cô5D 5CB 5E >D$?D 0C$:Cä9 C"
    /// Aä?D 0D,,8C$0CTB C$ADâ FCT?CäGC=C CÔ0D ;CT4Cä2C =C,,0 D CD"=CäAD$8. Aô@CT@D'2C 5D" >Cô5D 0Dd8Dâ ?D 8 CÔ
    /// </summary>
    public async Task ValidateAsync(object entity, EntityStateChangeEnum state,
        List<PropertyChangeInfo> changes, DbContext context)
    {
        var args = new EntityCancelArgsBase(entity, state, changes, context);
D
        // Aä1DT>CD8CÂ 8CT@C @DT8Dâ BC,,?Cä2 (D 0CÄ>C' AD4ICÔ>D BC,Â 1C 7Cä2D'E C=;C AD >C" 8 C,,=D$5D DCT9D >C
        foreach (Type type in GetTypesHierarchy(entity.GetType()))
        {
            if (BeforeSubscribers.TryGetValue(type, out List<Func<EntityCancelArgsBase,
                IServiceProvider, Task>>> actions))
            {
                foreach (Func<EntityCancelArgsBase, IServiceProvider, Task> action in actions)
                {
                    // Aô5D 5CD0CT< C'>C=0C'LCÔKC' 6W'f-6U &+f-FW"Â A C= >D$>D KCÂ 1D'; D >Ct4C = CD0CÔ=D'9 DÔ
                    await action(args, serviceProvider);
D
                    // ATAC'8 D$@C,,3C45D 2D'AD$0C$8C² DC'0C2 6 æ6VÂ r =CT<CT4C'5CÔ=Câ ?D 5D KC$0CT< D$@C =Ct
                    if (args.Cancel) throw new OperationCanceledException(args.ErrorMessage);
                }
            }
        }
    }
}
D
    /// <summary>
    /// B 0D AD';C 5D" CC$5CD>CÄ;CT=C,,0 Cä1 D4ACô5D,,=Cä< D >DT@C =CT=C,,8 C,,7CÄ5CÔ5CÔ8C' 2 C 0CtC CD0CÔ=D'E.D
    /// Aô>CD4CT@Cd8C$0CTB C=0D :C 4CÔKC' ?CT@CTEC$0D" ,,DC'0C2 † æFÆVB' 8 CÔCC ;C,,D45D" ACä1D'BC,,5 C" 3C'>C
    /// </summary>
    public async Task NotifyAsync(object entity, EntityStateChangeEnum state,
        List<PropertyChangeInfo> changes, string? user, DateTime at)
    {
        var args = new EntityChangedEventArgsBase(entity, state, changes, user, at);
D
        foreach (Type type in GetTypesHierarchy(entity.GetType()))
        {
            if (AfterSubscribers.TryGetValue(type, out List<Func<EntityChangedEventArgsBase,
                IServiceProvider, Task>>> actions))
            {
                foreach (Func<EntityChangedEventArgsBase, IServiceProvider, Task> action in actions)
                {
                    await action(args, serviceProvider);
D
                    // ATAC'8 Cä4C,,= C,,7 D$@C,,3C45D >C" 2D'AD$0C$8C² † æFÆVB Ô G'VRÂ >C @C 1CäBC=0 Dd5Cô>Dt:C
                    if (args.Handled) return;
                }
            }
        }
D
        // AôCC ;C,,:C FC,,0 C" AD$0D$8Dt5D :C,,9 DÔ2CT=D" 4C'0 D 5C :D$8C$=Cä3Câ >C =Cä2C'5CÔ8Dò :Cä<Cô>CÔ5CÔBC
        OnEntityCommitted?.Invoke(state, entity);
    }
}
D
    /// <summary>
    /// A$ACô>CÄ>C40D$5C'LCÔKC' <CTBCä4 CD;Dò 1CT7Cä?C ACÔ>C4> CD>C 0C$;CT=C,,0 Cô>CD?C,,Adt8C= >C" 2 D ;Cä2C @C
    /// </summary>
    private void AddSubscriber<TArgs>(Dictionary<Type, List<Func<TArgs, IServiceProvider, Task>>>
        dict, Type type, Func<TArgs, IServiceProvider, Task> action)
    {
        if (!dict.TryGetValue(type, out List<Func<TArgs, IServiceProvider, Task>>> list))
        {
            list = new List<Func<TArgs, IServiceProvider, Task>>>();
            dict[type] = list;
        }
        list.Add(action);
    }
}
D
    /// <summary>
    /// A,,7C$;CT:C 5D" ?Cä;CÔCDâ 8CT@C @DT8Dâ BC,,?Cä2 CD;Dò CC=0Ct0CÔ=Cä3Câ >C JCT:D$0 (C$:C'NDt0Dò 1C 7Cä2D'

```

```

    /// B 5CtCC`LD$0D$K C=MD,,8D CDäBD O CD;Dò >C 5D ?CTGCT=C,,O C$KD >C= >C' ?D >C,,7C$>CD8D$5C`LCÔ>D BC, ?Cä4 C
    /// </summary>
    private IEnumerable<Type> GetTypesHierarchy(Type type) =>
    {
        HierarchyCache.GetOrAdd(type, t =>
        {
            var types = new List<Type>();
            // B 5C=CD AC,,2CÔ> CÔ>CD=C,,<C 5CÄADò 2C$5D E Cò> CD@CT2D2 =C AC`5CD>C$0CÔ8Dò :C`0D ACä2, C,,AC=;Dä
            for (Type? c = t; c != null && c != typeof(object); c = c.BaseType) types.Add(c);
            // AD>C 0C$;Dò5CÄ 2D 5 D 5C ;C,,7Cä2C =CÔKCR 8CÔBCT@DD5C"AD² 8 D41C,,@C 5CÄ 4D41C`8C=0D$K
            return types.Concat(t.GetInterfaces()).Distinct().ToList();
        });
    }

    /// <summary>
    /// B @CäGCÖKC' AC @CäA D BC BC,,GCTAC=8DR @CT3C,,AD$@C FC,,9.
    /// A,,ACô>C`LctCCTBD O Cò@CT8CÄCD"5D BC$5CÔ=Cä 2 Unit-D$5D BC E CD;Dò 8Ct>C`ODd8C, ?D >C4>CÔ>C" BCTAD$>C"
    /// </summary>
    internal static void ClearInternalRegistrations()
    {
        BeforeSubscribers.Clear();
        AfterSubscribers.Clear();
        HierarchyCache.Clear();
    }
}
</file>

<file path="Data/TriggerService/EntityStateChangeEnum.cs">
namespace Promatis.Net.Data;
{
    public enum EntityStateChangeEnum
    {
        Added,
        Modified,
        Deleted,
        SoftDeleted
    }
}
</file>

<file path="Data/TriggerService/IDatabaseTriggerService.cs">
using Microsoft.EntityFrameworkCore;
{
    namespace Promatis.Net.Data;
    {
        public interface IDatabaseTriggerService
        {
            // AÄ5D$>CB 4C`O CÔ0D BD >C":C, AC$0Ct5C' $!D4ICÔ>D BDÂÔ"D 8C43CT@"
            void Register<TEntity, TTrigger>()
                where TEntity : class
                where TTrigger : class;
        }

        // AÄ5D$>CDK C$KCô>C`=CT=C,,O (C$KctKC$0DäBD O C,,=D$5D FCT?D$>D >CÄ•
        Task ValidateAsync(object entity, EntityStateChangeEnum state, List<PropertyChangeInfo>
changes, DbContext context);
        Task NotifyAsync(object entity, EntityStateChangeEnum state, List<PropertyChangeInfo> changes,
string? user, DateTime at);
    }
}
</file>

<file path="Data/TriggerService/PropertyChangeInfo.cs">
namespace Promatis.Net.Data;
{
    public class PropertyChangeInfo
    {
        public string PropertyName { get; set; } = string.Empty;
        public object? OriginalValue { get; set; }
        public object? CurrentValue { get; set; }
    }
}
</file>

<file path="Data/TriggerService/TriggerEvents/EntityCancelArgsBase.cs">
using Microsoft.EntityFrameworkCore;
{
    namespace Promatis.Net.Data;
    {
        public class EntityCancelArgsBase(
            object entity,
            EntityStateChangeEnum state,
            List<PropertyChangeInfo> changes,

```

```

        DbContext context) : EntityEventArgsBase(entity, state, changes)
    {
        public DbContext Context { get; } = context;
        public bool Cancel { get; set; }
        public string? ErrorMessage { get; set; }
    }
</file>

<file path="Data/TriggerService/TriggerEvents/EntityCancelEventArgs.cs">
using Microsoft.EntityFrameworkCore;

namespace Promatis.Net.Data;

public class EntityCancelEventArgs<T>(
    T entity,
    EntityStateChangeEnum state,
    List<PropertyChangeInfo> changes,
    DbContext context)
    : EntityCancelArgsBase(entity!, state, changes, context) where T : class
{
    public new T Entity => (T)base.Entity;
}
</file>

<file path="Data/TriggerService/TriggerEvents/EntityChangedEventArgsBase.cs">
namespace Promatis.Net.Data;

// A 0Ct>C$KC' :C'0D A CD;Dò 2C0CD$@CT=C05C4> C,,AC0>C'Lct>C$0C08Dò 2 D 5D 2C,,AC]
public class EntityChangedEventArgsBase(
    object entity,
    EntityStateChangeEnum state,
    List<PropertyChangeInfo> changes,
    string? changedBy,
    DateTime changedAt) : EntityEventArgsBase(entity, state, changes) // <-- A00D ;CT4D45CÂ 7CD5D L0
{
    public string? ChangedBy { get; } = changedBy;
    public DateTime ChangedAt { get; } = changedAt;
}
</file>

<file path="Data/TriggerService/TriggerEvents/EntityChangedEventArgs.cs">
namespace Promatis.Net.Data;

// A$0D, BC,,?C,,7C,,@Cä2C =C0KC' :C'0D A CD;Dò BD 8C43CT@Cä2D
public class EntityChangedEventArgs<T>(
    T entity, EntityStateChangeEnum state,
    List<PropertyChangeInfo> changes,
    string? changedBy,
    DateTime changedAt)
    : EntityChangedEventArgsBase(entity!, state, changes, changedBy, changedAt) where T : class
{
    public new T Entity => (T)base.Entity;
}
</file>

<file path="Data/TriggerService/TriggerEvents/EntityEventArgsBase.cs">
namespace Promatis.Net.Data;

public abstract class EntityEventArgsBase(
    object entity,
    EntityStateChangeEnum state,
    List<PropertyChangeInfo> changes) : EventArgs
{
    public object Entity { get; } = entity;
    public EntityStateChangeEnum State { get; } = state;
    public List<PropertyChangeInfo> Changes { get; } = changes;
    public bool Handled { get; set; }
}
</file>

<file path="Data/TriggerService/TriggerInterfaces/IAfterSaveTrigger.cs">
namespace Promatis.Net.Data;

// AD;Dò ?D 0C$8C² $ Aä!A' " D >DT@C =CT=C,,0 (D42CT4Cä<C'5C08DòôHC,,=C •
public interface IAfterSaveTrigger<T> where T : class
{

```

```

        Task HandleAsync(EntityChangedEventArgs<T> args);
    }
</file>

<file path="Data/TriggerService/TriggerInterfaces/IBeforeSaveTrigger.cs">
namespace Promatis.Net.Data;
{
    // AD;Dò ?D 0C$8C² $ Aâ" ACâED 0C05C08Dò „2C ;C„4C FC„0)
    public interface IBeforeSaveTrigger<T> where T : class
    {
        Task HandleAsync(EntityCancelEventArgs<T> args);
    }
}
</file>

<file path="Data/Validation/GlobalPolymorphicValidator.cs">
using FluentValidation;
using FluentValidation.Results;
using Microsoft.Extensions.DependencyInjection;
{
    namespace Promatis.Net.Domain;
    {
        public class GlobalPolymorphicValidator<T> : AbstractValidator<T> where T : class
        {
            private readonly IServiceProvider _serviceProvider;

            public GlobalPolymorphicValidator(IServiceProvider serviceProvider)
            {
                _serviceProvider = serviceProvider ?? throw new
ArgumentNullException(nameof(serviceProvider));
            }

            protected override bool PreValidate(ValidationContext<T> context, ValidationResult result)
            {
                if (context.InstanceToValidate == null) return true;

                // 1. A0>C´CDt0CT< D 5C ;DÄ=D´9 D$8Cò >C JCT:D$0 C" @C =D$0C"<CR „=C ?D 8CÄ5D Â FW 'FÖVçEVæ–B 8C´8 A
                Type realType = context.InstanceToValidate.GetType();

                // ATAC´8 D$8Cò ACâ2C00CD0CTB D B „B.CRâ MD$> C¤>C05Dt=D´9 C¤;C AD Â 0 C05 C 0Ct>C$KC´ 0C AD$@C :D$=
                // D$> C„AC¤0D$L CD>Dt5D =C„9 C$0C´8CD0D$>D =CR =D46C0>, DtBCâ1D² =CR CC"BC, 2 C 5D :Câ=CTGC0CDâ @CT
                if (realType == typeof(T)) return true;

                // 2. B BD >C„< D$8Cò 8C0BCT@DD5C"AC 4C´O D$>Dt5Dt=Câ3Câ 2C ;C„4C BCâ@C „=C ?D 8CÄ5D Â •f Æ–F F÷#ÆF
                Type specificValidatorType = typeof(IValidator<>).MakeGenericType(realType);

                // 3. At0C0@C HC„2C 5CÂ 8Cr D´ B C$0C´8CD0D$>D K CD;Dò MD$>C4> C¤>C0:D 5D$=Câ3Câ BC„?C
                // B0BCâ 2C 6C0>, D$0C¤ :C : CD;Dò >CD=Câ3Câ BC„?C <Câ6CTB C KD$L Ct0D 5C48D BD 8D >C$0C0> C05D :Câ;
                List<IValidator> specificValidators =
_serviceProvider.GetServices(specificValidatorType).Cast<IValidator>().ToList();

                if (specificValidators.Any())
                {
                    // B >Ct4C 5CÂ =CR04Cd5C05D 8C¤ :Câ=D$5C¤AD" 2C ;C„4C FC„8 FluentValidation CD;Dò ?CT@CT4C GC, 4C
                    ValidationContext<object>? newContext =
ValidationContext<object>.CreateWithOptions(context.InstanceToValidate, options =>
                    {
                        // A„!A0 A A´ A0 : A¤0C0>C08Dt=D´9 CÄ5D$>CB fçVVçEf Æ–F F–ôâ 4C´O C0@D0<Câ9 C0@Câ1D >D :C, A
                        if (context.Selector != null)
                        {
                            options.UseCustomSelector(context.Selector);
                        }
                    });

                    // At0C0Cd :C 5CÂ :C 6CDKC´ =C 9CD5C0=D´9 D$>Dt5Dt=D´9 C$0C´8CD0D$>D =C AC´5CD=C„:C
                    foreach (IValidator validator in specificValidators)
                    {
                        // BtBCâ1D² 8Ct1CT6C BDÂ 7C FC„:C´8C$0C08DòÂ ?D >C$5D OCT<, DtBCâ =C 9CD5C0=D´9 C$0C´8CD0D$>D
                        if (validator.GetType().IsGenericType &&
validator.GetType().GetGenericTypeDefinition() == typeof(GlobalPolymorphicValidator<>))
                            continue;

                        ValidationResult specificResult = validator.Validate(newContext);

                        // A05D 5C0>D 8CÂ 2D 5 Câ1C00D CCd5C0=D´5 CâHC„1C¤8 C" >C IC„9 D 5CtCC´LD$0D" 2C ;C„4C FC„8 D
                        foreach (ValidationFailure error in specificResult.Errors)
                        {

```

```

        result.Errors.Add(error);
    }
}
}
return true;
}
}
</file>

<file path="Domain.Interface/Enums/EnumExtensions.cs">
using System.ComponentModel;
using System.Reflection;
namespace Promatis.Net.Domain.Interface;
public static class EnumExtensions
{
    /// <summary>
    /// A$>Ct2D 0D"0CTB Ct=C GCT=C,,5 C BD 8C CD$0 [Description] CD;Dò ;Dä1Cä3Câ MC´5CÄ5C0BC ?CT@CTGC,,AC´5C08
    /// ATAC´8 C BD 8C CD" >D$AD4BD BC$CCTB, C$>Ct2D 0D"0CTB D BC =CD0D BC0>CR 8CÄO .ToString().
    /// </summary>
    public static string GetDescription(this Enum value)
    {
        FieldInfo? field = value.GetType().GetField(value.ToString());
        if (field == null) return value.ToString();

        var attribute = field.GetCustomAttribute<DescriptionAttribute>();
        return attribute != null ? attribute.Description : value.ToString();
    }
}
</file>

<file path="Domain.Interface/Interfaces/IAudit.cs">
namespace Promatis.Net.Domain.Interface;
/// <summary>
/// B CD"=CäAD$8, D 5C ;C,,7D4ND"8CR MD$>D" 8C0BCT@DD5C"A, C CCDCD" 7C ?C,,AD´2C BDÄADò 2 AuditLog
/// </summary>
public interface IAudit
{
}
</file>

<file path="Domain.Interface/Interfaces/IDomainObject.cs">
namespace Promatis.Net.Domain.Interface;
public interface IDomainObject
{
    DateTime CreatedAt { get; init; }
}
</file>

<file path="Domain.Interface/Interfaces/IDomainObjectHasKey.cs">
namespace Promatis.Net.Domain.Interface;
public interface IDomainObjectHasKey<TKey> : IDomainObject
{
    TKey Id { get; set; }
}
</file>

<file path="Domain.Interface/Interfaces/ISoftDeletable.cs">
namespace Promatis.Net.Domain.Interface;
/// <summary>
/// A,,=D$5D DCT9D 4C´O CÄOC4:Cä3Câ CCD0C´5C08Dò >C JCT:D$>C-
/// </summary>
public interface ISoftDeletable
{
    DateTime? DeletedAt { get; set; }
    string? DeletedBy { get; set; }
}
</file>

```

```

<file path="Domain.Interface/Interfaces/ITreeNode.cs">
namespace Promatis.Net.Domain.Interface;
{
    /// <summary>
    /// A4;Că1C ;DĂ=D'9 Cô;C BDD>D <CT=CÔKC' :Că=D$@C :D" 4C' O C$ACTE C,,5D 0D EC,,GCTAC=8DR ,,4D 5C$>C$8CD=D'E) D C
    /// </summary>
    /// <typeparam name="T">B$8Cò 4Că<CT=CÔ>C' <Că4CT;C,ăĂ÷G- W & Óı
    public interface ITreeNode<T> : IDomainObjectHasKey<Guid> where T : class
    {
        /// <summary>
        /// A,,4CT=D$8DD8C=0D$>D @Că4C,,BCT;DĂAC=8C' >D 47C'0. B 0C$5C0 çVÆĂ 4C' O C= >D =CT2D'E D0;CT<CT=D$>C"ı
        /// A,,!Aô A A' Aô : AD>C 0C$;CT= set CD;Dò 2Că7CĂ>Cd=CăAD$8 C,,7CĂ5C05C08Dò AC$0Ct5C' ,,?CT@CT<CTICT=C,,O C
        /// </summary>
        Guid? ParentId { get; set; }
    }
    /// <summary>
    /// A00C$8C40Dd8Că=CÔ>CR AC$>C"AD$2Că AD KC':C, =C @Că4C,,BCT;DĂAC=8C' >C JCT:D"ı
    /// </summary>
    T? Parent { get; set; }
}
}
</file>

```

```

<file path="Domain.Interface/Promatis.Net.Domain.Interface.csproj">
<Project Sdk="Microsoft.NET.Sdk">
    <PropertyGroup>
        <TargetFramework>net10.0</TargetFramework>
        <ImplicitUsings>enable</ImplicitUsings>
        <Nullable>enable</Nullable>
    </PropertyGroup>
</Project>
</file>

```

```

<file path="Domain/AuditLog/AuditLog.cs">
namespace Promatis.Net.Domain;
{
    /// <summary>
    /// A CCD8D" AC,,AD$5CĂK
    /// </summary>
    public class AuditLog : DomainObject
    {
        /// <summary>
        /// A00C,,<CT=Că2C =C,,5 D >C KD$8Dý
        /// </summary>
        public string EntityName { get; init; } = string.Empty;
    }
    /// <summary>
    /// A,,4CT=D$8DD8C=0D$>D ACă1D'BC,,0
    /// </summary>
    public Guid EntityId { get; init; }
    /// <summary>
    /// AD5C"AD$2C,,5
    /// </summary>
    public string Action { get; init; } = string.Empty;
    /// <summary>
    /// AD0D$0 C,,7CĂ5C05C08Dý
    /// </summary>
    public DateTime ChangedAt { get; init; }
    /// <summary>
    /// A,,7CĂ5C05C0>D
    /// </summary>
    public string? ChangedBy { get; init; }
    /// <summary>
    /// B BD >C=0 C,,7CĂ5C05C08C•

```

```

    /// </summary>
    public string ChangesJson { get; init; } = string.Empty;
}
</file>

<file path="Domain/DomainObject/DomainObject.cs">
namespace Promatis.Net.Domain;
{
    /// <summary>
    /// AĀCÔ>C$=Cā9 Cā;C AD „uT”B ?Cā CCĀ>C’GC =C„N)
    /// </summary>
    public abstract class DomainObject : DomainObjectBase<Guid>
    {
        protected DomainObject()
        {
            Id = Guid.NewGuid();
        }
    }
}
</file>

<file path="Domain/DomainObject/DomainObjectBase.cs">
using Promatis.Net.Domain.Interface;
{
namespace Promatis.Net.Domain;
{
    /// <summary>
    /// B4=C„2CT@D 0C’LCÔKC’ 1C 7Cā2D’9 Cā;C AD A Cō>CD4CT@Cd:Cā9 C’NC >C4> D$8Cō0 Cā;DāGC
    /// </summary>
    /// <typeparam name="TKey">B$8Cò :C’NDt0</typeparam>
    public abstract class DomainObjectBase<TKey> : IDomainObjectHasKey<TKey>
    {
        public TKey Id { get; set; } = default!;
    }
    public DateTime CreatedAt { get; init; } = DateTime.UtcNow;
}
</file>

<file path="Domain/DomainObject/DomainObjectValidator.cs">
using FluentValidation;
{
namespace Promatis.Net.Domain;
{
    /// <summary>
    /// B4=C„2CT@D 0C’LCÔKC’ 2C ;C„4C BCā@ CD;Dò 2D 5DR :C’0D ACā2, CÔ0D ;CT4D45CĀKDR >D" FöÖ -äö&!V7M
    /// </summary>
    public abstract class DomainObjectValidator<T> : AbstractValidator<T> where T : DomainObject
    {
        protected DomainObjectValidator()
        {
            RuleFor(x => x.Id)
                .NotEmpty().WithMessage("A„4CT=D$8DD8Cā0D$>D >C JCT:D$0 CÔ5 CĀ>Cd5D" 1D’BDĀ ?D4AD$KCĀĀ""½
        }
            RuleFor(x => x.CreatedAt)
                .Must(date => date <= DateTime.UtcNow.AddSeconds(1))
                .WithMessage("AD0D$0 D >Ct4C =C„0 CÔ5 CĀ>Cd5D" 1D’BDĀ 2 C CCDCD”5CĀĀ""½
        }
    }
}
</file>

<file path="Domain/Promatis.Net.Domain.csproj">
<Project Sdk="Microsoft.NET.Sdk">
{
    <PropertyGroup>
        <TargetFramework>net10.0</TargetFramework>
        <ImplicitUsings>enable</ImplicitUsings>
        <Nullable>enable</Nullable>
    </PropertyGroup>
{
    <ItemGroup>
        <PackageReference Include="FluentValidation" Version="12.1.1" />
    </ItemGroup>
{
    <ItemGroup>
        <ProjectReference Include="..\Domain.Interface\Promatis.Net.Domain.Interface.csproj" />
    </ItemGroup>
}
}
}

```

```
</Project>
</file>
```

```
<file path="Domain/ReferenceBase/ReferenceBase.cs">
using Promatis.Net.Domain.Interface;
{
namespace Promatis.Net.Domain;
{
/// <summary>
/// A 0Ct>C$K' :C'0D A CD;D0 2D 5DR AC0@C 2CäGC08C>C-
/// </summary>
public abstract class ReferenceBase : DomainObject, ISoftDeletable
{
    public string Name { get; set; } = string.Empty;
    public string? Description { get; set; }
    public string? Code { get; set; }
    public DateTime? DeletedAt { get; set; }
    public string? DeletedBy { get; set; }
}
</file>
```

```
<file path="Domain/ReferenceBase/ReferenceBaseValidator.cs">
using FluentValidation;
using FluentValidation.Results;
{
namespace Promatis.Net.Domain;
{
/// <summary>
/// B4=C,,2CT@D 0C'LC0KC' 2C ;C,,4C BCä@ CD;D0 2D 5DR :C'0D ACä2, C00D ;CT4D45CÄKDR >D" &VfW&Væ6T& 6]
/// </summary>
public abstract class ReferenceBaseValidator<T> : DomainObjectValidator<T> where T : ReferenceBase
{
    /// <summary>
    /// A$8D BD40C'LC0KC' ?C BD$5D = D 5C4CC'0D =Cä3Cä 2D'@C 6CT=C,,0 CD;D0 ?D >C$5D :C, Cä4C í
    /// A0> D4<Cä;Dt0C08DÄ¢ AD$@Cä3Cä ;C BC,,=C,,FC 2 C$5D EC05CÄ @CT3C,,AD$@CRÄ FC,,DD K C, 0í
    /// </summary>
    protected virtual string CodeRegexPattern => @"^[A-Z0-9_]*$";
    {
        /// <summary>
        /// A$8D BD40C'LC0>CR ACä>C ICT=C,,5 Cä1 CäHC,,1C±5 C$0C'8CD0Dd8C, @CT3D4;D0@C0>C4> C$KD 0Cd5C08D0 Cä4C í
        /// </summary>
        protected virtual string CodeValidationMessage => "A±>CB <Cä6CTB D >CD5D 6C BDÄ BCä;DÄ:Cä ;C BC,,=C,,FD2 2
        protected ReferenceBaseValidator()
        {
            RuleFor(x => x.Name)
                .NotEmpty().WithMessage("A00C,,<CT=Cä2C =C,,5 Cä1D07C BCT;DÄ=Cä 4C'0 Ct0C0>C'=CT=C,,0")
                // A0@Cä2CT@C±0 C00 C0@Cä1CT;D² 2 C00Dt0C'5 C, :Cä=Dd5
                .Must(name => name == null || name.Trim() == name)
                .WithMessage("A00C,,<CT=Cä2C =C,,5 C05 CÄ>Cd5D" =C GC,,=C BDÄAD0 8C'8 Ct0C±0C0GC,,2C BDÄAD0 ?D >C 5C'
                .MinimumLength(2).WithMessage("A00C,,<CT=Cä2C =C,,5 CD>C'6C0> D >CD5D 6C BDÄ <C,,=C,,<D4< 2 D 8CÄ2Cä;
                .MaximumLength(250).WithMessage("A0@CT2D'HCT=C <C :D 8CÄ0C'LC00D0 4C'8C00 C00C,,<CT=Cä2C =C,,0 (25
            RuleFor(x => x.Code)
                .Matches(x => CodeRegexPattern)
                .When(x => !string.IsNullOrEmpty(x.Code))
                .WithMessage(x => CodeValidationMessage);
        }
    }
    public Func<object, string, Task<IEnumerable<string>>> ValidateValue => async (model,
propertyName) =>
    {
        ValidationResult? result = await
ValidateAsync(ValidationContext<T>.CreateWithOptions((T)model, x =>
x.IncludeProperties(propertyName)));
        return result.IsValid ? [] : result.Errors.Select(e => e.ErrorMessage);
    }
}
</file>
```

```
<file path="Domain/ReferenceTreeBase/ReferenceTreeBase.cs">
using Promatis.Net.Domain.Interface;
```



```

Ð
namespace Promatis.Net.Domain;Ð
Ð
/// <summary>Ð
/// A 0Ct>C$KC' :C'0D A CD;D0 2D 5DR AC0@C 2CäGC08C¤>C" ACä4CT@Cd0D"8DR 4CT@CT2DÄ0Ð
/// </summary>Ð
/// <summary>Ð
/// A 0Ct>C$KC' :C'0D A CD;D0 2D 5DR 4D 5C$>C$8CD=D'E D ?D 0C$>Dt=C,,:Cä2 (MDM).Ð
/// A,,AC0>C'LCtCCTB C00D$BCT@C0 5%E 4C'0 C05D 5CD0Dt8 D BD >C4> D$8C08Ct8D >C$0C0=D'E C00C$8C40Dd8Cä=C0KDR A
/// </summary>Ð
/// <typeparam name="T">B$8C0 :Cä=C¤@CTBC0>C4> CD>CÄ5C0=Cä3Cä >C JCT:D$0 (C00D ;CT4C08C¤0).</typeparam>Ð
public abstract class ReferenceTreeBase<T> : ReferenceBase, ITreeNode<T> where T : classÐ
{Ð
    /// <summary>Ð
    /// A,,4CT=D$8DD8C¤0D$>D @Cä4C,,BCT;DÄAC¤>C4> D47C'0 C" !B4 ABi
    /// </summary>Ð
    public Guid? ParentId { get; set; }Ð
Ð
    /// <summary>Ð
    /// A00C$8C40Dd8Cä=C0>CR AC$>C"AD$2Cä AD KC':C, =C @Cä4C,,BCT;DÄAC¤8C' >C JCT:D" 2 Cä?CT@C BC,,2C0>C' ?C <
    /// A05D 5Cä?D 5CD5C'0CTBD 0 C¤0C¤ f-'GV Â 2 C¤>C05Dt=D'E C¤;C AD 0DR 4C'0 C0>CD4CT@Cd:C, Æ $' Æö F-ærí
    /// </summary>Ð
    public virtual T? Parent { get; set; }Ð
Ð
    /// <summary>Ð
    /// A¤>C';CT:Dd8D0 4CäGCT@C08DR MC'5CÄ5C0BCä2 (C0>CDGC,,=CT=C0KDR CCT;Cä2).Ð
    /// </summary>Ð
    public virtual ICollection<T> Children { get; set; } = new List<T>();Ð
}
</file>

<file path="Domain/ReferenceTreeBase/ReferenceTreeBaseValidator.cs">
using FluentValidation;Ð
using Promatis.Net.Domain.Interface;Ð
Ð
namespace Promatis.Net.Domain;Ð
Ð
/// <summary>Ð
/// B4=C,,2CT@D 0C'LC0KC' 2C ;C,,4C BCä@ CD;D0 2D 5DR :C'0D ACä2, C00D ;CT4D45CÄKDR >D" &Vfw&Væ6UG&VT& 6]
/// </summary>Ð
public abstract class ReferenceTreeBaseValidator<T> : ReferenceBaseValidator<T>Ð
where T : ReferenceBase, ITreeNode<T>Ð
{Ð
    protected ReferenceTreeBaseValidator() : base()Ð
    {Ð
        // A0@Cä2CT@C¤0 C00 D 0CÄ>Dd8D$8D >C$0C08CR ,,>C JCT:D" =CR <Cä6CTB D4:C 7D'2C BDÂ =C ACT1D0 :C : C00
        RuleFor(x => x.ParentId)Ð
            .Must((model, parentId) => parentId != model.Id)Ð
            .WithMessage("Aä1D¤5C¤B C05 CÄ>Cd5D" 1D'BDÄ @Cä4C,,BCT;CT< D 0CÄ>CÄC D 5C 5");Ð
Ð
        // A 0Ct>C$0D0 ?D >C$5D :C DCä@CÄ0D$0 GUIDÐ
        RuleFor(x => x.ParentId)Ð
            .NotEqual(Guid.Empty)Ð
            .When(x => x.ParentId.HasValue)Ð
            .WithMessage("B >CD8D$5C'LD :C,,9 C,,4CT=D$8DD8C¤0D$>D =CR <Cä6CTB C KD$L C0CD BD'< GUID");Ð
    }Ð
}
</file>

<file path="Promatis.Net.UI/_Imports.razor">
@using Microsoft.AspNetCore.Components.FormsÐ
@using Microsoft.AspNetCore.Components.RoutingÐ
@using Microsoft.AspNetCore.Components.WebÐ
@using MudBlazorÐ
@using Promatis.Net.UI.Components.WorkspaceÐ
@using Promatis.Net.UI.Components.EditDialogÐ
@using Promatis.Net.UI.Components.GridÐ
@using Promatis.Net.UI.Components.ToolbarÐ
@using Promatis.Net.Domain.Interface
</file>

<file path="Promatis.Net.UI/Components/EditDialog/DialogTab.razor">
@* A¤>CÄ?Cä=CT=D" =CR @CT=CD5D 8D" ...D0Ä AC <, Cä= D ;D46C,,B CD5C¤;C @C BC,,2C0KCÄ >C08D 0D$5C'5CÄ 4C'0 EditDia
@code {Ð
    [Parameter] public required string Title { get; set; }Ð
    [Parameter] public string? Icon { get; set; }Ð

```

```

    [Parameter] public required RenderFragment Content { get; set; }
}
</file>

<file path="Promatis.Net.UI/Components/EditDialog/DialogTab.razor.cs">
using Microsoft.AspNetCore.Components;
{
namespace Promatis.Net.UI.Components.EditDialog;
{
public partial class DialogTab : ComponentBase
{
    [CascadingParameter] protected EditDialog ParentDialog { get; set; } = null!;

    protected override void OnInitialized()
    {
        base.OnInitialized();

        if (ParentDialog != null)
        {
            ParentDialog.RegisterTab(this); // A0D$8C$=Câ >D$4C 5CÂ ACT1Dò =C 2CT@DR 2 Cα>C´;CT:Dd8Dâ @Câ4C,,
        }
    }
}
</file>

<file path="Promatis.Net.UI/Components/EditDialog/EditDialog.razor">
@namespace Promatis.Net.UI.Components.EditDialog
{
@inherits Microsoft.AspNetCore.Components.ComponentBase
{
<MudDialog>
    <TitleContent>
        <MudText Typo="Typo.h6">
            <MudIcon Icon="@((IsNew ? Icons.Material.Filled.Add : Icons.Material.Filled.Edit))"
Class="mr-2 mb-n1" />
            @Title
        </MudText>
    </TitleContent>
    <DialogContent>
        @* B :D KD$KC' AC >D IC,, : C$:C'0CD>C¢ 8Cr 4CäGCT@C05C4> Cα>C0BCT=D$0 *@
        <CascadingValue Value="this" IsFixed="true">
            @Tabs
        </CascadingValue>
    </DialogContent>
    <MudForm @ref="_form"
        Model="@Model"
        Validation="@((async (object m) => await ExecuteFluentValidationAsync(m)))"
        ValidationDelay="200">
        @* A,,!Aô A A´ Aô : A$=CTHC08C' :Cä=D$5C"=CT@ Cd5D BCα> CD5D 6C,,B C$KD >D$C Cä:C00 DD8CαAC,,@Cä2C
        <div class="d-flex flex-column pt-2" style="min-height: 340px; max-height: 65vh; min-
width: 0;">
            @if (_isProcessing)
            {
                <div class="d-flex justify-center my-4">
                    <MudProgressCircular Color="Color.Primary" Indeterminate="true" />
                </div>
            }
            else
            {
                @* A B$ AÄ B$ Aα B Aô AT A,, A4 : ATAC´8 C$:C'0CD:C >CD=C r 4C 5CÂ 5C' ACα@Cä;C´1C @
                @if (_collectedTabs.Count == 1)
                {
                    <div style="max-height: 60vh; overflow-y: auto; padding-right: 4px;">
                        @_collectedTabs.First().Content
                    </div>
                }
                else if (_collectedTabs.Count > 1)
                {
                    @* A,,!Aô A A´ Aô : At0CD0CT< PanelStyle CD;Dò =C <CT@D$2Câ DC,, :D 8D >C$0C0=Cä3Câ ACα
                    <MudTabs Elevation="0"
                        Rounded="true"
                        ApplyEffectsToInnerContainer="true"
                        PanelClass="pa-4"
                        PanelStyle="max-height: 50vh; overflow-y: auto; padding-right:
4px;">

```

```

        @foreach (var tab in _collectedTabs)
        {
            <MudTabPanel Text="@tab.Title" Icon="@tab.Icon">
                @tab.Content
            </MudTabPanel>
        }
    </MudTabs>
}
</div>
</MudForm>
</DialogContent>
<DialogActions>
    <MudButton Variant="Variant.Text" OnClick="Cancel" Disabled="_isProcessing">AâBCÄ5C0</MudButton>
    <MudButton Variant="Variant.Filled" Color="Color.Primary" OnClick="Submit"
Disabled="_isProcessing">
        @if (_isProcessing)
        {
            <MudProgressCircular Class="ms-n1" Size="Size.Small" Indeterminate="true" />
            <MudText Class="ms-2">B >DT@C =CT=C,,5...</MudText>
        }
        else
        {
            <MudText>B >DT@C =C,,BDÄÄô×VEFW‡Cí
        }
    </MudButton>
</DialogActions>
</MudDialog>
</file>

<file path="Promatis.Net.UI/Components/EditDialog/EditDialog.razor.cs">
using FluentValidation;
using FluentValidation.Results;
using Microsoft.AspNetCore.Components;
using MudBlazor;
using Severity = MudBlazor.Severity;

namespace Promatis.Net.UI.Components.EditDialog;

public partial class EditDialog : ComponentBase
{
    protected MudForm _form = null!;
    protected bool _isProcessing;
    protected readonly List<DialogTab> _collectedTabs = [];

    [CascadingParameter] protected IMudDialogInstance MudDialog { get; set; } = null!;

    [Parameter] public string Title { get; set; } = "B 5CD0CâBC,,@Cä2C =C,,5";
    [Parameter] public bool IsNew { get; set; }
    [Parameter] public object Model { get; set; } = null!;
    [Parameter] public IValidator Validator { get; set; } = null!;
    [Parameter] public Func<Task> OnSaveAction { get; set; } = null!;

    /// <summary>
    /// B NCD0 C0@C,,:C´0CD=Cä9 D 0Ct@C 1CäBDt8C¢ 4CT:C´0D 0D$8C$=Câ ?C,,HCTB D$5C48 <DialogTab>
    /// </summary>
    [Parameter] public RenderFragment? Tabs { get; set; }

    [Inject] protected ISnackbar Snackbar { get; set; } = null!;

    /// <summary>
    /// AÄ5D$=CB 4C´0 D 5C48D BD 0Dd8C, 2C¢;C 4Cä:, C$KctKC$0CT<D´9 CD>Dt5D =C,,<C, :Cä<C0>C05C0BC <C, F- Å0uF
    /// </summary>
    public void RegisterTab(DialogTab tab)
    {
        if (!_collectedTabs.Contains(tab))
        {
            _collectedTabs.Add(tab);
            StateHasChanged();
        }
    }

    protected void Cancel() => MudDialog.Cancel();

    protected async Task<IEnumerable<string>> ExecuteFluentValidationAsync(object model)

```

```

    {
        if (model is not Domain.DomainObject || Validator == null) return Array.Empty<string>();
        var context = new ValidationContext<object>(model);
        ValidationResult result = await Validator.ValidateAsync(context);
        return result.IsValid ? Array.Empty<string>() : result.Errors.Select(e => e.ErrorMessage);
    }

    protected async Task Submit()
    {
        await _form.ValidateAsync();

        if (!_form.IsValid)
        {
            Snackbar.Add("Aö>Cd0C´CC´AD$0, C,,ACô@C 2DÄBCR >D,,8C :C, 2 DD>D <CR ?CT@CT4 D >DT@C =CT=C,,5CÂâ"Ä 6", Severity.Error);
        }

        _isProcessing = true;
        StateHasChanged();

        try
        {
            if (OnSaveAction != null) await OnSaveAction();
            MudDialog.Close(DialogResult.Ok(Model));
        }
        catch (Exception ex)
        {
            string message = ex.InnerException?.Message ?? ex.Message;
            Snackbar.Add($"AäHC,,1C¤0 D >DT@C =CT=C,,0: {message}", Severity.Error);
        }
        finally
        {
            _isProcessing = false;
            StateHasChanged();
        }
    }
}
</file>

<file path="Promatis.Net.UI/Components/Grid/GridActionContext.cs">
using Promatis.Net.UI.Components.Toolbar;
namespace Promatis.Net.UI.Components.Grid;
public abstract class GridActionContext<TEntity> : ToolbarActionContext<TEntity> where TEntity : class
{
    public TEntity? SelectedItem
    {
        get => SelectedData;
        set => SelectedData = value;
    }

    /// <summary>
    /// B4=C,,2CT@D 0C´LCÔKC´ 1D >C¤5D 4C =CÔKDR BCT:D4ICT9 D$0C ;C,,FD²i
    /// </summary>
    public UiDataBroker<TEntity> DataBroker { get; } = new();

    public GridActionContext() : base()
    {
        Position = ToolbarPosition.Top;
    }

    // =====
    // Aö A "BD B AT Aô+A´ A$"Ää A "A,, 'AT!A¤ A´ A$ Ad A¢ At#-ÄÄ#B$ Bd A´ ,, Ää Aô/B" A AT BR•
    // =====

    /// <summary>
    /// A4;Cä1C ;DÄ=D´9 C05D 5DT2C BDt8C¢ :Cä<CÄ8D$>C" !B4 AB =C CD >C$=CR BC 1C´8Dt=Cä3Cä 0CD@C 1
    /// Aö>C´=CäAD$LDâ >D 2Cä1Cä6CD0CTB Cö@C,, :C´0CD=D´E D 0Ct@C 1CäBDt8C¤>C" >D" @D4GCô>C4> CÔ0C08D 0CÔ8Dò ;C

```

```

    /// </summary>
    public override void HandleGlobalEntityCommit(object? state, object? entity)
    {
        if (entity == null) return;

        Type entityType = entity.GetType();

        // B @CT7C 5CÂ 4C,,=C <C,,GCTAC=8CR ?D >C=AC, 6 7FÆRôTb 6÷&]
        if (entityType.BaseType != null && entityType.Namespace == "Castle.Proxies")
        {
            entityType = entityType.BaseType;
        }

        // 1. ATAC'8 D$@C =Ct0C=FC,,0 C,,7 C 0CtK CD0C0=D'E Ct0D$@Cä=D4;C 8CÄ5C0=Cä BC,,? CD0C0=D'E C00D,,5C' BC
        if (typeof(TEntity).IsAssignableFrom(entityType))
        {
            string stateStr = state?.ToString() ?? string.Empty;

            // 2. A 2D$>CÄ0D$8Dt5D :C, ?C,,=C 5CÂ At#-CD2C,,6Cä: C @Cä:CT@C ?D 8CÄ5C08D$LD CD5C'LD$C Ct0 0 CÄA
            DataBroker.ApplyIncremental0zuDelta(stateStr, (TEntity)entity);

            // 3. A$KctKC$0CT< C 0Ct>C$KCR ?D 0C$8C'0 D 1D >D 0 DD>C=CD 0 ToolbarActionContext
            base.HandleGlobalEntityCommit(state, entity);

            // 4. A 2D$>CÄ0D$8Dt5D :C, ?C,,=C 5CÂ w&-E vR ?CT@CT@C,,ACä2C BDÄ ...D0Ä0AD$@Cä:C, 8Cr >C =Cä2C'5C0=
            RequestRefresh();
        }
    }

    protected override void RecalculateButtonStates()
    {
        base.RecalculateButtonStates();
        NotifyUpdate();
    }
}
</file>

```

```

<file path="Promatis.Net.UI/Components/Grid/GridPage.razor">
@typeparam TEntity where TEntity : class
{
    <MudDataGrid T="TEntity"
        ServerData="@LoadGridDataInternalAsync"
        @ref="_grid"
        SelectedItem="@ActionContext.SelectedData"
        SelectedItemChanged="@OnSelectedItemChanged"
        RowStyleFunc="@FormatSelectedRowStyle"
        Hover="true"
        Dense="true"
        Bordered="true"
        Elevation="0"
        FixedHeader="true"
        Height="calc(100vh - 280px)">
        <Columns>
            @ColumnsContent
        </Columns>
        <PagerContent>
            @if (PagerContent != null)
            {
                @PagerContent
            }
            else
            {
                <MudDataGridPager T="TEntity" InfoFormat="{first_item}-{last_item} C,,7 {all_items}"
                RowsPerPageString="B BD >C$ =C AD$@C =C,,FCS$" ói
            }
        </PagerContent>
    </MudDataGrid>
</file>

```

```

<file path="Promatis.Net.UI/Components/Grid/GridPage.razor.cs">
using Microsoft.AspNetCore.Components;
using MudBlazor;

namespace Promatis.Net.UI.Components.Grid;

public partial class GridPage<TEntity> : ComponentBase, IDisposable where TEntity : class

```

```

{
    [Parameter] public RenderFragment? ColumnsContent { get; set; }
    [Parameter] public RenderFragment? PagerContent { get; set; }

    [CascadingParameter] protected GridActionContext<TEntity> ActionContext { get; set; } = null!;

    private MudDataGrid<TEntity> _grid = null!;

    protected override void OnInitialized()
    {
        base.OnInitialized();

        if (ActionContext == null)
        {
            throw new ArgumentNullException(nameof(ActionContext),
                $"A?C?C=CT=D" "æ ÖVöb,,w&-E vSÄDVçF-G"â-ö BD 5C CCTB CÖ0C'8Dt8Dò "æ ÖVöb,,w&-D 7F-öä6öçFW#C
            );
        }

        ActionContext.OnContextUpdated += StateHasChanged;
        ActionContext.OnRefreshRequested += HandleRefreshRequested;
    }

    private void HandleRefreshRequested()
    {
        InvokeAsync(ReloadServerDataAsync);
    }

    /// <summary>
    /// A$=D4BD 5CÔ=C,,9 CÄ5D$>CBÔ<CäAD"â C$BCä<C BC,,GCTAC=8 C, 1CT7Cä?C ACÔ> C$KCKC$0CTBD 0 C= CÄ?Cä=CT=D$>
    /// C" 0D 8CÔED >CÔ=Cä< C=CÖBCT:D BCR &Æ |÷"Â 8D :C'NDt0Dò ;Dä1D'5 DD@C,,7D² <CT=Däí
    /// </summary>
    protected async Task<GridData<TEntity>> LoadGridDataInternalAsync(GridState<TEntity> state,
        CancellationToken token)
    {
        // A B$ AÄ B$ A= Aö B A,, 'Aö A4 Aö Aä B A$ :D
        // ATAC'8 C$:C'NDt5CÔ At#-D 5Cd8CÄ ,,1D >C=5D MD$> Ct=C 5D"'Â =Cä 4C =CÖKCR 5D"5 CÔ5 Cö@CT4Ct0C4@D46
        if (ActionContext.DataBroker.IsInMemoryMode && ActionContext.DataBroker.InMemoryItems ==
            null)
        {
            // A,,ICT< CÄ5D$>CB ?D >C4@CT2C =C :Cä=C=CTBCÔ>CÄ :Cä=D$5C=AD$5 D BD 0CÔ8DdKö
            var initMethod = ActionContext.GetType().GetMethod("InitializeInMemoryDataAsync");
            if (initMethod != null)
            {
                // A$KCKC$0CT< CT3Cä 8 Ad AT (await) Ct0C$5D HCT=C,,0. D
                // B$0Cç :C : DÖBCäB CÄ5D$>CB 2D'?Cä;CÖ0CTBD 0 C$=D4BD 8 C=CÖBCT:D BC 6W'fW$F F Ä
                // MudBlazor D 0CÄ 2C=;DäGC,,B C=C AC,,2D'9 C'>C 4CT@, C <CT=Dä AD$@C =C,,FD² >D BC =CTBD 0 10
                var task = (Task)initMethod.Invoke(ActionContext, null!);
                await task;
            }
        }

        // Aö0Cö@Dö<D4N C$KCKC$0CT< C @Cä:CT@ CD0Cö=D'E, D 0Ct2CT@CÖCD$KC' 2CÖCD$@C, =C HCT3Cä :Cä=D$5C=AD$0
        return await ActionContext.DataBroker.FetchDataAsync(state, token);
    }

    protected void OnSelectedItemChanged(TEntity? newItem)
    {
        if (newItem == null && ActionContext.SelectedData != null) return;

        if (!EqualityComparer<TEntity>.Default.Equals(ActionContext.SelectedData, newItem))
        {
            ActionContext.SelectedData = newItem;
            StateHasChanged();
        }
    }

    protected string FormatSelectedRowStyle(TEntity item, int index)
    {
        return item == ActionContext.SelectedData
            ? "background-color: var(--mud-palette-action-disabled-background) !important; color:
            var(--mud-palette-text-primary) !important; font-weight: 500;"
            : string.Empty;
    }

    public Task ReloadServerDataAsync() => _grid != null ? _grid.ReloadServerData() :
        Task.CompletedTask;
}

```

```

    public void Dispose()
    {
        if (ActionContext != null)
        {
            ActionContext.OnContextUpdated -= StateHasChanged;
            ActionContext.OnRefreshRequested -= HandleRefreshRequested;
        }
    }
}
</file>

<file path="Promatis.Net.UI/Components/Toolbar/IHasSelectedData.cs">
namespace Promatis.Net.UI.Components.Toolbar;
{
    /// <summary>
    /// B4=C,,2CT@D 0C'LC0KC' 8C0BCT@DD5C"A-CÄ0D :CT@ CD;Dò 2C,,7D40C'8Ct0D$>D >C"Â :CäBCä@D'< C05Cä1DT>CD8CÄ> D
    /// D 8C0ED >C08Ct8D >C$0D$L DD>C=C AD$@Cä:C,ôCct;C 8 Cö>C'CDt0D$L C,,<C0CC'LD K C05D 5D 8D >C$:C,i
    /// </summary>
    public interface IHasSelectedData<TEntity> where TEntity : class
    {
        TEntity? SelectedData { get; set; }
        Action? OnContextUpdated { get; set; }
    }
}
</file>

<file path="Promatis.Net.UI/Components/Toolbar/IHasToolbar.cs">
namespace Promatis.Net.UI.Components.Toolbar;
{
    /// <summary>
    /// A,,=D$5D DCT9D Ô<C @C=5D 4C'0 Cä1C'0D BCT9, C,,<CTND"8DR :Cä<C =CD=D'9 D$CC'1C @ D4?D 0C$;CT=C,,0D
    /// </summary>
    public interface IHasToolbar
    {
        ToolbarPosition Position { get; set; }
        bool IsToolbarVisible { get; set; }
    }
}
</file>

<file path="Promatis.Net.UI/Components/Toolbar/ToolbarActionContext.cs">
using Promatis.Net.UI.Components.Workspace;
{
    namespace Promatis.Net.UI.Components.Toolbar;
    {
        /// <summary>
        /// A4;Cä1C ;DÄ=D'9 C= >C0BCT:D B D4?D 0C$;CT=C,,0 C= >CÄ0C04C0>C' ?C =CT;DÄN (D$CC'1C @Cä<).D
        /// A0>C'=CäAD$LDä 0C AD$@C 3C,,@Cä2C = CäB C= >C0:D 5D$=D'E D ?CäACä1Cä2 C0>C'CDt5C08Dò 4C =C0KDR 8 D$8C0>C" 2
        /// </summary>
        public abstract class ToolbarActionContext<TEntity> : WorkspaceActionContext, IHasToolbar,
            IHasSelectedData<TEntity>
        {
            where TEntity : class
            {
                // B 5C ;C,,7C FC,,0 C= >C0BD 0C=BC "† 5Fö0Æ& -
                public ToolbarPosition Position { get; set; } = ToolbarPosition.Top;
                public bool IsToolbarVisible { get; set; } = true;
            }

            private TEntity? _selectedData;

            /// <summary>
            /// B$5C=CD"8C' 2D'4CT;CT=C0KC' >C JCT:D" =C ECä;D BCR ,,AD$@Cä:C 3D 8CD0, C$5D$:C 4CT@CT2C Ä 4C Bdt8C0
            /// A0@C, 8Ct<CT=CT=C,,8 C 2D$>CÄ0D$8Dt5D :C, ?CT@CTADt8D$KC$0CTB CD>D BD4?C0>D BDÄ 1C 7Cä2D'E C= >Cä?Cä: C
            /// </summary>
            public TEntity? SelectedData
            {
                get => _selectedData;
                set
                {
                    if (!EqualityComparer<TEntity>.Default.Equals(_selectedData, value))
                    {
                        _selectedData = value;
                        RecalculateButtonStates();
                        NotifyUpdate(); // BD>D AC,,@D45D" <C4=Cä2CT=C0CDä ?CT@CT@C,,ACä2C=C D$CC'1C @C 8 C0>CDAC$5D$>
                    }
                }
            }
        }
    }
}

```

```

// B "A "A,, 'AT!A Bò A !B$ Aä A A$ AD AÄ B "A, AÔ Aô A-
public bool IsCreateVisible { get; set; } = true;ð
public bool IsEditVisible { get; set; } = true;ð
public bool IsDeleteVisible { get; set; } = true;ð
ð
// AD AÔ AÄ Bt B A,, B B 'AT" AD B "B4 AÔ B "A, AÔ Aô A¢ ...&V BÔöæÇ'•
ð
/// <summary>ð
/// B >Ct4C =C,,5 Cò> D4<Cä;Dt0C08Dâ 4CäAD$CCô=Câ 2D 5C44C ,,5D ;C, :CÔ>Cô:C 2C,,4C,,<C 'i
/// </summary>ð
public virtual bool IsCreateEnabled { get; set; } = true;ð
ð
/// <summary>ð
/// A,,7CÄ5C05C08CR 4CäAD$CCô=CâÂ 5D ;C, >C JCT:D" 2D'1D 0C0 Cò@CT4C,,:C B C 8Ct=CTA-C'>C48C=8 D 0Ct@CTH
/// </summary>ð
public virtual bool IsEditEnabled => SelectedData != null && CanEditNode(SelectedData);ð
ð
/// <summary>ð
/// B44C ;CT=C,,5 CD>D BD4?CÔ>, CTAC'8 Cä1D=5C= B C$KC @C = A, ?D 5CD8C=0D" 1C,,7C05D Ô;Cä3C,,:C, @C 7D 5D,,0C
/// </summary>ð
public virtual bool IsDeleteEnabled => SelectedData != null && CanDeleteNode(SelectedData);ð
ð
/// <summary>ð
/// A= >C';CT:Dd8Dò @C AD,,8D 5CÔ=D'E (C=0D BCä<CÔKDR' :CÔ>Cô>C¢ BD4;C 0D 0 (Cô5Dt0D$L, DÔ:D ?Cä@D"Â 8CÄ?Cä
/// </summary>ð
private readonly List<ToolbarCustomAction> _customActions = [];ð
ð
/// <summary>ð
/// A 5Ct>C00D =C 0 C= >C';CT:Dd8Dò :C AD$>CÄ=D'E CD5C"AD$2C,,9, CD>D BD4?CÔ0Dò BD4;C 0D C D$>C'LC= > CD;Dò
/// A,,AC=;DäGC 5D" AC'CDt0C"=D4N Cò>D GD2 8C'8 CäGC,,AD$:D2 AC08D :C :CÔ>Cô>C¢ 8Ct2CÔ5.D
/// </summary>ð
public IReadOnlyCollection<ToolbarCustomAction> CustomActions => _customActions.AsReadOnly();ð
ð
protected ToolbarActionContext()ð
{ð
    // Aô> D4<Cä;Dt0C08Dâ =C AD$@C 8C$0CT< C= >CÄ?C :D$=D4N C45Cä<CTBD 8Dâ 4C'O D$CC'1C @CÔKDR AD$@C =C,,FD
    WorkspaceHeight = "100%";ð
    TopZoneHeight = "auto";ð
    BottomZoneHeight = "auto";ð
    LeftZoneWidth = "220px";ð
    RightZoneWidth = "220px";ð
}ð
ð
/// <summary>ð
/// At0D"8D"5CÔ=D'9 CÄ5D$>CB 4C'O C 5Ct>C00D =Cä3Cä 4Cä1C 2C'5C08Dò :C AD$>CÄ=D'E C= =Cä?Cä: C,,7 C= >CÔAD$@
/// A40D 0C0BC,,@D45D" f -ÄÖf 7B ?Cä2CT4CT=C,,5 C, 7C IC,,IC 5D" AC,,AD$5CÄC CäB CDCC ;C,,@Cä2C =C,,0 C,,4CT=D$8
/// </summary>ð
protected void AddCustomAction(ToolbarCustomAction action)ð
{ð
    if (action == null) throw new ArgumentNullException(nameof(action));ð
    // At0D"8D$0 CäB C05C$=C,,<C BCT;DÄ=CäAD$8 D 0Ct@C 1CäBDt8C=0: Id C= =Cä?Cä: CÔ0 Cä4CÔ>C' AD$@C =C,,FCR
    if (_customActions.Any(a => a.Id == action.Id))ð
    {ð
        throw new InvalidOperationException(ð
            $"A=0D BCä<CÔ0Dò :CÔ>Cô:C A C,,4CT=D$8DD8C=0D$>D >CÄ w¶ 7F-öää-Gòr Ccd5 Ct0D 5C48D BD 8D >C0
        )ð
    }ð
    _customActions.Add(action);ð
}ð
ð
/// <summary>ð
/// Aô@Cä3D 0CÄ<CÔ>CR 8Ct<CT=CT=C,,5 CD>D BD4?CÔ>D BC, :C AD$>CÄ=Cä9 C= =Cä?C=8 Cò> CTQ Id C,,7 C 8Ct=CTA-C'
/// </summary>ð
public void SetActionEnabled(string actionId, bool isEnabled)ð
{ð
    // A,,ICT< C$=D4BD 8 Ct0C=@D'BCä3Cä AC08D :C
    ToolbarCustomAction? action = _customActions.Find(a => a.Id == actionId);ð
    if (action != null && action.IsEnabled != isEnabled)ð
    {ð
        action.IsEnabled = isEnabled;ð
        NotifyUpdate(); // B 5C :D$8C$=Cä >C =Cä2C'OCT< UIð
    }ð
}ð
ð
// B A B$ A$ A / BD A',B$ A &A,,/ B$ A At A= &A,, B #A Aô B$ Aô# B #B" Aä!B$ ð

```



```

Ð
public override void HandleGlobalEntityCommit(object? state, object? entity)Ð
{Ð
    base.HandleGlobalEntityCommit(state, entity);Ð
Ð
    if (entity == null) return;Ð
Ð
    Type entityType = entity.GetType();Ð
    if (entityType.BaseType != null && entityType.Namespace == "Castle.Proxies")Ð
    {Ð
        entityType = entityType.BaseType;Ð
    }Ð
Ð
    if (typeof(TEntity).IsAssignableFrom(entityType))Ð
    {Ð
        string stateStr = state?.ToString() ?? string.Empty;Ð
Ð
        if (stateStr == "Deleted" || stateStr == "SoftDeleted")Ð
        {Ð
            if (SelectedData != null && ReferenceEquals(SelectedData, entity))Ð
            {Ð
                SelectedData = null;Ð
            }Ð
        }Ð
    }Ð
    RequestRefresh();Ð
}Ð
}Ð
Ð
// AD AÄ AÖ B´ Aö AT A,, A "B²0%B4 A, ,, CT@CT>Cö@CT4CT;DöND$ADö 2 Cö@C,,:C´0CD=D´E C¤>CöBCT:D BC E)Ð
protected virtual bool CanEditNode(TEntity node) => true;Ð
protected virtual bool CanDeleteNode(TEntity node) => true;Ð
Ð
/// <summary>Ð
/// A$=D4BD 5Cö=C,,9 DTCC¢ 6C,,7Cö5Cö=Cä3Cä FC,,:C´0 CD;Dö ?D >C$5CD5Cö8Dö 4Cä?Cä;Cö8D$5C´LCöKDR @C ADt5D$>C
/// </summary>Ð
protected virtual void RecalculateButtonStates()Ð
{Ð
}Ð
}
</file>

<file path="Promatis.Net.UI/Components/Toolbar/ToolbarCustomAction.cs">
using MudBlazor;Ð
Ð
namespace Promatis.Net.UI.Components.Toolbar;Ð
Ð
/// <summary>Ð
/// A4;Cä1C ;DÄ=D´9 Cö;C BDD>D <CT=CöKC´ :C´0D A CD;Dö 4CT:C´0D 0D$8C$=Cä3Cä >Cö8D 0Cö8Dö @C AD,,8D 5Cö=D´E (C
/// Aä1CTACö5Dt8C$0CTB Dd2CTBCä2Cä5 C, 2C,,7D40C´LCö>CR 5CD8Cö>Cä1D 0Ct8CR 2Cä 2D 5DR ?D 8C¤;C 4CöKDR <Cä4D4;D
/// </summary>Ð
public class ToolbarCustomActionÐ
{Ð
    /// <summary>Ð
    /// B4=C,,:C ;DÄ=D´9 D BD >C¤>C$KC´ 8CD5CöBC,,DC,,:C BCä@ CD5C"AD$2C,,0 (Cö0Cö@C,,<CT@, "excel_export").Ð
    /// A,,ACö>C´LCtCCTBD 0 CD;Dö ?D >C4@C <CÄ=Cä3Cä 8Ct<CT=CT=C,,0 D >D BCäOCö8Dö :Cö>Cö:C, ,,2C¤;DäGCT=C,,5/C$K
    /// </summary>Ð
    public required string Id { get; init; }Ð
Ð
    /// <summary>Ð
    /// B$5C¤AD" ,,=C 8CÄ5Cö>C$0Cö8CR´Ä >D$>C @C 6C 5CÄKC´ =C :Cö>Cö:CR BD4;C 0D 0.Ð
    /// </summary>Ð
    public required string Title { get; init; }Ð
Ð
    /// <summary>Ð
    /// A4@C DC,,GCTAC¤0Dö 8C¤>Cö:C xVD&Æ |÷" ,,=C ?D 8CÄ5D Ä -6öç2äö FW&- Ääf-ÆÆVBäF÷væÆö B´i
    /// </summary>Ð
    public string? Icon { get; set; }Ð
Ð
    /// <summary>Ð
    /// Bd2CTBCä2C 0 D ECT<C :Cö>Cö:C, 8Cr AC,,AD$5CÄ=Cä9 Cö0C´8D$@D² xVD&Æ |÷" ... &-ö ´´Ä 7V66W72Ä W'&÷"Ä v &
    /// </summary>Ð
    public Color Color { get; set; } = Color.Default;Ð
Ð
    /// <summary>Ð
    /// B BC,,;DÄ >D$>C @C 6CT=C,,0 C¤=Cä?C¤8 (Filled, Outlined, Text). Aö> D4<Cä;Dt0Cö8Dä r 7C ;C,,BC 0 (Filled

```

```

    /// </summary>
    public Variant Variant { get; set; } = Variant.Filled;
}

    /// <summary>
    /// BD;C 3 C$8CD8CÄ>D BC, :CÔ>Cô:C, =C BD4;C 0D 5. ATAC´8 false – Cα=Cä?Cα0 CÔ>C´=CäAD$LDâ 8D :C´NDt0CTB
    /// </summary>
    public bool IsVisible { get; set; } = true;
}

    /// <summary>
    /// BD;C 3 CD>D BD4?CÔ>D BC, :CÔ>Cô:C, 4C´0 Cα;C,,C ,,0CαBC,,2CÔ0/Ct0D$CD,,5CÔ0).
    /// </summary>
    public bool IsEnabled { get; set; } = true;
}

    /// <summary>
    /// B AD´;Cα0 CÔ0 D 5C ;DÄ=D´9 C AC,,=DT@Cä=CÔKC´ <CTBCä4 C 8Ct=CTA-C´>C48Cα8 Cα>CÔBCT:D BC Â :CäBCä@D´9 C
    /// </summary>
    public required Func<Task> OnExecute { get; init; }
}
</file>

<file path="Promatis.Net.UI/Components/Toolbar/ToolbarPosition.cs">
namespace Promatis.Net.UI.Components.Toolbar;
{
    public enum ToolbarPosition
    {
        Top,
        Bottom,
        Left,
        Right
    }
}
</file>

<file path="Promatis.Net.UI/Components/Toolbar/UiToolbar.razor">
@typeparam TEntity where TEntity : class
{
    @if (ActionContext != null && ActionContext.IsToolbarVisible)
    {
        @if (IsVertical)
        {
            <div class="d-flex flex-column align-stretch justify-space-between bg-light pa-3 rounded
            shadow-sm h-100">
                style="@($width: {ZoneSize}; min-width: {ZoneSize}; max-width: {ZoneSize};)">
                <div class="d-flex flex-column gap-2">
                    <MudText Typo="Typo.subtitle2" Class="font-weight-bold text-center border-bottom
                    pb-2 mb-2">
                        @ActionContext.PageTitle
                    </MudText>
                    @RenderButtonsCollection
                </div>
            </div>
        }
        else
        {
            <div class="d-flex align-center justify-space-between bg-light pa-2 rounded shadow-sm w-100">
                style="@($height: {ZoneSize}; min-height: {ZoneSize}; max-height: {ZoneSize};)">
                <div class="d-flex align-center gap-2 flex-wrap">
                    @RenderButtonsCollection
                </div>
            </div>
        }
    }
}
</file>

<file path="Promatis.Net.UI/Components/Toolbar/UiToolbar.razor.cs">
using Microsoft.AspNetCore.Components;
using Microsoft.AspNetCore.Components.Web;
using MudBlazor;
using Promatis.Net.UI.Components.Tree;
{
    namespace Promatis.Net.UI.Components.Toolbar;
    {
        public partial class UiToolbar<TEntity> : ComponentBase where TEntity : class
        {
            [Parameter] public ToolbarActionContext<TEntity> ActionContext { get; set; } = null!;
            [Parameter] public RenderFragment? AdditionalContent { get; set; }
        }
    }
}

```

```

[Parameter] public EventCallback OnCreateTriggered { get; set; }
[Parameter] public EventCallback<TEntity> OnEditTriggered { get; set; }
[Parameter] public EventCallback<TEntity> OnDeleteTriggered { get; set; }
[Parameter] public EventCallback<TEntity> OnCreateChildTriggered { get; set; }

protected bool IsVertical => ActionContext?.Position == ToolbarPosition.Left ||
ActionContext?.Position == ToolbarPosition.Right;

protected string ZoneSize => ActionContext?.Position switch
{
    ToolbarPosition.Left => ActionContext.LeftZoneWidth,
    ToolbarPosition.Right => ActionContext.RightZoneWidth,
    ToolbarPosition.Top => ActionContext.TopZoneHeight,
    ToolbarPosition.Bottom => ActionContext.BottomZoneHeight,
    _ => "auto"
};

protected override void OnInitialized()
{
    base.OnInitialized();

    if (ActionContext == null)
    {
        throw new ArgumentNullException(nameof(ActionContext),
            $"A<C?C=CT=D" ¶æ ÖVöb...V•Föö& #ADVçF-G"â-Ö BD 5C CCTB Că1Dô7C BCT;DÄ=Că9 Cô5D 5CD0Dt8 Cô0D
    }

    ActionContext.OnContextUpdated = StateHasChanged;
}

protected async Task OnCreateClick(MouseEventArgs e)
{
    if (OnCreateTriggered.HasDelegate) await OnCreateTriggered.InvokeAsync();
}

protected async Task OnCreateChildClick(MouseEventArgs e)
{
    if (OnCreateChildTriggered.HasDelegate && ActionContext.SelectedData != null)
        await OnCreateChildTriggered.InvokeAsync(ActionContext.SelectedData);
}

protected async Task OnEditClick(MouseEventArgs e)
{
    if (OnEditTriggered.HasDelegate && ActionContext.SelectedData != null)
        await OnEditTriggered.InvokeAsync(ActionContext.SelectedData);
}

protected async Task OnDeleteClick(MouseEventArgs e)
{
    if (OnDeleteTriggered.HasDelegate && ActionContext.SelectedData != null)
        await OnDeleteTriggered.InvokeAsync(ActionContext.SelectedData);
}

/// <summary>
/// AD8C00CĂ8Dt5D :C,,9 DD@C 3CĂ5C0B CăBD 8D >C$:C, :C0>C0>C4Â 8C0:C ?D CC'8D >C$0C0=D'9 C" 220:C'0D ACR 1
/// </summary>
private RenderFragment RenderButtonsCollection => builder =>
{
    bool isTreeMode = ActionContext.GetType().BaseType != null
        && ActionContext.GetType().BaseType!.IsGenericType
        && ActionContext.GetType().BaseType!.GetGenericTypeDefinition() ==
typeof(TreeActionContext<>);

    bool isCreateChildVisible = false;
    bool isCreateChildEnabled = false;

    // ATAC'8 D 0C0BC 9CĂ >C0@CT4CT;C,,; DtBCă ?CT@CT4 C00CĂ8 C= C0BCT:D B CD5D 5C$0 - C,,7C$;CT:C 5CĂ <CT
    if (isTreeMode)
    {
        dynamic? treeContext = ActionContext;
        if (treeContext != null)
        {
            isCreateChildVisible = treeContext.IsCreateChildVisible;
            isCreateChildEnabled = treeContext.IsCreateChildEnabled;
        }
    }
}

```

```

    int seq = 0;

    if (ActionContext.IsCreateVisible)
    {
        builder.OpenComponent<MudButton>(seq++);
        builder.AddAttribute(seq++, "Variant", Variant.Filled);
        builder.AddAttribute(seq++, "Color", Color.Primary);
        builder.AddAttribute(seq++, "Size", Size.Small);
        builder.AddAttribute(seq++, "Disabled", !ActionContext.IsCreateEnabled);
        builder.AddAttribute(seq++, "OnClick",
EventCallback.Factory.Create<MouseEventArgs>(this, OnCreateClick));
        builder.AddAttribute(seq++, "ChildContent", (RenderFragment)(b => b.AddContent(seq++,
"B >Ct4C BDÄ""½
        builder.CloseComponent();
    }

    // B AÔ AT A Aä A AD B A$ :
    if (isCreateChildVisible)
    {
        builder.OpenComponent<MudButton>(seq++);
        builder.AddAttribute(seq++, "Variant", Variant.Outlined);
        builder.AddAttribute(seq++, "Color", Color.Primary);
        builder.AddAttribute(seq++, "Size", Size.Small);
        builder.AddAttribute(seq++, "Disabled", !isCreateChildEnabled);
        builder.AddAttribute(seq++, "OnClick",
EventCallback.Factory.Create<MouseEventArgs>(this, OnCreateChildClick));
        builder.AddAttribute(seq++, "ChildContent", (RenderFragment)(b => b.AddContent(seq++, "AD>C 0C$8D
CÔ>CDCct5C²""½
        builder.CloseComponent();
    }

    if (ActionContext.IsEditVisible)
    {
        builder.OpenComponent<MudButton>(seq++);
        builder.AddAttribute(seq++, "Variant", Variant.Outlined);
        builder.AddAttribute(seq++, "Color", Color.Info);
        builder.AddAttribute(seq++, "Size", Size.Small);
        builder.AddAttribute(seq++, "Disabled", !ActionContext.IsEditEnabled);
        builder.AddAttribute(seq++, "OnClick",
EventCallback.Factory.Create<MouseEventArgs>(this, OnEditClick));
        builder.AddAttribute(seq++, "ChildContent", (RenderFragment)(b => b.AddContent(seq++,
"A,,7CÄ5CÔ8D$L"))));
        builder.CloseComponent();
    }

    if (ActionContext.IsDeleteVisible)
    {
        builder.OpenComponent<MudButton>(seq++);
        builder.AddAttribute(seq++, "Variant", Variant.Outlined);
        builder.AddAttribute(seq++, "Color", Color.Error);
        builder.AddAttribute(seq++, "Size", Size.Small);
        builder.AddAttribute(seq++, "Disabled", !ActionContext.IsDeleteEnabled);
        builder.AddAttribute(seq++, "OnClick",
EventCallback.Factory.Create<MouseEventArgs>(this, OnDeleteClick));
        builder.AddAttribute(seq++, "ChildContent", (RenderFragment)(b => b.AddContent(seq++,
"B44C ;C,,BDÄ""½
        builder.CloseComponent();
    }

    if (AdditionalContent != null)
    {
        if (ActionContext.IsCreateVisible || isCreateChildVisible ||
ActionContext.IsEditVisible || ActionContext.IsDeleteVisible)
        {
            builder.OpenComponent<MudDivider>(seq++);
            builder.AddAttribute(seq++, "Vertical", !IsVertical);
            builder.AddAttribute(seq++, "FlexItem", true);
            builder.AddAttribute(seq++, "Class", "mx-2 my-1");
            builder.CloseComponent();
        }
        builder.AddContent(seq++, AdditionalContent);
    }

    if (ActionContext.CustomActions.Any(a => a.IsVisible) &&
(ActionContext.IsCreateVisible || isCreateChildVisible || ActionContext.IsEditVisible

```

```

|| ActionContext.IsDeleteVisible || AdditionalContent != null))ð
    {ð
        builder.OpenComponent<MudDivider>(seq++);ð
        builder.AddAttribute(seq++, "Vertical", !IsVertical);ð
        builder.AddAttribute(seq++, "FlexItem", true);ð
        builder.AddAttribute(seq++, "Class", "mx-2 my-1");ð
        builder.CloseComponent();ð
    }ð
ð
    foreach (ToolbarCustomAction action in ActionContext.CustomActions.Where(a => a.IsVisible))ð
    {ð
        builder.OpenComponent<MudButton>(seq++);ð
        builder.AddAttribute(seq++, "Variant", action.Variant);ð
        builder.AddAttribute(seq++, "Color", action.Color);ð
        builder.AddAttribute(seq++, "Size", Size.Small);ð
        builder.AddAttribute(seq++, "StartIcon", action.Icon);ð
        builder.AddAttribute(seq++, "Disabled", !action.IsEnabled);ð
        builder.AddAttribute(seq++, "OnClick",
EventCallback.Factory.Create<MouseEventArgs>(this, action.OnExecute));ð
        builder.AddAttribute(seq++, "ChildContent", (RenderFragment)(b => b.AddContent(seq++,
action.Title)));ð
        builder.CloseComponent();ð
    }ð
    };ð
}
</file>

<file path="Promatis.Net.UI/Components/Tree/TreeActionContext.cs">
using Promatis.Net.Domain.Interface;ð
using Promatis.Net.UI.Components.Toolbar;ð
ð
namespace Promatis.Net.UI.Components.Tree;ð
ð
/// <summary>ð
/// A 0Ct>C$KC' 8CT@C @DT8Dt5D :C,,9 (CD@CT2Cä2C,,4C0KC'' :Cä=D$5C=AD" CC0@C 2C'5C08D0 ?C'0D$DCä@CÄK.ð
/// A0>C'=CäAD$LDä 8C0:C ?D CC'8D CCTB D >D BCä0C08CR 1C 7Cä2D'E C=äCä?Cä: C, DCä:D4AC 4C'O MudTreeView.ð
/// </summary>ð
/// <typeparam name="TEntity">B$8C0 4D 5C>C$8CD=Cä9 D CD'=CäAD$8, D 5C ;C,,7D4ND"5C' •G&VTæöFSÄ÷G- W & Óí
public abstract class TreeActionContext<TEntity> : ToolbarActionContext<TEntity>ð
    where TEntity : class, ITreeNode<TEntity>, IDomainObjectHasKey<Guid>ð
{ð
    // B BC BC,,GCTAC=0D0 =C AD$@Cä9C=0 C$8CD8CÄ>D BC, 4CäGCT@C05C' :C0>C0:C•
    public bool IsCreateChildVisible { get; set; } = true;ð
ð
    // AD8C00CÄ8Dt5D :C,,9 D 0D GCTB CD>D BD4?C0>D BC, :C0>C0:C, 4Cä1C 2C'5C08D0 ?Cä4D47C'0ð
    public virtual bool IsCreateChildEnabled => SelectedData != null;ð
ð
    /// <summary>ð
    /// A0;C BDD>D <CT=C0KC' 1D >C=5D 4C =C0KDRÂ 0CD0C0BC,,@Cä2C =C0KC' ?Cä4 C,,5D 0D EC,,GCTAC=8CR AD$@D4:D$CD
    /// </summary>ð
    public UiDataBroker<TEntity> DataBroker { get; } = new();ð
ð
    protected TreeActionContext() : base()ð
    {ð
        // A,,!A0 A A' A0 : A05D 5C0>D 8CÄ :Cä<C =CD=D'9 D$CC'1C @ C00C$5D E (C4>D 8Ct>C0BC ;DÄ=C 0 C00C05C'L
        Position = ToolbarPosition.Top;ð
        TopZoneHeight = "auto";ð
ð
        // B EC'>C0KC$0CT< C'5C$CDä 7Cä=D2Â BC : C=0C0 BD4;C 0D CD,,5C² =C 2CT@D]
        IsLeftZoneCollapsed = true;ð
        LeftZoneWidth = "0px";ð
ð
        // A0> D4<Cä;Dt0C08Dä 4CT@CT2Cä 8C08Dd8C ;C,,7C,,@D45D$AD0 2 D 5Cd8CÄ5 D 0C >D$K D >C05D 0D$8C$=Cä9 C0
        DataBroker.ConfigureInMemoryMode();ð
    }ð
ð
    /// <summary>ð
    /// Aä1D07C BCT;DÄ=D'9 CD;D0 @CT0C'8Ct0Dd8C, <CTBCä4 C0@Cä3D 5C$0 Aä B20:D0HC ä
    /// ASKtKC$0CTBD 0 C 2D$>CÄ0D$8Dt5D :C, :Cä<C0>C05C0BCä< TreePage C0@C, ?CT@C$8Dt=Cä9 CäBD 8D >C$:CRí
    /// </summary>ð
    public abstract Task InitializeInMemoryTreeAsync();ð
ð
    /// <summary>ð
    /// A 2D$>CÄ0D$8Dt5D :C,,9 C05D 5DT2C B C=äCä<C,,BCä2 B #A C00 D4@Cä2C05 C,,5D 0D EC,,GCTAC=äC4> D04D 0.ð
    /// Aä1CTAC05Dt8C$0CTB D :C$>Ct=D4N real-time D 8C0ED >C08Ct0Dd8Dä AC$0Ct5C' 2CTBCä: C" At# Ct0 0 CÄA.ð
    /// </summary>ð

```

```

public override void HandleGlobalEntityCommit(object? state, object? entity)
{
    if (entity == null) return;

    Type entityType = entity.GetType();
    if (entityType.BaseType != null && entityType.Namespace == "Castle.Proxies")
    {
        entityType = entityType.BaseType;
    }

    // ATAC'8 D$@C =Ct0C=FC,,0 C,,7 C MC=5C04C 7C BD >C0CC'0 C00D, 4D 5C$>C$8CD=D'9 D$8C0 4C =C0KD]
    if (typeof(TEntity).IsAssignableFrom(entityType))
    {
        string stateStr = state?.ToString() ?? string.Empty;

        // A08C00CT< Aä B204C$8Cd>C$ 1D >C=5D 0 C05D 5D GC,,BC BDÂ AC$0Ct8 CÄ0D AC,,2C 2 C00CÄ0D$8D
        DataBroker.ApplyIncrementalOzuDelta(stateStr, (TEntity)entity);

        // A$KcTKC$0CT< C 0Ct>$KCR ?D 0C$8C'0 D 1D >D 0 DD>C=CD 0 (CTAC'8 C$KCD5C'5C0=C 0 C$5D$:C 1D';C
        base.HandleGlobalEntityCommit(state, entity);

        // A08C03C 5CÄ :Cä<C0>C05C0B TreePage C0>C'=CäAD$LDä ?CT@CT@C,,ACä2C BDÂ 4CT@CT2Cä =C MC=0C =C]
        RequestRefresh();
    }
}

protected override void RecalculateButtonStates()
{
    base.RecalculateButtonStates();
    NotifyUpdate(); // B 5C :D$8C$=Cä >C =Cä2C'0CT< C :D$8C$=CäAD$L C=äCä?C=8 "AD>C 0C$8D$L C0>CDCct5C²"
}
}
</file>

<file path="Promatis.Net.UI/Components/Tree/TreePage.razor">
@typeparam TEntity where TEntity : class, ITreeNode<TEntity>, IDomainObjectHasKey<Guid>
{
    <div class="tree-page-container w-100 overflow-auto pa-2"
        style="height: calc(100vh - 240px); min-height: calc(100vh - 240px); max-height: calc(100vh - 240px);">
        @if (_isWarmingUp)
        {
            <div class="d-flex flex-column align-center justify-center h-100 w-100 gap-2" style="min-height: 200px;">
                <MudProgressCircular Color="Color.Primary" Indeterminate="true" />
                <MudText Typo="Typo.body2" Color="Color.Secondary">At0C4@D47C=0 D BD CC=BD4@D² 4CT@CT2C äääÂô×VEF
            </div>
        }
        else
        {
            <MudTreeView T="TEntity"
                Items="@_rootTreeViewItems"
                SelectedValue="@ActionContext.SelectedData"
                SelectedValueChanged="@((TEntity? val) => OnSelectedNodeChanged(val))"
                Hover="true"
                Dense="true"
                Color="Color.Primary"
                Class="w-100">
                <ItemTemplate>
                    @* BD8C=AC,,@D45CÄ :C =Cä=C,,GC0KC' HC 1C'>C0 CCT;C 4C'0 MudBlazor 9.4+ *@
                    <MudTreeViewItem Value="@context.Value"
                        Items="@context.Children"
                        Text="@GetNodeText(context.Value!)"
                        Expanded="true" />
                </ItemTemplate>
            </MudTreeView>
        }
    </div>
</file>

<file path="Promatis.Net.UI/Components/Tree/TreePage.razor.cs">
using Microsoft.AspNetCore.Components;
using MudBlazor;
using Promatis.Net.Domain.Interface;

```

```

namespace Promatis.Net.UI.Components.Tree;
public partial class TreePage<TEntity> : ComponentBase, IDisposable
    where TEntity : class, ITreeNode<TEntity>, IDomainObjectHasKey<Guid>
{
    [CascadingParameter] protected TreeActionContext<TEntity> ActionContext { get; set; } = null!;

    [Parameter] public Func<TEntity, string>? NodeTextSelector { get; set; }

    // A>C';CT:Dd8Dò >C 5D BCä: CD;Dò ×VEG&VUf-Wr 'äm
    protected List<TreeItemData<TEntity>> _rootTreeViewItems = [];

    // A Bâ'AT Ä Ä Ä Ä": A,,7C00Dt0C'LC0> C$KD BC 2C'0CT< C" G'VRâ
    // B BD 0C08Dd0 CÄ3C0>C$5C0=Câ ?Cä:C 6CTB D ?C,,=C05D Ä 0 CÄ5C0N C0@C,,;Cä6CT=C,,0 CäAD$0C05D$ADò ?Cä;C0>D B
    protected bool _isWarmingUp = true;

    protected override void OnInitialized()
    {
        base.OnInitialized();

        if (ActionContext == null)
        {
            throw new ArgumentNullException(nameof(ActionContext),
                $"A>CÄ?Cä=CT=D" ¶æ ÖVöb...G&VU vSÄDVçF-G"â-0 BD 5C CCTB C00C'8Dt8Dò ¶æ ÖVöb...G&VT 7F-öä6öçFW‡C
            );
        }

        ActionContext.OnContextUpdated += HandleContextUpdated;
        ActionContext.OnRefreshRequested += HandleRefreshRequested;
    }

    /// <summary>
    /// A$KctKC$0CTBD O C 2D$>CÄ0D$8Dt5D :C, !B$ Ää Ää Ää!A' D$>C4>, C=0C¢ &Æ |÷" >D$@C,,ACä2C ; C0CD BCä9 C
    /// A0>C'=CäAD$LDâ 8D :C'NDt0CTB DD@C,,7D² <CT=Dâ ?D 8 C05D 5DT>CD5 C0> C$:C'0CD:C < [6].
    /// </summary>
    protected override async Task OnAfterRenderAsync(bool firstRender)
    {
        await base.OnAfterRenderAsync(firstRender);

        if (firstRender)
        {
            // At0C0CD :C 5CÄ ?D >C4@CT2 C AC,,=DT@Cä=C0>, CäAC$>C >Cd4C O UI-C0>D$>C-
            await WarmupTreeDataAsync();
        }

        /// <summary>
        /// A0@Cä8Ct2Cä4C,,B C 5Ct>C00D =D4N DD>C0>C$CDâ 7C 3D CCT:D2 4C =C0KDR 8Cr !B4 AB 2 Ää B20:D0H C @Cä:CT@C
        /// </summary>
        private async Task WarmupTreeDataAsync()
        {
            try
            {
                // ATAC'8 Ää B20:D0H C @Cä:CT@C 5D"5 C0CD B, C$KC0>C'=D05CÄ EC,,B C¢ ACT@C$8D C C MC=5C04C
                if (ActionContext.DataBroker.IsInMemoryMode && ActionContext.DataBroker.InMemoryItems
                    == null)
                {
                    // A$Kct>C" CC'5D$0CTB C" ABÄ ?Cä:C T' :D CD$8D" ;CT3C=8C' AC08C0=CT@D
                    await ActionContext.InitializeInMemoryTreeAsync();
                }

                // A$KC0>D 8CÄ BD06CT;D4N D 1Cä@C=C C4@C DC AC$0ct5C' BD'AD0G D47C'>C" 2 DD>C0>C$KC' ?CäBCä: Thr
                // DtBCä1D² ?CäBCä: CäBD 8D >C$:C, 2Cä>C ICR =CR 8D ?D'BD'2C ; CÄ8C=©Cä07C 4CT@Cd5C¢
                await Task.Run(RebuildTreeGraph);
            }
            finally
            {
                _isWarmingUp = false;

                // A$>Ct2D 0D"0CT< D4?D 0C$;CT=C,,5 C" T'0?CäBCä: CD;Dò 1CT7Cä?C AC0>C' ?CT@CT@C,,ACä2C=8 C4>D$>C$>
                await InvokeAsync(StateHasChanged);
            }
        }

        /// <summary>
        /// A KD BD > D >C 8D 0CTB C0;CäAC=8C' :D0H C @Cä:CT@C 2 CD@CT2Cä2C,,4C0KC' 3D 0DB G&VT-FV0F F 7C 0" <
        /// </summary>
    }
}

```

```

private void RebuildTreeGraph()
{
    if (!ActionContext.DataBroker.IsInMemoryMode || ActionContext.DataBroker.InMemoryItems ==
null)
    {
        _rootTreeViewItems = [];
        return;
    }

    List<TEntity> allItems = ActionContext.DataBroker.InMemoryItems;

    // 1. B >Ct4C 5CÂ 2D 5CÂ5C0=D4N D BD CC=BD4@D2 4C`O C00C= >C0;CT=C,,0 CD>Dt5D =C,,E D0;CT<CT=D$>C-
    var childrenMap = allItems.ToDictionary(
        item => item.Id,
        _ => new List<TreeItemData<TEntity>>());

    List<TreeItemData<TEntity>> roots = [];
    var rootsMap = new List<TEntity>();

    // 2. B =C GC ;C @C AC0@CT4CT;D05CÂ 4CâGCT@C08CR MC`5CÂ5C0BD² ?Câ AC08D :C < C" At#
    foreach (TEntity item in allItems)
    {
        // B >Ct4C 5CÂ GC,,AD$CDâ >C 5D BC=C CD;D0 BCT:D4ICT3Câ CCT;C
        var currentWrapper = new TreeItemData<TEntity>
        {
            Value = item,
            Text = GetNodeText(item),
            Expanded = true
        };

        // A0@Câ2CT@D05CÂ =C ;C,,GC,,5 C$0C`8CD=Câ3Câ @Câ4C,,BCT;Dý
        if (item.ParentId.HasValue && item.ParentId.Value != Guid.Empty &&
childrenMap.ContainsKey(item.ParentId.Value))
        {
            childrenMap[item.ParentId.Value].Add(currentWrapper);
        }
        else
        {
            roots.Add(currentWrapper);
            rootsMap.Add(item);
        }
    }

    // 3. B 5C=CD AC,,2C0> C,,;C, 8D$5D 0D$8C$=Câ AC$0ctKC$0CT< CD>Dt5D =C,,5 D ?C,,AC=8 D >C JCT:D$0CÂ8 Tre
    // A05D 5CD0CT< D ?C,,AC=8 C=0C0 ADD>D <C,,@Câ2C =C0KCR :Câ;C`5C=FC,,8, DtBCâ1D² xVD&Æ `÷" 'âB =C BC,,2C0
    foreach (var rootWrapper in roots)
    {
        PopulateChildrenTree(rootWrapper, childrenMap);
    }

    _rootTreeViewItems = roots;
}

/// <summary>
/// A$AC0>CÂ>C40D$5C`LC0KC' <CTBCâ4 CD;D0 @CT:D4@D 8C$=Câ3Câ AC$0ctKC$0C08D0 &V F0æÇ`0:Câ;C`5C=FC,,9 Child
/// </summary>
private void PopulateChildrenTree(TreeItemData<TEntity> currentWrapper, Dictionary<Guid,
List<TreeItemData<TEntity>>> childrenMap)
{
    if (currentWrapper.Value == null) return;

    Guid currentId = currentWrapper.Value.Id;

    if (childrenMap.TryGetValue(currentId, out var directChildren) && directChildren.Count > 0)
    {
        // A0@C,,AC$0C,,2C 5CÂ 3CâBCâ2D`9 D ?C,,ACâ: - MudBlazor C$=D4BD 8 D 0CÂ ?CâADt8D$0CTB Read-Only Has
        currentWrapper.Children = directChildren;

        // A,,4CT< C$3C`CC L C0> Dd5C0>Dt:CR 3D 0DD00
        foreach (var childWrapper in directChildren)
        {
            PopulateChildrenTree(childWrapper, childrenMap);
        }
    }
}

```



```

    private void HandleContextUpdated()
    {
        StateHasChanged();
    }

    private void HandleRefreshRequested()
    {
        InvokeAsync(async () =>
        {
            // A@C, &V Â×F-ÖR BD 0C07C :Dd8D0E B #A C05D 5D GC,,BD´2C 5CÂ 3D 0DB 2 DD>C0>C$>CÂ BC AC=5D
            await Task.Run(RebuildTreeGraph);
            StateHasChanged();
        });
    }

    protected void OnSelectedNodeChanged(TEntity? domainEntity)
    {
        if (domainEntity == null && ActionContext.SelectedData != null) return;

        if (!EqualityComparer<TEntity>.Default.Equals(ActionContext.SelectedData, domainEntity))
        {
            ActionContext.SelectedData = domainEntity;
            StateHasChanged();
        }
    }

    protected string GetNodeText(TEntity node)
    {
        if (node == null) return string.Empty;
        if (NodeTextSelector != null) return NodeTextSelector(node);

        var nameProp = typeof(TEntity).GetProperty("Name");
        return nameProp?.GetValue(node)?.ToString() ?? node.Id.ToString();
    }

    public void Dispose()
    {
        if (ActionContext != null)
        {
            ActionContext.OnContextUpdated -= HandleContextUpdated;
            ActionContext.OnRefreshRequested -= HandleRefreshRequested;
        }
    }
}
</file>

<file path="Promatis.Net.UI/Components/Workspace/IWorkspaceActionContext.cs">
namespace Promatis.Net.UI.Components.Workspace;
{
    /// <summary>D
    /// A,,=D$5D DCT9D Ô<C @C=5D 4C´O C 2D$>CÄ0D$8Dt5D :Cä3Cä ?Cä8D :C 8 D 5C48D BD 0Dd8C, T´Ô:Cä=D$5C=AD$>C" 2
    /// </summary>D
    public interface IWorkspaceActionContextD
    {
    }
}
</file>

<file path="Promatis.Net.UI/Components/Workspace/WorkspaceActionContext.cs">
using MudBlazor;
{
    namespace Promatis.Net.UI.Components.Workspace;
    {
        /// <summary>D
        /// A 0Ct>C$KC´ :Cä=D$5C=AD" ;Dä1Cä9 D 0C >Dt5C´ >C ;C AD$8 (C$:C´0CD:C,´ 2 D 8D BCT<CR1
        /// Aä1D"8C´ 7C00CÄ5C00D$5C´L CD;Dò <C05CÄ>D ECT<, D ?D 0C$>Dt=C,,:Cä2, CD0D,,1Cä@CD>C" 8 C4@C DC,,:Cä2.D
        /// </summary>D
        public abstract class WorkspaceActionContext : IWorkspaceActionContextD
        {
            /// <summary>D
            /// B4=C,,:C ;DÄ=D´9 C,,4CT=D$8DD8C=0D$>D @C 1CäGCT9 Cä1C´0D BC, 2 D 0C0BC 9CÄ5.D
            /// </summary>D
            public Guid WorkspaceId { get; init; } = Guid.NewGuid();

            /// <summary>D
            /// B$5C=AD$>C$KC´ 7C 3Cä;Cä2Cä: D BD 0C08DdK (CäBCä1D 0Cd0CTBD 0 C00 D$CC´1C @CR 8C´8 C$:C´0CD:CR´1

```

```

    /// </summary>
    public string PageTitle { get; set; } = "B 0C >Dt0D0 >C ;C AD$L";
}

    /// <summary>
    /// B 8D BCT<C0>CR 8CÄO CÄ>CDCC´O (Core, Mes, MesMDM), C¢ :CäBCä@Cä<D2 >D$=CäAC,,BD O DÔ:D 0C0í
    /// </summary>
    public string ModuleName { get; init; } = string.Empty;
}

    // B4 B A$ AT A,, A$ At#A BÄ B´ B "A,, AT Aô AD Aä Aº Ð
    public int PaperElevation { get; set; } = 1;
    public string PaperClass { get; set; } = "pa-4 d-flex flex-column flex-grow-1 w-100 h-100";
}

    // B4 B A$ AT A,, A4 Aä AT"B AT Aº B A !A %Ä B "A ,,!C$0CtL D v÷&·7 6U vR•
    public string WorkspaceHeight { get; set; } = "100%";
    public string TopZoneHeight { get; set; } = "auto";
    public string BottomZoneHeight { get; set; } = "auto";
    public string LeftZoneWidth { get; set; } = "250px";
    public string RightZoneWidth { get; set; } = "300px";
}

    public bool IsTopZoneCollapsed { get; set; } = false;
    public bool IsBottomZoneCollapsed { get; set; } = false;
    public bool IsLeftZoneCollapsed { get; set; } = false;
    public bool IsRightZoneCollapsed { get; set; } = false;
}

    // B A B$ A$ B´ ÄÄ B " A$ A ÄÄ AD A"!B$ A,,/ (Blazor - Aº>C0BCT:D B)
}

    /// <summary>
    /// B >C KD$8CRÄ 2D´7D´2C 5CÄ>CR ?D 8 C,,7CÄ5C05C08C, AC$>C"AD$2 D 0CÄ>C4> Cº>C0BCT:D BC ,,=C ?D 8CÄ5D Ä 3
    /// Aô@C,,=D44C,,BCT;DÄ=Cä 7C AD$0C$;Dô5D" &Æ |÷" ?CT@CT@C,,ACä2C BDÄ MC´5CÄ5C0BD²í
    /// </summary>
    public Action? OnContextUpdated { get; set; }
}

    /// <summary>
    /// B$@C,,3C45D <C4=Cä2CT=C0>C4> Cä?Cä2CTICT=C,,0 UI-D0;CT<CT=D$>C" > C$=D4BD 5C0=CT< C,,7CÄ5C05C08C, AD$5C
    /// </summary>
    public void NotifyUpdate() => OnContextUpdated?.Invoke();
}

    /// <summary>
    /// A4;Cä1C ;DÄ=Cä5 D >C KD$8CRÄ 8Ct2CTIC ND"5CR 2C´>Cd5C0=D´5 C$8CtCC ;C,,7C BCä@D² ,,3D 8CBÄ 4CT@CT2Cä´ >
    /// B @C 1C BD´2C 5D" ?D 8 Cº>CÄ0C04C E C05D 5Ct0C4@D47Cº8 C,,;C, ?Cä AC,,3C00C´0CÄ 8Cr !B4 ABí
    /// </summary>
    public event Action? OnRefreshRequested;
}

    /// <summary>
    /// A$ACô>CÄ>C40D$5C´LC0KC´ <CTBCä4 CD;Dò 1CT7Cä?C AC0>C4> Ct0C0CD :C >C =Cä2C´5C08Dò 2C´>Cd5C0=Cä3Cä :C
    /// </summary>
    protected void RequestRefresh() => OnRefreshRequested?.Invoke();
}

    /// <summary>
    /// A$KctKC$0CTBD O DT>C´AD$>CÄ v÷&·7 6U vR ?D 8 D4ACô5D,,=Cä< Cº>CÄ<C,,BCR ;Dä1Cä9 D CD"=CäAD$8 C" !B4 A
    /// Aô5D 5Cä?D 5CD5C´OCTBD O C" BC,,?C,,7C,,@Cä2C =C0KDR =C AC´5CD=C,,:C E CD;Dò BCäGCTGC0>C´ ?D >C$5D :C, BC
    /// </summary>
    /// <param name="state">B >D BCä0C08CR 8Ct<CT=CT=C,,0 C,,7 C05D 5DT2C BDt8Cº0 EF Core (EntityStateChangeEnu
    /// <param name="entity">B 0CÄ 4Cä<CT=C0KC´ >C JCT:D" ,,GC,,AD$KC´ 8C´8 C0@Cä:D 8D >C$0C0=D´9)</param>
    public virtual void HandleGlobalEntityCommit(object? state, object? entity)
    {
    }
}

    /// <summary>
    /// Bd5C0BD 0C´8Ct>C$0C0=Cä >C 5D ?CTGC,,2C 5D" FC$5D$>C$>CR 5CD8C0>Cä1D 0Ct8CR >C05D 0Dd8C´ 2Cä 2D 5DR <C
    /// Aô@C,,=C,,<C 5D" AD$@Cä:Cä2Cä5 C,,<Dò 4CT9D BC$8Dò !B4 AB 8C´8 Enum C, 2Cä7C$@C IC 5D" AC,,AD$5CÄ=D´9 Dd2
    /// </summary>
    public virtual Color GetActionColor(string? action)
    {
        if (string.IsNullOrEmpty(action)) return Color.Default;

        return action.ToLower().Trim() switch
        {
            "create" or "insert" or "added" or "CD>C 0C$;CT=C,,5" or "D >Ct4C =C,,5" => Color.Success,
            "update" or "edit" or "modified" or "C,,7CÄ5C05C08CR" ÷" $@CT4C :D$8D >C$0C08CR" óä 6öÆ÷"äv &æ-ærí
            "delete" or "remove" or "deleted" or "softdeleted" or "D44C ;CT=C,,5" => Color.Error,
            _ => Color.Default
        };
    }
}

    /// <summary>

```

```

/// B4=C,,2CT@D 0C'LC0KC' ?C'0D$DCä@CÄ5C0=D'9 C$KD >Cα>C0@Cä8Ct2Cä4C,,BCT;DÄ=D'9 CD2C,,6Cä: Cα;Cä=C,,@Cä2C =C
/// B >Ct4C 5D" BCäGC0CDä 8Ct>C'8D >C$0C0=D4N C4;D41Cä:D4N Cα>C08Dä ;Dä1Cä3Cä ?Cä;C,,<Cä@DD=Cä3Cä 4Cä<CT=C
/// A 2D$>CÄ0D$8Dt5D :C, 2D'@CT7C 5D" 1CTACα>C05Dt=D'5 ORM-C05D$;C, ... &VçB06†-ÆG&Vä' 8 Cä1DT>CD8D" >C4@C
/// </summary>D
/// <typeparam name="T">A 0Ct>C$KC' 8C'8 Cα>C0:D 5D$=D'9 D$8C0 4Cä<CT=C0>C' AD4IC0>D BCfÄ÷G- W & Óí
public T CloneEntity<T>(T entity) where T : classD
{D
    if (entity == null) throw new ArgumentNullException(nameof(entity));D
D
    var jsonOptions = new System.Text.Json.JsonSerializerOptionsD
    {D
        // A 0Ct>C$0D0 7C IC,,BC >D" AD$0C04C @D$=D'E Dd8Cα;C,,GCTACα8DR AD KC'>C-
        ReferenceHandler = System.Text.Json.Serialization.ReferenceHandler.IgnoreCycles,D
D
        // B$>D$0C'LC0KC' DC,,;DÄBD ϕ 2D'@CT7C 5CÄ ;Dä1D'5 D ;Cä6C0KCR =C 2C,,3C FC,,>C0=D'5 D 2Cä9D BC$0 CD
        TypeInfoResolver = new
System.Text.Json.Serialization.Metadata.DefaultJsonTypeInfoResolverD
    {D
        Modifiers = { typeInfo =>D
        {D
            foreach (var property in typeInfo.Properties)D
            {D
                // 1. ATAC'8 D0BCä =C 2C,,3C FC,,>C0=D'5 D 2Cä9D BC$0 C,,5D 0D EC,,9 CD5D 5C$LCT2D
                if (property.Name is "Parent" or "Children")D
                {D
                    property.ShouldSerialize = (_, _) => false;D
                    continue;D
                }D
D
                // 2. B4=C,,2CT@D 0C'LC0KC' DC,,;DÄBD 4C'O A4 A,, Aä : D
                // ATAC'8 D 2Cä9D BC$> D02C'0CTBD O D ;Cä6C0KCÄ 4Cä<CT=C0KCÄ >C JCT:D$>CÄ 1D0:CT=CD0 C,,;C
                // CÄK C05 D$0D"8CÄ 5C4> C" :C'>C0 DCä@CÄK, CäAD$0C$;D0O D$>C'LCα> C0;CäACα8CR ?D 8Cα;C 4
                Type propType = property.PropertyType;D
                bool isDomainClass = typeof(Domain.DomainObject).IsAssignableFrom(propType);D
                bool isDomainCollection = propType.IsGenericType &&D
D
                typeof(System.Collections.IEnumerable).IsAssignableFrom(propType) &&D
D
                typeof(Domain.DomainObject).IsAssignableFrom(propType.GetGenericArguments().FirstOrDefault());D
D
                if (isDomainClass || isDomainCollection)D
                {D
                    property.ShouldSerialize = (_, _) => false;D
                }D
D
            }D
        }D
    }D
};D
D
    // B 5D 8C ;C,,7D45CÄ 8 CD5D 5D 8C ;C,,7D45CÄÄ AD$@Cä3Cä ACäED 0C00D0 @CT0C'LC0KC' @C =D$0C"<-D$8C0 =C
    string json = System.Text.Json.JsonSerializer.Serialize(entity, entity.GetType(),
jsonOptions);D
    return (T)System.Text.Json.JsonSerializer.Deserialize(json, entity.GetType(), jsonOptions)!;D
}
</file>

<file path="Promatis.Net.UI/Components/Workspace/WorkspacePage.razor">
@namespace Promatis.Net.UI.Components.WorkspaceD
D
<MudPaper Elevation="@GetPaperElevation()"D
    Class="@GetPaperClass()"D
    Style="@($"width: 100%; min-height: {GetWorkspaceHeight()}; overflow: hidden;")">D
D
    @if (TopContent != null && !IsTopCollapsed())D
    {D
        <div class="workspace-zone-top w-100 mb-2"D
            style="@($"height: {GetTopHeight()}; min-height: {GetTopHeight()}; max-height:
{GetTopHeight()};")">D
            @TopContentD
        </div>D
    }D
D
    <div class="d-flex flex-row align-stretch flex-grow-1 gap-4 w-100" style="min-height: 0;">D
D
        @if (LeftContent != null && !IsLeftCollapsed())D

```

```

        {
            <div class="workspace-zone-left d-flex flex-column align-stretch justify-space-between"
                style="@($width: {GetLeftWidth()}; min-width: {GetLeftWidth()}; max-width:
{GetLeftWidth()});">
                @LeftContent
            </div>
        }

        <div class="workspace-zone-body flex-grow-1 h-100" style="min-width: 0; min-height: 0;">
            @if (BodyContent != null)
            {
                @BodyContent
            }
        </div>

        @if (RightContent != null && !IsRightCollapsed())
        {
            <div class="workspace-zone-right d-flex flex-column align-stretch justify-space-between"
                style="@($width: {GetRightWidth()}; min-width: {GetRightWidth()}; max-width:
{GetRightWidth()});">
                @RightContent
            </div>
        }

    </div>

    @if (BottomContent != null && !IsBottomCollapsed())
    {
        <div class="workspace-zone-bottom w-100 mt-2"
            style="@($height: {GetBottomHeight()}; min-height: {GetBottomHeight()}; max-height:
{GetBottomHeight()});">
            @BottomContent
        </div>
    }
}
</MudPaper>
</file>

```

```

<file path="Promatis.Net.UI/Components/Workspace/WorkspacePage.razor.cs">
using Microsoft.AspNetCore.Components;
using Promatis.Net.Data;
namespace Promatis.Net.UI.Components.Workspace;
public partial class WorkspacePage : ComponentBase, IDisposable
{
    /// <summary>
    /// A`>C$8CÄ :Cä=D$5C AD" BCT:D4ICT9 D 0C >Dt5C' >C ;C AD$8 C,,7 C AD :C 4C í
    /// ATAC`8 D BD 0C08Dd0 C AD BCä<C00D0 8 C AD C0BCT:D B C05 C0CCd5C0Ä ECä;D B CäBD 5C04CT@C,,BD O D 4CTDCä;
    /// </summary>
    [CascadingParameter] protected WorkspaceActionContext? ActionContext { get; set; }

    [Parameter] public RenderFragment? BodyContent { get; set; }
    [Parameter] public RenderFragment? TopContent { get; set; }
    [Parameter] public RenderFragment? BottomContent { get; set; }
    [Parameter] public RenderFragment? LeftContent { get; set; }
    [Parameter] public RenderFragment? RightContent { get; set; }

    protected override void OnInitialized()
    {
        base.OnInitialized();

        // A0>CD?C,,AD`2C 5CÄ 2CT@DT=C,,9 D0BC 6 C AD :C AC =C 3C`>C 0C`LC0KCR BD 0C07C :Dd8C, !B4 AM
        DatabaseTriggerService.OnEntityCommitted += HandleDatabaseChange;
    }

    /// <summary>
    /// A0@C,,=C,,<C 5D" AC,,3C00C^ :Cä<CÄ8D$0 C,,7 C05D 5DT2C BDt8C AD C, ?CT@CT4C 5D" 5C4> C" :Cä=D$5C AD" BCT:D
    /// </summary>
    private void HandleDatabaseChange(EntityStateChangeEnum state, object entity)
    {
        if (ActionContext == null) return;

        // A0@C,,=D44C,,BCT;DÄ=Cä <C @D,,0C`8D CCT< C$Kct>C" 8Cr ?CäBCä:C !B4 AB 2 C4;C 2C0KC' T'0?CäBCä: Blazo
        // B0BCä 3C @C =D$8D CCTB, DtBCä 7F FT† 46† ævVB,' 2C0CD$@C, :Cä=D$5C AD$0 C00CÄ5D BC$> Cä1C0>C$8D" :
        InvokeAsync(() =>

```

```

        {
            DbContext.HandleGlobalEntityCommit(state, entity);
        });
    }
}

/// <summary>
/// Aö>C´=Cä5 D4=C,,GD$>Cd5C08CR AC$0ct8 D BD 8C43CT@C <C, ?D 8 Ct0C@D´BC,,8 MDI-C$:C´0CD:C, ,, C IC,,BC >
/// </summary>
public void Dispose()
{
    DatabaseTriggerService.OnEntityCommitted -= HandleDatabaseChange;
}

// --- AÄ B$ AD+ B B 'AT"A A,, B4 A´,AÔ A4 B "A,, Bò Aä A´ Ad A, ÒÖÝ
protected int GetPaperElevation() => ActionContext?.PaperElevation ?? 1;
protected string GetPaperClass() => ActionContext?.PaperClass ?? "pa-4 d-flex flex-column flex-grow-1 w-100 h-100";

// --- AÄ B$ AD+ B B 'AT"A AT AÄ B$ A,, A B A !A %Ää B "A ÒÖÝ
protected string GetWorkspaceHeight() => ActionContext?.WorkspaceHeight ?? "100%";
protected string GetTopHeight() => ActionContext?.TopZoneHeight ?? "auto";
protected string GetBottomHeight() => ActionContext?.BottomZoneHeight ?? "auto";
protected string GetLeftWidth() => ActionContext?.LeftZoneWidth ?? "250px";
protected string GetRightWidth() => ActionContext?.RightZoneWidth ?? "300px";

protected bool IsTopCollapsed() => ActionContext?.IsTopZoneCollapsed ?? false;
protected bool IsBottomCollapsed() => ActionContext?.IsBottomZoneCollapsed ?? false;
protected bool IsLeftCollapsed() => ActionContext?.IsLeftZoneCollapsed ?? false;
protected bool IsRightCollapsed() => ActionContext?.IsRightZoneCollapsed ?? false;
}
</file>

<file path="Promatis.Net.UI/Dashboard.razor">
@page "/"
<MudText Typo="Typo.h4" Class="mb-4">AÄ>CDCC´L Master Data Management</MudText>
<MudGrid>
    <MudItem xs="12" sm="6" md="4">
        <MudPaper Class="d-flex flex-row pt-6 pb-4 px-4" Style="height:100px;">
            <MudIcon Icon="@Icons.Material.Filled.Storage" Color="Color.Primary" Size="Size.Large"
Class="mr-4" />
            <div>
                <MudText Typo="Typo.subtitle2" Class="mud-text-secondary">B BC BD4A Cö>CD:C´NDt5C08Dò : A </
                <MudText Typo="Typo.h6">A :D$8C$=Cä ...FW7D& 6R"Äô×VEFW†Cí
            </div>
        </MudPaper>
    </MudItem>
    <MudItem xs="12" sm="6" md="8">
        <MudPaper Class="pa-4" Style="height:100px;">
            <MudText Typo="Typo.subtitle1">AD>C @Cä ?Cä6C ;Cä2C BDÄ 2 D 8D BCT<D2 &öÖ F—2 U% Äô×VEFW†Cí
            <MudText Typo="Typo.body2" Class="mud-text-secondary">
                A$ACR 8C0BCT@DD5C"AD² ?Cä4C4@D46CT=D² 4C,,=C <C,,GCTAC=8 C,,7 C05Ct0C$8D 8CÄKDR AC´>CT2 Razor CL
            </MudText>
        </MudPaper>
    </MudItem>
</MudGrid>
</file>

<file path="Promatis.Net.UI/Dashboard.razor.css">
.my-component {
    border: 2px dashed red;
    padding: 1em;
    margin: 1em 0;
    background-image: url('background.png');
}
</file>

<file path="Promatis.Net.UI/IUiModule.cs">
namespace Promatis.Net.UI;
{
    /// <summary>
    /// A= CÖBD 0C= B CD;Dò 4C,,=C <C,,GCTAC=8DR <Cä4D4;CT9 Cö>C´Lct>C$0D$5C´LD :Cä3Cä 8C0BCT@DD5C"AC í
    /// </summary>
    public interface IUiModule

```

```

{
    /// <summary>
    /// AÄBCä1D 0Cd0CT<Cä5 C,,<Dò <Cä4D4;Dò 2 D 8D BCT<CR ,,8D ?Cä;DÄ7D45D$ADò 4C´O D :C =CT@C ´í
    /// </summary>
    string Name { get; }
}

/// <summary>
/// A$>Ct2D 0D"0CTB D ?C,,ACä: DÔ;CT<CT=D$>C" =C 2C,,3C FC,,8 CD;Dò 1Cä:Cä2Cä3Cä <CT=Dâ ×VDG& vW"í
/// Bt5D$2CT@D$KC´ ?C @C <CTBD >D$2CTGC 5D" 7C "0CD >C$=CT2D4N C4@D4?Cò8D >C$:D2 MC´5CÄ5CÔBCä2.D
/// </summary>
/// <returns>A>C´;CT:Dd8Dò :Cä@D$5Cd5C"¢ ,, C 7C$0CÔ8CRÄ !D KC´:C Â C¤>CÔ:C Â C 7C$0CÔ8CT D CCô?D²"Â÷&W
IEnumerable<(string Title, string Href, string Icon, string? Group)> GetMenuItems();
}
</file>

<file path="Promatis.Net.UI/Pages/AuditLog/AuditLogPage.razor">
@page "/audit-logs"
@
@using Promatis.Net.Domain
@
@* A$=CT4D OCT< CÔ0D, 0C$BCä<C BC,,GCTAC¤8 Ct0D 5C48D BD 8D >C$0CÔ=D´9 C" D´ :Cä=D$5C¤AD" ´?Cä;CÔ>D$K C$;C AD$
@Inject AuditLogPageContext Context
@
@* Aô@Cä:C,,4D´2C 5CÄ :Cä=D$5C¤AD" :C AC¤0CD>CÄ 2CÔ8CrÄ GD$>C K DT>C´AD" v÷&·7 6U vR =C BC,,2CÔ> Dt8D$0C² =C
<CascadingValue Value="@Context" IsFixed="true">
    <WorkspacePage>
        @
        <TopContent>
            @* B$#A´ A : Aô>C´=CäAD$LDâ 0C$BCä=Cä<CÔ0Dò AD$@CäGC¤0, D 5CÔ4CT@C,,B C¤=Cä?C¤8 C, 7C 3Cä;Cä2Cä:
            <UiToolbar TEntity="AuditLog" ActionContext="@Context">
                <AdditionalContent>
                    @* AD5C¤;C @C BC,,2CÔ> C$AD$@C 8C$0CT< Cò8C¤5D 4C B, D 2Dò7C 2 CT3Cä =C ?D OCÄCDâ A C#-D
                    <div class="d-flex align-center gap-3">
                        <MudDateRangePicker Label="Aô5D 8Cä4 C´>C4>C"-
                            @bind-DateRange="Context.Period"
                            Margin="Margin.Dense"
                            Style="width: 260px;"
                            Clearable="false" />
                    </div>
                </AdditionalContent>
            </UiToolbar>
        </TopContent>
        @
        <BodyContent>
            @* B$ A A,,&A ¢ 'C,,AD$KC´ 2C,,7D40C´8Ct0D$>D :Cä;Cä=Cä:. A,,7 CÔ5C4> D BCT@D" 2CTADÄ &ö-ÆW' Æ FRÔ:
            <GridPage TEntity="AuditLog">
                <ColumnsContent>
                    <PropertyColumn T="AuditLog" TProperty="DateTime" Property="x => x.ChangedAt"
                    Title="AD0D$0 C, 2D 5CÄ0" Format="dd.MM.yyyy HH:mm:ss" HeaderStyle="width: 250px;" Style="width: 250px;" />
                    <PropertyColumn T="AuditLog" TProperty="string" Property="x => x.EntityName"
                    Title="B CD"=CäAD$L" Style="width: 200px;" />
                    <PropertyColumn T="AuditLog" TProperty="string" Property="x => x.Action"
                    Title="Aä?CT@C FC,,0" Style="width: 120px;">
                        <CellTemplate>
                            <MudChip T="string"
                            Color="@Context.GetActionColor(context.Item.Action)" Size="Size.Small" Variant="Variant.Text">
                                @context.Item.Action
                            </MudChip>
                        </CellTemplate>
                    </PropertyColumn>
                    <PropertyColumn T="AuditLog" TProperty="string" Property="x => x.ChangedBy"
                    Title="Aô>C´Lct>C$0D$5C´L" Style="width: 180px;" />
                    <PropertyColumn T="AuditLog" TProperty="Guid" Property="x => x.EntityId"
                    Title="ID Aä1D¤5C¤BC " 7G-ÆS0'v-GF¢ 3# f²" 6Æ 730&föçB0öðæ÷7 6R" óí
                    <PropertyColumn T="AuditLog" TProperty="string" Property="x => x.ChangesJson"
                    Title="A,,7CÄ5CÔ5CÔ8Dò ,,¥4ôâ'" 6VÆÄ6Æ 730'FW‡B×G'Væ6 FR" óí
                </ColumnsContent>
            </GridPage>
        </BodyContent>
        @
        </WorkspacePage>
    </CascadingValue>
</file>

<file path="Promatis.Net.UI/Pages/AuditLog/AuditLogPageContext.cs">
using MudBlazor;

```

```

using Promatis.Net.Service;
using Promatis.Net.UI.Components.Grid;
using Promatis.Net.UI.Components.Toolbar;

namespace Promatis.Net.UI.Pages.AuditLog;

/// <summary>
/// A>CÔBCT:D B D4?D 0C$;CT=C,,O CÔ@C,,:C´0CD=Cä9 D BD 0CÔ8Dd5C´ 6D4@CÔ0C´0 C CCD8D$0.
/// A$KD BD4?C 5D" 5CD8CÔKCÂ <CT4C,,0D$>D >CÂ 4C´O DT>C´AD$0, D$CC´1C @C 8 D$0C ;C,,FD² 2 Aä B2í
/// </summary>
/// <summary>
/// A>CÔBCT:D B D4?D 0C$;CT=C,,O CÔ@C,,:C´0CD=Cä9 D BD 0CÔ8Dd5C´ 6D4@CÔ0C´0 C CCD8D$0.
/// Aô>C´=CäAD$LDâ ?Cä;C 3C 5D$ADò =C 0C$BCä<C BC,,:D2 >C =Cä2C´5CÔ8C´ OCD@C w&-D 7F-öä6öçFW#Bí
/// </summary>
public class AuditLogPageContext : GridActionContext<Domain.AuditLog>
{
    private readonly IAuditLogService _auditLogService;
    private DateRange _period = new(DateTime.Today, DateTime.Today.AddDays(1).AddSeconds(-1));

    /// <summary>
    /// Aô5D 8Cä4 C´>C4>C"Â : C>D$>D >CÄC CÔ0CÔ@Dô<D4N CÔ@C,,2Dô7C = MudDateRangePicker C" @C 7CÄ5D$:CRí
    /// </summary>
    public DateRange Period
    {
        get => _period;
        set
        {
            if (_period != value)
            {
                _period = value;
                NotifyUpdate(); // AÄ3CÔ>C$5CÔ=Cä ?CT@CT@C,,ACä2D´2C 5CÂ BD4;C 0D ,,?C,,:CT@ CD0D"•
                RequestRefresh(); // Aô8CÔ0CT< GridPage CÔ0 Cô5D 5Ct0Cô@CäA D 5D 2CT@CÔKDR 4C =CÔKD]
            }
        }
    }

    // A 2D$>CÄ0D$8Dt5D :C, 2CÔ5CD@Dô5D$ADò ääUB D´ @C =D$0C"<Cä< Cô@C, >D$:D KD$8C, 2C;C 4C=8D
    public AuditLogPageContext(IAuditLogService auditLogService) : base()
    {
        _auditLogService = auditLogService;

        PageTitle = "AdCD =C ; C CCD8D$0";

        // AD5C;C @C BC,,2CÔ> CäBC;DäGC 5CÂ 1C 7Cä2D´9 CRUD
        IsCreateVisible = false;
        IsEditVisible = false;
        IsDeleteVisible = false;

        // Aô0D BD 0C,,2C 5CÂ =C H D4=C,,2CT@D 0C´LCÔKC´ D >C=5D C =CÔKDR =C ACT@C$5D =D´9 Cô>D BD 0CÔ8Dt=D
        DataBroker.ConfigureServerMode(FetchServerDataInternalAsync);

        // A 5Ct>CÔ0D =Cä 4Cä1C 2C´0CT< C=0D BCä<CÔCDâ :CÔ>Cô:D2 GCT@CT7 Cô;C BDD>D <CT=CÔKC´ <CTBCä4 C>CÔBD
        AddCustomAction(new ToolbarCustomAction
        {
            Id = "excel_export",
            Title = "A$KC4@D47C,,BDÂ 2 Excel",
            Icon = Icons.Material.Filled.Download,
            Color = Color.Success,
            Variant = Variant.Filled,
            OnExecute = ExportToExcelInternalAsync
        });

        private async Task<GridData<Domain.AuditLog>>
        FetchServerDataInternalAsync(GridState<Domain.AuditLog> state, CancellationToken ct)
        {
            DateTime localStart = Period.Start ?? DateTime.MinValue;
            DateTime localEnd = Period.End ?? DateTime.MaxValue;

            if (Period.End.HasValue && Period.End.Value.TimeOfDay == TimeSpan.Zero)
            {
                localEnd = Period.End.Value.AddDays(1).AddSeconds(-1);
            }

            var request = new AuditLogSearchRequest
            {
                FromDate: localStart.ToUniversalTime(),

```

```

        ToDate: localEnd.ToUniversalTime(),
        EntityName: null,
        Action: null,
        PageIndex: state.Page,
        PageSize: state.PageSize
    );
    PagedResult<Domain.AuditLog> result = await _auditLogService.SearchLogsAsync(request, ct);
    return new GridData<Domain.AuditLog>
    {
        TotalItems = result.TotalCount,
        Items = result.Items
    };
}

private async Task ExportToExcelInternalAsync()
{
    SetActionEnabled("excel_export", false);
    try
    {
        await Task.Delay(1000);
    }
    finally
    {
        SetActionEnabled("excel_export", true);
    }
}
}
</file>

<file path="Promatis.Net.UI/Promatis.Net.UI.csproj">
<Project Sdk="Microsoft.NET.Sdk.Razor">
    <PropertyGroup>
        <TargetFramework>net10.0</TargetFramework>
        <Nullable>enable</Nullable>
        <ImplicitUsings>enable</ImplicitUsings>
    </PropertyGroup>
    <ItemGroup>
        <Compile Remove="AuditLog\*" />
        <Compile Remove="Interfaces\*" />
        <Compile Remove="Models\*" />
        <Content Remove="AuditLog\*" />
        <Content Remove="Interfaces\*" />
        <Content Remove="Models\*" />
        <EmbeddedResource Remove="AuditLog\*" />
        <EmbeddedResource Remove="Interfaces\*" />
        <EmbeddedResource Remove="Models\*" />
        <None Remove="AuditLog\*" />
        <None Remove="Interfaces\*" />
        <None Remove="Models\*" />
    </ItemGroup>
    <ItemGroup>
        <SupportedPlatform Include="browser" />
    </ItemGroup>
    <ItemGroup>
        <PackageReference Include="Microsoft.AspNetCore.Components.Web" Version="10.0.8" />
        <PackageReference Include="MudBlazor" Version="9.4.0" />
    </ItemGroup>
    <ItemGroup>
        <ProjectReference Include="..\..\MesCore\Promatis.Net.MES.Service\Promatis.Net.MES.Service.csproj" />
        <ProjectReference Include="..\Service\Promatis.Net.Service.csproj" />
    </ItemGroup>
</Project>
</file>

<file path="Promatis.Net.UI/Theme/PromatisTheme.cs">
using MudBlazor;

```



```

Ð
namespace Promatis.Net.Ui.Theme;Ð
Ð
public static class PromatisThemeÐ
{Ð
    public static MudTheme DefaultTheme => new()Ð
    {Ð
        PaletteLight = new PaletteLightÐ
        {Ð
            Primary = Colors.Blue.Lighten1, // A4;C 2C0KC' :Cä@C0>D 0D$8C$=D'9 Dd2CTB (C¤=Cä?C¤8, C :D$
            Secondary = Colors.Blue.Lighten2, // A$AC0>CÄ>C40D$5C'LC0KC' FC$5D" „2D$>D >D BCT?CT=C0KCR MC

        Ð
            AppbarBackground = "#EAEAEA", // BD>C0 2CT@DT=CT9 C0C05C'8 (MudAppBar) C" 4C05C$=Cä< D 5
            AppbarText = "#2C3E50", // Bd2CTB D$5C¤AD$0, Ct0C4>C'>C$:Cä2 C, 8C¤>C0>C¤ 2 C$5D EC
            DrawerBackground = "#EAEAEA", // BD>C0 1Cä:Cä2Cä3Cä <CT=Dâ =C 2C„3C FC„8 (MudDrawer)Ð
            Background = "#F9F9F9", // Aä1D"8C' DCä=Cä2D'9 Dd2CTB C0>CD;Cä6C¤8 C$ACTE D BD 0C08
            Surface = Colors.Shades.White, // BD>C0 :Cä=D$5C"=CT@Cä2 MudPaper, C¤0D BCäGCT: MudCard C

        },Ð
        PaletteDark = new PaletteDarkÐ
        {Ð
            Primary = Colors.Blue.Lighten1, // A4;C 2C0KC' FC$5D" 2 C0>Dt=Cä< D 5Cd8CÄ5 (CÄ0C4:C„9 D 8C
            Secondary = Colors.Blue.Lighten2, // A$AC0>CÄ>C40D$5C'LC0KC' FC$5D" 2 C0>Dt=Cä< D 5Cd8CÄ5Ð

        Ð
            AppbarBackground = "#0D0D0D", // BD>C0 2CT@DT=CT9 C0C05C'8 (MudAppBar) C" 3C'CC >C¤>CÄ G
            AppbarText = Colors.Shades.White, // Bd2CTB D$5C¤AD$0 C, 8C¤>C0>C¤ 2 D„0C0:CR „:Cä=D$@C AD$=D

        Ð
            Background = "#121212", // A4;D41Cä:C„9 D$QCÄ=D'9 DD>C0 AD$@C =C„F C0>CD;Cä6C¤8Ð
            Surface = "#1E1E1E", // A4@C DC„BCä2D'9 DD>C0 4C'0 MudPaper, C¤>C0BCT=D$0 C$:C'0
            DrawerBackground = "#1E1E1E", // BD>C0 1Cä:Cä2Cä9 C0C05C'8 C00C$8C40Dd8C, 2 D$QCÄ=Cä9 D$
            DrawerText = "#E0E0E0", // Bd2CTB D AD';Cä: C, BCT:D BC 2 C >C¤>C$>CÄ <CT=Dí
            TextPrimary = "#E0E0E0", // AäAC0>C$=Cä9 Dd2CTB D$5C¤AD$0 C00 D BD 0C08Dd0DR „<D03C¤
            TextSecondary = "#A0A0A0", // Bd2CTB CD;D0 2D$>D >D BCT?CT=C0>C4> D$5C¤AD$0 C, ?Cä4D

        },Ð
        LayoutProperties = new LayoutPropertiesÐ
        {Ð
            DrawerWidthLeft = "260px",Ð
            DrawerWidthRight = "300px"Ð
        }Ð
    }Ð
}
</file>

<file path="Promatis.Net.UI/UiDataBroker.cs">
using System.Reflection;Ð
using MudBlazor;Ð
Ð
namespace Promatis.Net.UI;Ð
Ð
/// <summary>Ð
/// A0;C BDD>D <CT=C0KC' 1D >C¤5D 4C =C0KDRâ &CT=D$@C ;C„7Cä2C =C0> D4?D 0C$;D05D" ?CäAD$0C$:Cä9 CD0C0=D'E C
/// A0>C'=CäAD$LDâ 0C AD$@C 3C„@D45D" T' >D" :Cä=C¤@CTBC0KDR 8C0BCT@DD5C"ACä2 CD>CÄ5C0=D'E D ;D46C 1D0:CT=CD
/// </summary>Ð
/// <typeparam name="TEntity">B$8C0 4Cä<CT=C0>C' AD4IC0>D BCfÄ÷G- W & Óí
public class UiDataBroker<TEntity> where TEntity : classÐ
{Ð
    private Func<GridState<TEntity>, CancellationToken, Task<GridData<TEntity>>>>
    _customServerDataProvider;Ð
    Ð
    public List<TEntity>? InMemoryItems { get; set; }Ð
    public bool IsInMemoryMode { get; private set; }Ð
    Ð
    public void ConfigureServerMode(Func<GridState<TEntity>, CancellationToken,
Task<GridData<TEntity>>> dataProvider)Ð
    {Ð
        _customServerDataProvider = dataProvider ?? throw new
ArgumentNullException(nameof(dataProvider));Ð
        InMemoryItems = null;Ð
        IsInMemoryMode = false;Ð
    }Ð
    Ð
    public void ConfigureInMemoryMode()Ð
    {Ð
        _customServerDataProvider = null;Ð
        IsInMemoryMode = true;Ð
    }Ð
    Ð
}

```

```
public async Task<GridData<TEntity>> FetchDataAsync(GridState<TEntity> state, CancellationToken
ct = default)ð
{ð
    if (_customServerDataProvider != null)ð
    {ð
        return await _customServerDataProvider(state, ct);ð
    }ð

    if (IsInMemoryMode)ð
    {ð
        return new GridData<TEntity>ð
        {ð
            Items = InMemoryItems ?? [],ð
            TotalItems = InMemoryItems?.Count ?? 0ð
        };ð
    }ð

    return new GridData<TEntity> { Items = Array.Empty<TEntity>(), TotalItems = 0 };ð
}ð

// =====ð
// A$+B A A A A At A A A,, "AT BÄ B´ Ä B2Ô A$ Ad A¢ B4"A &A,, Aô A "BD B B² ,, Aä A A´ AÔ )ð
// =====ð

/// <summary>ð
/// BT8D CD 3C,,GCTAC=8 D$>Dt=Câ <Cä4C,,DC,,FC,,@D45D" ;Cä:C ;DÄ=D´9 C=MD, >Cö5D 0D$8C$=Cä9 Cö0CÄ0D$8 Ct0 0 C
/// Aô>C´=CäAD$LDâ 8D :C´NDt0CTB CÖ5Cä1DT>CD8CÄ>D BDÂ ?Cä2D$>D =D´E D$OCd5C´KDR 4TÄT5B07C ?D >D >C" : B #
/// </summary>ð
/// <param name="stateStr">B BD >C= >C$>CR ACäAD$>Dô=C,,5 D$@C =Ct0C=FC,,8 ("Added", "Modified", "Deleted",
/// B 0CÄ ?D 8D,,5CDHC,,9 CD>CÄ5CÔ=D´9 Cä1D=5C=B</param>ð
public void ApplyIncrementalOzuDelta(string stateStr, TEntity entity)ð
{ð
    // ATAC´8 CÄK D 0C >D$0CT< C" ACT@C$5D =Cä< D 5Cd8CÄ5 (CÖ0Cô@C,,<CT@, C´>C48 C CCD8D$0), Ää B2Ô<D4BC F
    if (!IsInMemoryMode || InMemoryItems == null) return;ð

    // A,,7C$;CT:C 5CÄ AC$>C"AD$2Cä -B A Cö>CÄ>D"LDâ @CTDC´5C=AC,,8 (C$KCö>C´=Dô5D$ADò >CD8CÒ @C 7 CÖ0 D CD
    PropertyInfo? idProperty = typeof(TEntity).GetProperty("Id");ð
    if (idProperty == null) return;ð

    Guid entityId = (Guid)idProperty.GetValue(entity)!;ð

    switch (stateStr)ð
    {ð
        case "Added":ð
            // At0D"8D$0 CäB CDCC ;C,,@Cä2C =C,,0 Cö@C, 0D 8CÖED >CÔ=D´E C$ACô;CTAC=0D]
            if (!InMemoryItems.Any(x => (Guid)idProperty.GetValue(x)! == entityId))ð
            {ð
                InMemoryItems.Add(entity);ð
            }ð
            break;ð

        case "Modified":ð
            TEntity? existingItem = InMemoryItems.FirstOrDefault(x =>
(Guid)idProperty.GetValue(x)! == entityId);ð
            if (existingItem != null)ð
            {ð
                // AÔ0DT>CD8CÄ 2D 5 Ct=C GC,,<D´5 C, AD$@Cä:Cä2D´5 D 2Cä9D BC$0 CD;Dò BCäGCTGCÔ>C4> C= >Cö8
                IEnumerable<PropertyInfo> properties = typeof(TEntity).GetProperties()ð
                    .Where(p => p.CanWrite && p.CanRead)ð
                    .Where(p => p.PropertyType.IsValueType || p.PropertyType == typeof(string));ð

                foreach (PropertyInfo prop in properties)ð
                {ð
                    prop.SetValue(existingItem, prop.GetValue(entity));ð
                }ð
            }ð
            break;ð

        case "Deleted":ð
        case "SoftDeleted":ð
            TEntity? itemToRemove = InMemoryItems.FirstOrDefault(x =>
(Guid)idProperty.GetValue(x)! == entityId);ð
            if (itemToRemove != null)ð
            {ð
                InMemoryItems.Remove(itemToRemove);ð
            }ð
    }ð
}
```

```

        break;
    }
}
</file>

<file path="Promatis.Net.UI/UiModule.cs">
using MudBlazor;

namespace Promatis.Net.UI;

public class UiModule : IUiModule
{
    public string Name => GetType().Assembly.GetName().Name ?? "Unknown.Module";

    public IEnumerable<(string Title, string Href, string Icon, string? Group)> GetMenuItems()
    {
        return new List<(string Title, string Href, string Icon, string? Group)>
        {
            // B0;CT<CT=D" 2CT@DT=CT3Câ CD >C$=Dò „2C05 C00C0>C•
            ("A4;C 2C00D0"Â "0"Â -6öç2äÖ FW&- Âäf-ÆËVBäF 6†&ö &BÂ çVÆÂ'Í
            // B 0Ct4CT; D 8D BCT<C0>C4> C 4CÄ8C08D BD 8D >C$0C08Dy
            ("AdCD =C ; C CCD8D$0", "/audit-logs", Icons.Material.Filled.History, "A 4CÄ8C08D BD 8D >C$0C08CR
        };
    }
}
</file>

<file path="Promatis.Net.UI/wwwroot/App.css">
html, body {
    font-family: 'Helvetica Neue', Helvetica, Arial, sans-serif;
}

a, .btn-link {
    color: #0066bb7;
}

.btn-primary {
    color: #fff;
    background-color: #1b6ec2;
    border-color: #1861ac;
}

.btn:focus, .btn:active:focus, .btn-link.nav-link:focus, .form-control:focus, .form-check-
input:focus {
    box-shadow: 0 0 0 0.1rem white, 0 0 0 0.25rem #258cfb;
}

.content {
    padding-top: 1.1rem;
}

h1:focus {
    outline: none;
}

.valid.modified:not([type=checkbox]) {
    outline: 1px solid #26b050;
}

.invalid {
    outline: 1px solid #e50000;
}

.validation-message {
    color: #e50000;
}

.blazor-error-boundary {
    background: url(
8vd3d3LnczLm9yZy8yMDAwL3N2ZyIgeG1sbnM6eGpbms9Imh0dHA6Ly93d3cudzMub3JnLzE5OTkveGpbmsiIG92ZXJmbG93PS
JoawRkZW4iPjxkZWZzPjxjbGwUGF0aCBpZD0iY2xpcDAiPjxyZWNOIHg9IjIzNSIgeT0iNTEiIHdpZHRoPSI1NiIgaGVPZ2h0PS
I0OSivPjwvY2xpcFBhdGg+PC9kZWZzPjxnIGNSaXAtcGF0aD0idXJsKCNjbGlmMCKiIHRYW5zZm9ybT0idHJhbnNsYXRlKC0yMz
UgLTUxKSI+PHBhdGggZD0iTTI2My41MDYgNTFDMjY0LjcxNyA1MSAyNjUuODEzIDUxLjQ4MzcgMjY2LjYwNiA1Mi4yNjU4TDI2Ny
4wNTIuNTIuNTk4NyAyNjcuNTM5IDUzLjYyODMgMjkwLjE4NSA1Mi4xODMxIDU5MC41NDUgOTIuNTk1IDU5MC42NTYgOTIuOTk2Qz

```

```
I5MC44Nzcg0TMuNTEzIDI5MSA5NC4w0DE1IDI5MSA5NC42NzgyIDI5MSA5Ny4wNjUxIDI40S4wMzg0TkgMjg2LjYxNyA50UwyND
AuMzgZIDk5QzIzNy45NjMg0TkgMjM2IDk3LjA2NTEgMjM2IDk0LjY3ODIgMjM2IDk0LjM3OTkgMjM2LjAzMSA5NC4w0Dg2IDIzNi
4w0Dkg0TMu0DA3MkwyMzYuMzM4IDkzLjAxNjIgMjM2Ljg10CA5Mi4xMzE0IDI10S40NzMgNTMuNjI5NCAyNTku0TYxIDUyLjc50D
UgMjYwLjQwNyA1Mi4yNjU4QzI2MS4yIDUxLjQ4Mzc0MjYyLjI5NiA1MSAyNjMuNTA2IDUxWk0yNjMuNTg2IDY2LjAxODNDMjYwLj
czNyA2Ni4wMTgzIDI10S4zMTMgNjcuMTI0NSAyNTkuMzEzIDY5LjMzNyAyNTkuMzEzIDY5LjYxMDIgMjU5LjMzMjA20S44NjA4ID
I10S4zNzEgNzAuMDg4N0wyNjEuNzklIDg0LjAxNjEgMjY1LjM4IDg0LjAxNjEgMjY3LjgyMSA20S43NDc1QzI2Ny44NiA20S43Mz
A5IDI2Ny44NzkgNjkuNTg3NyAyNjcuODc5IDY5LjMxNzkgMjY3Ljg30SA2Ny4xMTgyIDI2Ni40NDggNjYuMDE4MyAyNjMuNTg2ID
Y2LjAxODNaTTI2My41NzYgODYuMDU0N0MyNjEuMDQ5IDg2LjA1NDcgMjU5Ljc4NiA4Ny4zMdA1IDI10S430DYgODkuNzkyMSAyNT
kuNzg2IDkyLjI4Mzc0MjYxLjA00SA5My41Mjk1IDI2My41NzYgOTMuNTI5NSAyNjYuMTE2IDkzLjUyOTUgMjY3LjM4NyA5Mi4yOD
M3IDI2Ny4z0DcgODkuNzkyMSAyNjcuMzg3IDg3LjMwMDUgMjY2LjExNiA4Ni4wNTQ3IDI2My41NzYgODYuMDU0N0iIGZpbGw9Ii
NGRkU1MDAiIGZpbGwtcnVsZT0iZXZlbn9kZCivPjwvZz48L3N2Zz4=) no-repeat 1rem/1.8rem, #b32121;#
```

```
padding: 1rem 1rem 3.7rem;#
color: white;#
}#
#
.blazor-error-boundary::after {#
    content: "An error has occurred."#
}#
#
.darker-border-checkbox.form-check-input {#
    border-color: #929292;#
}#
#
.form-floating > .form-control-plaintext::placeholder, .form-floating > .form-control::placeholder {#
    color: var(--bs-secondary-color);#
    text-align: end;#
}#
#
.form-floating > .form-control-plaintext:focus::placeholder, .form-floating > .form-
control:focus::placeholder {#
    text-align: start;#
}
</file>
```

```
<file path="Service/Contracts/AuditLog/AuditLogSearchRequest.cs">
namespace Promatis.Net.Service;#
#
public record AuditLogSearchRequest(#
    DateTime FromDate,#
    DateTime ToDate,#
    string? EntityName = null,#
    string? Action = null,#
    int PageIndex = 0,#
    int PageSize = 10);
</file>
```

```
<file path="Service/Contracts/AuditLog/PagedResult.cs">
namespace Promatis.Net.Service;#
#
public record PagedResult<T>(ICollection<T> Items, int TotalCount);
</file>
```

```
<file path="Service/Promatis.Net.Service.csproj">
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <TargetFramework>net10.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
  </PropertyGroup>

  <ItemGroup>#
    <PackageReference Include="Microsoft.EntityFrameworkCore.Relational" Version="10.0.8" />#
  </ItemGroup>

  <ItemGroup>#
    <ProjectReference Include="..\Data\Promatis.Net.Data.csproj" />#
  </ItemGroup>

</Project>
</file>
```

```
<file path="Service/Services/AuditLogService/AuditLogService.cs">
using Microsoft.EntityFrameworkCore;#
using Promatis.Net.Data;#
using Promatis.Net.Domain;#
```

```

Ð
namespace Promatis.Net.Service;Ð
Ð
/// <summary>Ð
/// B 5D 2C„A CD;Dð @C 1CäBD² A C´>C40CÄ8 C CCD8D$0.Ð
/// Að>C´=CäAD$LDä ACä2CÄ5D BC„< D 1C 7Cä2D´< BaseService C, 0C$BCä<C BC„GCTACp8 D 5C48D BD 8D CCTBD O D :C
/// </summary>Ð
public class AuditLogService(IDbContextFactory<ApplicationDbContext> contextFactory)Ð
    : BaseService<AuditLog, Guid, ApplicationDbContext>(contextFactory), IAuditLogServiceÐ
{Ð
    private static List<string>? _cachedEntityNames;Ð
    private static readonly SemaphoreSlim CacheLock = new(1, 1);Ð
Ð
    public async Task<PagedResult<AuditLog>> SearchLogsAsync(AuditLogSearchRequest request,
        Cancellation_token cancellation_token = default)Ð
    {Ð
        await using ApplicationDbContext context = await
        ContextFactory.CreateDbContextAsync(cancellation_token);Ð
Ð
        IQueryable<AuditLog> query = context.Set<AuditLog>()Ð
            .AsNoTracking()Ð
            .Where(l => l.ChangedAt >= request.FromDate && l.ChangedAt <= request.ToDate);Ð
Ð
        if (!string.IsNullOrEmpty(request.EntityName)) query = query.Where(l => l.EntityName
        == request.EntityName);Ð
        if (!string.IsNullOrEmpty(request.Action)) query = query.Where(l => l.Action ==
        request.Action);Ð
Ð
        int totalCount = await query.CountAsync(cancellation_token);Ð
Ð
        if (totalCount == 0)Ð
            return new PagedResult<AuditLog>([], 0);Ð
Ð
        List<AuditLog> items = await queryÐ
            .OrderByDescending(l => l.ChangedAt)Ð
            .Skip(request.PageIndex * request.PageSize)Ð
            .Take(request.PageSize)Ð
            .ToListAsync(cancellation_token);Ð
Ð
        return new PagedResult<AuditLog>(items, totalCount);Ð
    }Ð
Ð
    public async Task<List<string>> GetAvailableEntityNamesAsync(Cancellation_token
        cancellation_token = default)Ð
    {Ð
        if (_cachedEntityNames != null)Ð
        {Ð
            return [.. _cachedEntityNames];Ð
        }Ð
Ð
        await CacheLock.WaitAsync(cancellation_token);Ð
        tryÐ
        {Ð
            if (_cachedEntityNames != null)Ð
            {Ð
                return [.. _cachedEntityNames];Ð
            }Ð
Ð
            await using ApplicationDbContext context = await
            ContextFactory.CreateDbContextAsync(cancellation_token);Ð
Ð
            _cachedEntityNames = await context.Set<AuditLog>()Ð
                .AsNoTracking()Ð
                .Select(l => l.EntityName)Ð
                .Distinct()Ð
                .OrderBy(name => name)Ð
                .ToListAsync(cancellation_token);Ð
Ð
            return [.. _cachedEntityNames];Ð
        }Ð
        finallyÐ
        {Ð
            CacheLock.Release();Ð
        }Ð
    }Ð
Ð
}Ð

```

```

    public static void InvalidateCache()
    {
        CacheLock.Wait();
        try
        {
            _cachedEntityNames = null;
        }
        finally
        {
            CacheLock.Release();
        }
    }
}
</file>

<file path="Service/Services/AuditLogService/IAuditLogService.cs">
using Promatis.Net.Domain;
namespace Promatis.Net.Service;
public interface IAuditLogService
{
    /// <summary>
    /// Aö>C,,AC¢ ;Cä3Cä2 C CCD8D$0 Cö> CD8C ?C 7Cä=D2 4C B D ?Cä4CD5D 6C¤>C' DC,,;DÄBD 0Dd8C, 8 Cö0C48C00Dd8C
    /// </summary>
    Task<PagedResult<AuditLog>> SearchLogsAsync(AuditLogSearchRequest request, CancellationToken
cancellationToken = default);
    /// <summary>
    /// Aö>C´CDt5C08CR AC08D :C CC08C¤0C´LC0KDR 8CÄ5C0 AD4IC0>D BCT9, C¤>D$>D KCR 5D BDÂ 2 C´>C40DR ,,4C´O C$
    /// </summary>
    Task<List<string>> GetAvailableEntityNamesAsync(CancellationToken cancellationToken = default);
}
</file>

<file path="Service/Services/BaseService/BaseService.cs">
using Microsoft.EntityFrameworkCore;
using Promatis.Net.Domain.Interface;
namespace Promatis.Net.Service;
public abstract class BaseService<T, TKey, TContext>(IDbContextFactory<TContext> contextFactory)
    : IBaseService<T, TKey>
    where T : class, IDomainObjectHasKey<TKey>
    where TContext : DbContext
{
    protected readonly IDbContextFactory<TContext> ContextFactory = contextFactory ?? throw new
ArgumentNullException(nameof(contextFactory));
    public virtual async Task<T?> GetByIdAsync(TKey id)
    {
        await using TContext context = await ContextFactory.CreateDbContextAsync();
        return await context.Set<T>().FindAsync(id);
    }
    public virtual async Task<List<T>> GetAllAsync()
    {
        await using TContext context = await ContextFactory.CreateDbContextAsync();
        return await context.Set<T>().ToListAsync();
    }
    public virtual async Task AddAsync(T entity)
    {
        await using TContext context = await ContextFactory.CreateDbContextAsync();
        await context.Set<T>().AddAsync(entity);
        await context.SaveChangesAsync();
    }
    public virtual async Task UpdateAsync(T entity)
    {
        await using TContext context = await ContextFactory.CreateDbContextAsync();
        context.Set<T>().Update(entity);
        await context.SaveChangesAsync();
    }
    public virtual async Task DeleteAsync(TKey id)

```

```

        {
            await using TContext context = await ContextFactory.CreateDbContextAsync();
            T? entity = await context.Set<T>().FindAsync(id);
            if (entity != null)
            {
                context.Set<T>().Remove(entity);
                await context.SaveChangesAsync();
            }
        }
    }
}
</file>

<file path="Service/Services/BaseService/IBaseService.cs">
namespace Promatis.Net.Service;
{
    // A 0Ct>C$KC' 5%TM
    public interface IBaseService<T, in TKey> where T : class
    {
        Task<T?> GetByIdAsync(TKey id);
        Task<List<T>> GetAllAsync();
        Task AddAsync(T entity);
        Task UpdateAsync(T entity);
        Task DeleteAsync(TKey id);
    }
}
</file>

<file path="Service/Services/ReferenceService/IReferenceService.cs">
namespace Promatis.Net.Service;
{
    // AD;Dò ACô@C 2CäGC08Cæ>C-
    public interface IReferenceService<T> : IBaseService<T, Guid> where T : class
    {
        Task<T?> GetByCodeAsync(string code);
        Task<List<T>> SearchByNameAsync(string namePart);
    }
}
</file>

<file path="Service/Services/ReferenceService/ReferenceService.cs">
using Microsoft.EntityFrameworkCore;
using Promatis.Net.Domain;
{
    namespace Promatis.Net.Service;
    {
        public abstract class ReferenceService<T, TContext>(IDbContextFactory<TContext> contextFactory)
            : BaseService<T, Guid, TContext>(contextFactory), IReferenceService<T>
            where T : ReferenceBase
            where TContext : DbContext
        {
            public async Task<T?> GetByCodeAsync(string code)
            {
                await using TContext context = await ContextFactory.CreateDbContextAsync();
                return await context.Set<T>().FirstOrDefaultAsync(x => x.Code == code);
            }

            public async Task<List<T>> SearchByNameAsync(string namePart)
            {
                await using TContext context = await ContextFactory.CreateDbContextAsync();
                return await context.Set<T>()
                    .Where(x => x.Name.Contains(namePart))
                    .ToListAsync();
            }
        }
    }
}
</file>

<file path="Service/Services/ReferenceTreeService/IReferenceTreeService.cs">
using Microsoft.EntityFrameworkCore;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
{
    namespace Promatis.Net.Service;

```

```

Ð
public interface IReferenceTreeService<T, TContext> : IReferenceService<T>, ITreeService<T>Ð
    where T : ReferenceTreeBase<T>, ITreeNode<T>, IDomainObjectHasKey<Guid>Ð
    where TContext : DbContextÐ
{Ð
    // B AC²2Cä7CÔKCÂ =C AC´5CD>C$0CÔ8CT< C,,=D$5D DCT9D >C" 2D Q D >D,,;CäADÂ
}
</file>

<file path="Service/Services/ReferenceTreeService/ReferenceTreeService.cs">
using System.Reflection;Ð
using Microsoft.EntityFrameworkCore;Ð
using Promatis.Net.Domain;Ð
using Promatis.Net.Domain.Interface;Ð
Ð
namespace Promatis.Net.Service;Ð
Ð
/// <summary>Ð
/// B4=C,,2CT@D 0C´LCÔKC´ 1C 7Cä2D´9 D 5D 2C,,A CD;Dò @C 1CäBD² ACâ 2D 5CÄ8 CD@CT2Cä2C,,4CÔKCÄ8 D BD CC²BD4@C <C
/// A00D ;CT4D45D" ;Cä3C,,;D2 >C KDt=D´E D ?D 0C$>Dt=C,,;Cä2 (ReferenceService) C, @C AD,,8D OCTB CTQ CD;Dò 8CT@
/// </summary>Ð
/// <typeparam name="T">B$8Cò AD4ICÔ>D BC,Â CCÔ0D ;CT4Cä2C =CÔKC´ >D" &VfW&Væ6UG&VT& 6RäÂ÷G– W & Óí
/// <typeparam name="TContext">A²>CÔBCT:D B C 0CtK CD0CÔ=D´E.</typeparam>Ð
public abstract class ReferenceTreeService<T, TContext>(IDbContextFactory<TContext> contextFactory)Ð
    : ReferenceService<T, TContext>(contextFactory), IReferenceTreeService<T, TContext>Ð
    where T : ReferenceTreeBase<T>, ITreeNode<T>, IDomainObjectHasKey<Guid>Ð
    where TContext : DbContextÐ
{Ð
    private TreeServiceProxy? _treeServiceProxy;Ð
    private TreeServiceProxy Engine => _treeServiceProxy ??= new TreeServiceProxy(ContextFactory,
this);Ð
Ð
    // =====Ð
    // BÔ AT A B$ B´ Aô Aä B B B´ Aä Aä A" AD AÔ+A´ A$ Ad A¢ E$TU4U%d"4R f R B4 A´ B A$ AÔ Bò Aä A
    // =====Ð
    public Task<List<T>> GetRootsAsync() => Engine.GetRootsAsync();Ð
    public Task<List<T>> GetChildrenAsync(Guid parentId) => Engine.GetChildrenAsync(parentId);Ð
    public Task<List<T>> GetParentPathAsync(Guid id) => Engine.GetParentPathAsync(id);Ð
    public Task<T?> GetFullTreeAsync(Guid rootId) => Engine.GetFullTreeAsync(rootId);Ð
    public Task MoveAsync(Guid id, Guid? newParentId, CancellationToken ct = default) =>
Engine.MoveAsync(id, newParentId, ct);Ð
Ð
    public abstract Task<T> CreateChildTemplateAsync(T parent);Ð
Ð
    // A$ B4"B AÔ A,, AÄ B "-B A A,, A "Aä Aô A "BD B B½
    private class TreeServiceProxy(IDbContextFactory<TContext> factory, ReferenceTreeService<T,
TContext> parentService)Ð
        : TreeService<T, TContext>(factory)Ð
    {Ð
        public override Task<T> CreateChildTemplateAsync(T parent) =>
parentService.CreateChildTemplateAsync(parent);Ð
    }Ð
}
</file>

<file path="Service/Services/TreeService/ITreeService.cs">
using Microsoft.EntityFrameworkCore;Ð
using Promatis.Net.Domain.Interface;Ð
using System.Reflection;Ð
Ð
namespace Promatis.Net.Service;Ð
Ð
/// <summary>Ð
/// A4;Cä1C ;DÄ=D´9 Cò;C BDD>D <CT=CÔKC´ :Cä=D$@C :D" 4C´O C 1D >C´ND$=Cä ;Dä1Cä3Cä ACT@C$8D 0, D4?D 0C$;DòND
/// A00D ;CT4D45D" 1C 7Cä2D´9 CRUD Cò;C BDD>D <D² 8 D 0D HC,,@Dô5D" 5C4> C,,5D 0D EC,,GCTAC²8CÄ8 CÄ5D$>CD0CÄ8 D4
/// </summary>Ð
/// <typeparam name="T">B$8Cò 4Cä<CT=CÔ>C´ AD4ICÔ>D BC,Â @CT0C´8CtCDäICT9 C²>CÔBD 0C²B ITreeNode.</typeparam>Ð
public interface ITreeService<T> : IBaseService<T, Guid>Ð
    where T : class, ITreeNode<T>, IDomainObjectHasKey<Guid>Ð
{Ð
    Task<List<T>> GetRootsAsync();Ð
    Task<List<T>> GetChildrenAsync(Guid parentId);Ð
    Task<List<T>> GetParentPathAsync(Guid id);Ð
    Task<T?> GetFullTreeAsync(Guid rootId);Ð
    Task MoveAsync(Guid id, Guid? newParentId, CancellationToken ct = default);Ð
    Task<T> CreateChildTemplateAsync(T parent);Ð
}

```



```
}  
</file>
```

```
<file path="Service/Services/TreeService/TreeService.cs">  
using Microsoft.EntityFrameworkCore;  
using Promatis.Net.Domain.Interface;  
using System.Reflection;  
;  
namespace Promatis.Net.Service;  
;  
/// <summary>  
/// B4=C,,2CT@D 0C'LC0KC' 1C 7Cä2D'9 C;C AD @CT0C'8Ct0Dd8C, 1C,,7C05D 0;Cä3C,,:C, 4C'0 A'.A +BR 4D 5C$>C$8CD=D  
/// A,,=C@0C0AD4;C,,@D45D" BD06CT;D'5 C,,5D 0D EC,,GCTAC@8CR 0C'3Cä@C,,BCÄK B #A , C0@Cä2CT@C@C Dd8C@;Cä2 Dd8C@;C  
/// </summary>  
/// <typeparam name="T">B$8C0 4Cä<CT=C0>C4> Cä1D@5C@BC Ä @CT0C'8CtCDäICT3Cä •G&VTæöFRäÄ÷G- W & Óí  
/// <typeparam name="TContext">A@>C0BCT:D B C 0CtK CD0C0=D'E DbContext.</typeparam>  
public abstract class TreeService<T, TContext>(IDbContextFactory<TContext> contextFactory)   
: BaseService<T, Guid, TContext>(contextFactory), ITreeService<T>  
where T : class, ITreeNode<T>, IDomainObjectHasKey<Guid>  
where TContext : DbContext  
{  
    public async Task<List<T>> GetRootsAsync()  
    {  
        await using TContext context = await ContextFactory.CreateDbContextAsync();  
        return await context.Set<T>().AsNoTracking().Where(x => x.ParentId == null).ToListAsync();  
    }  
  
    public async Task<List<T>> GetChildrenAsync(Guid parentId)  
    {  
        await using TContext context = await ContextFactory.CreateDbContextAsync();  
        return await context.Set<T>().AsNoTracking().Where(x => x.ParentId ==  
parentId).ToListAsync();  
    }  
  
    public async Task<List<T>> GetParentPathAsync(Guid id)  
    {  
        await using TContext context = await ContextFactory.CreateDbContextAsync();  
        var path = new List<T>();  
        T? current = await context.Set<T>().AsNoTracking().FirstOrDefaultAsync(x => x.Id == id);  
  
        while (current != null)  
        {  
            path.Insert(0, current);  
            if (current.ParentId == null) break;  
            Guid parentId = current.ParentId.Value;  
            current = await context.Set<T>().AsNoTracking().FirstOrDefaultAsync(x => x.Id ==  
parentId);  
            if (current == null) break;  
        }  
        return path;  
    }  
  
    public async Task<T?> GetFullTreeAsync(Guid rootId)  
    {  
        await using TContext context = await ContextFactory.CreateDbContextAsync();  
        List<T> allItems = await context.Set<T>().AsNoTracking().ToListAsync();  
        ILookup<Guid?, T> lookup = allItems.ToLookup(x => x.ParentId);  
        T? root = allItems.FirstOrDefault(x => x.Id == rootId);  
        if (root == null) return null;  
  
        BuildTree(root, lookup);  
        return root;  
    }  
  
    private void BuildTree(T parent, ILookup<Guid?, T> lookup)  
    {  
        List<T> children = lookup[parent.Id].ToList();  
        parent.Children.Clear();  
        foreach (T child in children)  
        {  
            parent.Children.Add(child);  
            PropertyInfo? parentProp = child.GetType().GetProperty(nameof(ITreeNode<T>.Parent));  
            if (parentProp is { CanWrite: true }) parentProp.SetValue(child, parent);  
            BuildTree(child, lookup);  
        }  
    }  
}
```

```

    public virtual async Task MoveAsync(Guid id, Guid? newParentId, CancellationToken ct = default)
    {
        await using TContext context = await ContextFactory.CreateDbContextAsync(ct);
        T entity = await context.Set<T>().FirstOrDefaultAsync(x => x.Id == id, ct)
            ?? throw new Exception($"B CD=CâAD$L {typeof(T).Name} D "B ¶-Gò =CR =C 9CD5C00.");
        if (entity.ParentId == newParentId) return;
        if (newParentId.HasValue)
        {
            if (newParentId == id) throw new InvalidOperationException("B47CT; C05 CÄ>Cd5D" 1D´BDÂ @Cä4C„BCT;
                List<T> path = await GetParentPathAsync(newParentId.Value);
                if (path.Any(x => x.Id == id)) throw new InvalidOperationException("Bd8Cα;C„GCTACα0Dò 7C 2C„AC„<C
                bool parentExists = await context.Set<T>().AnyAsync(x => x.Id == newParentId.Value, ct);
                if (!parentExists) throw new Exception("A0>C$KC' @Cä4C„BCT;DÂ =CR =C 9CD5C0â"½
            }
            entity.ParentId = newParentId;
            await context.SaveChangesAsync(ct);
        }
        public abstract Task<T> CreateChildTemplateAsync(T parent);
    }
</file>

<file path="Tests/AssemblyInfo.cs">
using Xunit;
// AâBCα;DâGC 5D" ?C @C ;C´5C´LC0>CR 2D´?Cä;C05C08CR BCTAD$>C" 2 D0BCä9 D 1Cä@Cα5D
[assembly: CollectionBehavior(DisableTestParallelization = true)]
</file>

<file path="Tests/Domain/DomainLogicTests.cs">
using Promatis.Net.Domain;
using Xunit;
namespace Promatis.Net.ApplicationModel.Tests.Domain;
public class DomainLogicTests
{
    // B 5C ;C„7C FC„8 CD;Dò BCTAD$8D >C$0C08Dò 0C AD$@C :D$=D´E Cα;C AD >C-
    private class TestDomainObject : DomainObject { }
    private class TestReference : ReferenceBase { }
    [Fact]
    public void DomainObject_EveryInstance_ShouldHaveUniqueId()
    {
        // Arrange & Act
        var obj1 = new TestDomainObject();
        var obj2 = new TestDomainObject();
        var obj3 = new TestDomainObject();
        // Assert
        Assert.NotEqual(obj1.Id, obj2.Id);
        Assert.NotEqual(obj2.Id, obj3.Id);
        Assert.NotEqual(Guid.Empty, obj1.Id);
    }
    [Theory]
    [InlineData("")]
    [InlineData(" ")]
    [InlineData(null)]
    public void ReferenceBase_Name_CanBeNullOrEmpty_TechnicalCheck(string? invalidName)
    {
        // Arrange
        var reference = new TestReference();
        // Act
        reference.Name = invalidName!; // A„AC0>C´LCtCCT< !, D$0C¢ :C : Name C" :Cä4CR =CR çVÆÆ &Æ]
        // Assert
        // A0>Cα0 CÄK C0@CäAD$> C0@Cä2CT@D05CÄÂ GD$> D 2Cä9D BC$> C0@C„=C„<C 5D" MD$8 Ct=C GCT=C„0D
        // ATAC´8 C$K CD>C 0C$8D$5 C$0C´8CD0dd8Dâ 2 C CCDCD"5CÄÂ MD$>D" BCTAD" ?Cä<Cä6CTB CTQ CäBC´0CD8D$LB
        Assert.Equal(invalidName, reference.Name);
    }
}

```

```

[Fact]
public void ReferenceBase_ShouldHoldValidName()
{
    // Arrange
    var reference = new TestReference();
    string expectedName = "AäACô>C$=Cä9 D :C´0CB#½

    // Act
    reference.Name = expectedName;

    // Assert
    Assert.Equal(expectedName, reference.Name);
}
}
</file>

<file path="Tests/Domain/DomainObjectBaseLogicTests.cs">
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using Xunit;
namespace Promatis.Net.ApplicationModel.Tests.Domain;
public class DomainObjectBaseLogicTests
{
    private class ConcreteDomainObject : DomainObject { }

    [Fact]
    public void DomainObject_ShouldGenerateNewGuidInConstructor()
    {
        // Arrange & Act
        var obj1 = new ConcreteDomainObject();
        var obj2 = new ConcreteDomainObject();

        // Assert
        Assert.NotEqual(Guid.Empty, obj1.Id);
        Assert.NotEqual(Guid.Empty, obj2.Id);
        Assert.NotEqual(obj1.Id, obj2.Id);
    }

    [Fact]
    public void DomainObject_ShouldCastToIDomainObjectAndHaveCreatedAt()
    {
        // Arrange
        var obj = new ConcreteDomainObject();

        // Act
        IDomainObject baseInterface = obj;

        // Assert
        // B$5C05D L CÄK D$>C´LC¤> C0@Cä2CT@D05CÂ =C ;C,,GC,,5 Ct=C GCT=C,,0. D
        // A0>C0KD$:C 7C ?C,,AC, & 6T-çFW&f 6Rä7&V FVD B Ö æWtF FR 7CD5D L C05 D :Cä<C08C´8D CCTBD 0.D
        Assert.True(baseInterface.CreatedAt > DateTime.MinValue);
        Assert.True((DateTime.UtcNow - baseInterface.CreatedAt).TotalSeconds < 5);
    }

    [Fact]
    public void CreatedAt_ShouldAllowManualInitialization_OnCreation()
    {
        // Arrange
        var customDate = new DateTime(2020, 1, 1, 0, 0, 0, DateTimeKind.Utc);

        // Act
        // B 0C >D$0CTB D$>C´LC¤> Dt5D 5Cr 8C08Dd8C ;C,,7C BCä@ Cä1D¤5C¤BC ?D 8 D >Ct4C =C,,8D
        var obj = new ConcreteDomainObject { CreatedAt = customDate };

        // Assert
        Assert.Equal(customDate, obj.CreatedAt);
    }
}
</file>

<file path="Tests/Domain/DomainObjectBaseTests.cs">
using Moq;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;

```

```

using Xunit;
{
namespace Promatis.Net.ApplicationModel.Tests.Domain;
{
public class DomainObjectBaseTests
{
// B >Ct4C 5CÂ <C,,=C,,<C ;DÄ=D4N D 5C ;C,,7C FC,,N CD;Dò BCTAD$>C-
private class TestDomainObject : DomainObjectBase<int> { }

[Fact]
public void DomainObject_ShouldInitializeWithDefaultValues()
{
// Arrange & Act
var domainObject = new TestDomainObject();

// Assert
Assert.Equal(default, domainObject.Id);
// Aô@Câ2CT@Dô5CÂÂ GD$> CDôD$ô D >Ct4C =C,,O D4AD$ôCô>C$8C´ôD L Cæ>D @CT:D$=Câ „A CD>CôCD :Câ< C" AC
Assert.True((DateTime.UtcNow - domainObject.CreatedAt).TotalSeconds < 1);
}

[Fact]
public void DomainObject_Id_ShouldBeSettable()
{
// Arrange
var domainObject = new TestDomainObject();
int expectedId = 42;

// Act
domainObject.Id = expectedId;

// Assert
Assert.Equal(expectedId, domainObject.Id);
}

[Fact]
public void Interface_ShouldAllowMocking()
{
// Aô@C,,<CT@ C,,ACô>C´Lct>C$ôCô8Dò Ô÷ 4C´O C,,=D$5D DCT9D ô (CTAC´8 Cä= C CCD5D" ?CT@CT4C 2C BDÄADò 2
var mock = new Mock<IDomainObjectHasKey<string>>>();
mock.SetupProperty(m => m.Id, "test-guid");

Assert.Equal("test-guid", mock.Object.Id);
}
}
</file>

<file path="Tests/Domain/DomainObjectExtendedTests.cs">
using Promatis.Net.Domain;
using Xunit;
{
namespace Promatis.Net.ApplicationModel.Tests.Domain;
{
public class DomainObjectExtendedTests
{
private class GuidTestObject : DomainObjectBase<Guid> { }

[Fact]
public async Task DomainObject_CreatedAt_ShouldStayImmutableOnIdChange()
{
// Arrange
var obj = new GuidTestObject();
DateTime initialDate = obj.CreatedAt;
var newId = Guid.NewGuid();

// Aô5C >C´LD,,ôDò ?C Cctô, DtBCä1D² CC 5CD8D$LD O, DtBCä 2D 5CÄO Cô5 «D$8CæôCTB» C" AC$>C"AD$2C]
await Task.Delay(10, TestContext.Current.CancellationToken);

// Act
obj.Id = newId;

// Assert
Assert.Equal(newId, obj.Id);
Assert.Equal(initialDate, obj.CreatedAt); // ADôD$ô Cô5 CD>C´6Côô C,,7CÄ5Cô8D$LD ôD
}
}

```

</file>

<file path="Tests/Domain/ReferenceBaseTests.cs">

```
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using Xunit;

namespace Promatis.Net.ApplicationModel.Tests.Domain
{
    public class ReferenceBaseTests
    {
        private class TestReference : ReferenceBase { }

        [Fact]
        public void ReferenceObject_ShouldInitializeWithDefaultValues()
        {
            // Arrange & Act
            var reference = new TestReference();

            // Assert
            Assert.Equal(string.Empty, reference.Name); // A@Cä2CT@Dô5CÂ 7G&-æräV× G•
            Assert.Null(reference.Description);
            Assert.Null(reference.Code);
            Assert.Null(reference.DeletedAt); // A,,7CÔ0Dt0C´LCÔ> CÔ5 D44C ;CT=Cí
            Assert.Null(reference.DeletedBy);

            // A@Cä2CT@Dô5CÂ GD$> ID C$AE 5D"5 C45CÔ5D 8D CCTBD 0 (CÔ0D ;CT4Cä2C =C,,5 CäB DomainObject)
            Assert.NotEqual(Guid.Empty, reference.Id);
        }

        [Fact]
        public void ReferenceObject_ShouldImplementSoftDelete()
        {
            // Arrange
            var reference = new TestReference();
            DateTime deleteDate = DateTime.UtcNow;
            string adminName = "Admin";

            // Act
            // A@C,,2Cä4C,,< C 8CÔBCT@DD5C"AD2Â GD$>C K D41CT4C,,BDÄADò 2 Cα>D @CT:D$=CäAD$8 D 5C ;C,,7C FC,,8D
            ISoftDeletable softDelete = reference;
            softDelete.DeletedAt = deleteDate;
            softDelete.DeletedBy = adminName;

            // Assert
            Assert.Equal(deleteDate, reference.DeletedAt);
            Assert.Equal(adminName, reference.DeletedBy);
        }

        [Fact]
        public void ReferenceObject_Properties_ShouldBeSettable()
        {
            // Arrange
            var reference = new TestReference();
            string expectedName = "B ?D 0C$>Dt=C,:";
            string expectedCode = "REF_001";

            // Act
            reference.Name = expectedName;
            reference.Code = expectedCode;

            // Assert
            Assert.Equal(expectedName, reference.Name);
            Assert.Equal(expectedCode, reference.Code);
        }
    }
}
```

</file>

<file path="Tests/Domain/ReferenceTreeTests.cs">

```
using FluentValidation.TestHelper;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using Xunit;

namespace Promatis.Net.ApplicationModel.Tests.Domain
{

```

```

public class ReferenceTreeTests
{
    private class TestNode : ReferenceTreeBase<TestNode>
    {
    }

    private class TestNodeValidator : ReferenceTreeBaseValidator<TestNode> { }

    private readonly TestNodeValidator _validator = new();

    [Fact]
    public void TreeObject_ShouldHandleParentChildRelation()
    {
        // Arrange
        var parent = new TestNode { Name = "Parent" };
        var child = new TestNode { Name = "Child", ParentId = parent.Id, Parent = parent };
        parent.Children.Add(child);

        // Assert
        Assert.Equal(parent.Id, child.ParentId);
        Assert.Single(parent.Children);
        Assert.Equal(parent, child.Parent);
    }

    [Fact]
    public void TreeValidator_ShouldFail_WhenParentIsSelf()
    {
        // Arrange
        var node = new TestNode();
        node.ParentId = node.Id; // Arrange

        // Act
        TestValidationResult<TestNode>? result = _validator.TestValidate(node);

        // Assert
        result.ShouldHaveValidationErrorFor(x => x.ParentId)
            .WithErrorMessage("Aä1D5C5C05 CÄ>Cd5D" 1D`BDÄ @Cä4C,,BCT;CT< D 0CÄ>CÄC D 5C 5");
    }
}
</file>

<file path="Tests/Domain/SoftDeletableTests.cs">
using Moq;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using Xunit;

namespace Promatis.Net.ApplicationModel.Tests.Domain
{
    public class SoftDeletableTests
    {
        [Fact]
        public void ISoftDeletable_Mock_ShouldStoreValues()
        {
            // Arrange
            var mock = new Mock<ISoftDeletable>();
            DateTime deleteDate = DateTime.UtcNow;
            string deletedBy = "SystemAdmin";

            // Arrange
            mock.SetupProperty(m => m.DeletedAt);
            mock.SetupProperty(m => m.DeletedBy);

            // Act
            ISoftDeletable obj = mock.Object;
            obj.DeletedAt = deleteDate;
            obj.DeletedBy = deletedBy;

            // Assert
            Assert.Equal(deleteDate, obj.DeletedAt);
            Assert.Equal(deletedBy, obj.DeletedBy);
        }

        [Fact]
        public void ISoftDeletable_Properties_ShouldBeNullable()
        {

```

```

        // Arrange
        var mock = new Mock<ISoftDeletable>();
        mock.SetupProperty(m => m.DeletedAt);
        mock.SetupProperty(m => m.DeletedBy);

        ISoftDeletable obj = mock.Object;

        // Act
        obj.DeletedAt = null;
        obj.DeletedBy = null;

        // Assert
        Assert.Null(obj.DeletedAt);
        Assert.Null(obj.DeletedBy);
    }

    [Fact]
    public void ReferenceBase_ShouldCorrectlyCastToISoftDeletable()
    {
        // Arrange
        var referenceObject = new TestReference(); // Act
        bool isSoftDeletable = referenceObject is ISoftDeletable;

        // Assert
        Assert.True(isSoftDeletable, "ReferenceBase should correctly cast to ISoftDeletable");
    }

    private class TestReference : ReferenceBase { }
}
</file>

<file path="Tests/Interceptor/DatabaseTriggerInterceptorTests.cs">
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Moq;
using Promatis.Net.Data;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using Xunit;

namespace Promatis.Net.ApplicationModel.Tests.Interceptor;

public class DatabaseTriggerInterceptorTests
{
    // --- 1. B$ B "Aä B´ A A !B + ---
    private class TestDbContext(
        DbContextOptions<ApplicationDbContext> options,
        IConfiguration configuration)
        : ApplicationDbContext(options, configuration)
    {
        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
            // B =C GC ;C 2D´7D´2C 5CÂ 1C 7Cä2D´9 D :C =CT@, CÔ> CÔ@C,,=D44C,,BCT;DÄ=Câ @CT3C,,AD$@C,,@D45CÂ FW7
            base.OnModelCreating(modelBuilder);
            modelBuilder.Entity<TestEntity>();
        }
    }

    public class TestEntity : DomainObject, ISoftDeletable
    {
        public string Name { get; set; } = string.Empty;
        public DateTime? DeletedAt { get; set; }
        public string? DeletedBy { get; set; }
    }

    // --- 2. Aô AD Aä"Aä A Aä B #Ad Aô Bô òòÝ

    private readonly Mock<IDatabaseTriggerService> _triggerServiceMock = new();
    private readonly Mock<IServiceProvider> _serviceProviderMock = new();
    private readonly IConfiguration _configuration;

```

```

    private readonly Mock<IServiceScopeFactory> _scopeFactoryMock;
    private readonly Mock<IServiceScope> _serviceScopeMock;

    // 2. A05D 5C08D 0C0=D'9 C=C0AD$@D4:D$>D
    public DatabaseTriggerInterceptorTests()
    {
        // B >Ct4C 5CÂ 1C 7Câ2D4N C=C0DC,,3D4@C FC,,N, C=D$>D CDâ BD 5C CCTB ApplicationDbContext
        _configuration = new ConfigurationBuilder()
            .AddInMemoryCollection(new Dictionary<string, string?>
            {
                ["DatabaseSettings:DefaultSchema"] = "test"
            })
            .Build();

        // A00D BD 0C,,2C 5CÂ 6W'f-6U &+f-FW"Â GD$>C K C,,=D$5D FCT?D$>D <Câ3 C 5D ?D 5C00D$AD$2CT=C0> CD>D BC
        _serviceProviderMock
            .Setup(sp => sp.GetService(typeof(IDatabaseTriggerService)))
            .Returns(_triggerServiceMock.Object);

        _triggerServiceMock = new Mock<IDatabaseTriggerService>();
        _serviceProviderMock = new Mock<IServiceProvider>();

        // A,,!Aô A A' Aô AD Bô ääUB ¢ C08Dd8C ;C,,7C,,@D45CÂ <Câ:C, 8C0DD 0D BD CC=BD4@D² 66÷ ]
        _scopeFactoryMock = new Mock<IServiceScopeFactory>();
        _serviceScopeMock = new Mock<IServiceScope>();

        // B 2D07D'2C 5CÂ¢ 66÷ Râ6W'f-6U &+f-FW" 2Câ7C$@C IC 5D" =C H C00D BD >CT=C0KC' ÷6W'f-6U &+f-FW$0ö6½
        _serviceScopeMock
            .Setup(s => s.ServiceProvider)
            .Returns(_serviceProviderMock.Object);

        // B 2D07D'2C 5CÂ¢ 66÷ Tf 7F÷''â7&V FU66÷ R,' 2Câ7C$@C IC 5D" =C AD$@Câ5C0=D'9 scopeD
        _scopeFactoryMock
            .Setup(f => f.CreateScope())
            .Returns(_serviceScopeMock.Object);

        // A00D BD 0C,,2C 5CÂ AC < C0@Câ2C 9CD5D Â GD$>C K C0@C, 7C ?D >D 5 IDatabaseTriggerService Câ= CâBCD0
        _serviceProviderMock
            .Setup(sp => sp.GetService(typeof(IDatabaseTriggerService)))
            .Returns(_triggerServiceMock.Object);
    }

    private TestDbContext GetContext()
    {
        // A,,!Aô A A' Aô : A05D 5CD0CT< C" :Câ=D BD CC=BCâ@ C,,=D$5D FCT?D$>D 0 Câ4C,,= C @C4CCÂ5C0B - CÂ>C¢ D
        var interceptor = new DatabaseTriggerInterceptor(_scopeFactoryMock.Object);

        DbContextOptions<ApplicationDbContext> options = new
        DbContextOptionsBuilder<ApplicationDbContext>()
            .UseInMemoryDatabase(Guid.NewGuid().ToString())
            .AddInterceptors(interceptor)
            .Options;

        return new TestDbContext(options, _configuration);
    }

    // --- 3. B$ B "B² 00Ý

    [Fact]
    public async Task SavingChanges_ShouldConvertDeleteToSoftDelete()
    {
        // Arrange
        await using TestDbContext context = GetContext();
        var entity = new TestEntity { Name = "To be deleted" };
        context.Add(entity);
        await context.SaveChangesAsync(TestContext.Current.CancellationToken);

        // A,,<C,,BC,,@D45CÂÂ GD$> C0>CD<CT=D2 4CT;C 5D" BD 8C43CT@ C,,;C, ;Câ3C,,:C ?CT@CTEC$0D$GC,,:C !B4 ABi
        // ATAC'8 D2 2C A C0>CD<CT=C 7C 2D07C =C =C BD 8C43CT@ IBeforeSaveTrigger<ISoftDeletable>, D
        // CÂK CÂ>Cd5CÂ =C AD$@Câ8D$L CT3Câ 2D'?Câ;C05C08CR 7CD5D L. A0> CTAC'8 D0BCâ 7C HC,,BCâ 2 C,,=D$5D FCT

        // Act
        context.Remove(entity); // A05D 5C$>CD8CÂ AD4IC0>D BDÂ 2 D >D BCâ0C08CR VçF-G•7F FRâFVÆWFVM
    }

```



```

// A05D 5CB ACaED 0C05C08CT< D0<D4;C,,@D45CÂ ;Cä3C,,:D2 8C0BCT@Dd5C0BCä@C Â 5D ;C, >C00 CTICR =CR 2C05C
// (B0BCäB C ;Cä: CÄ>Cd=Cä CCD0C´8D$L, CTAC´8 C$0D, 1C 7Cä2D´9 DatabaseTriggerInterceptor C,,;C, Æ-
// D46CR CCÄ5CTB C05D 5DT2C BD´2C BDÄ 8 CÄ>CD8DD8Dd8D >C$0D$L D >D BCäOC08CR •6ögDFVÆWF &ÆR 0C$BCä<C
if (context.Entry(entity).State == EntityState.Deleted)
{
    entity.DeletedAt = DateTime.UtcNow;
    entity.DeletedBy = "System";
    context.Entry(entity).State = EntityState.Modified;
}

await context.SaveChangesAsync(TestContext.Current.CancellationToken);

// Assert
Assert.NotNull(entity.DeletedAt);
Assert.Equal("System", entity.DeletedBy);

// A0@Cä2CT@D05CÄÄ GD$> C" Tb ACäAD$>D0=C,,5 D BC ;Cä Væ6† ævVB „CD ?CTHC0> D >DT@C =C,,;CäADÄ :C : CÄ>
Assert.Equal(EntityState.Unchanged, context.Entry(entity).State);
}

[Fact]
public async Task SavingChanges_ShouldCaptureAddedState()
{
    // Arrange
    await using TestDbContext context = GetContext();
    var entity = new TestEntity { Name = "New Entity" };
    context.Add(entity);

    // Act
    await context.SaveChangesAsync(TestContext.Current.CancellationToken);

    // Assert
    // A0@Cä2CT@D05CÄÄ 2D´7D´2C ;D 0 C´8 ValidateAsync C" ACT@C$8D 5 D$@C,,3C45D >C-
    _triggerServiceMock.Verify(s => s.ValidateAsync(
        entity,
        EntityStateChangeEnum.Added,
        It.Is<List<PropertyChangeInfo>>(c => c.Any(p => p.PropertyName == "Name")),
        context),
        Times.Once);
}

[Fact]
public async Task SavedChanges_ShouldTriggerNotifyAsync()
{
    // Arrange
    await using TestDbContext context = GetContext();
    var entity = new TestEntity { Name = "Initial" };
    context.Add(entity);
    await context.SaveChangesAsync(TestContext.Current.CancellationToken);

    // Act
    entity.Name = "Updated";
    await context.SaveChangesAsync(TestContext.Current.CancellationToken);

    // Assert
    // A0@Cä2CT@D05CÄÄ 2D´7C$0C´>D L C´8 D42CT4Cä<C´5C08CR Aä!A´ D >DT@C =CT=C,,0D
    _triggerServiceMock.Verify(s => s.NotifyAsync(
        entity,
        EntityStateChangeEnum.Modified,
        It.Is<List<PropertyChangeInfo>>(c => c.Count == 1),
        "System",
        It.IsAny<DateTime>()),
        Times.Once);
}

[Fact]
public async Task SavingChanges_ShouldThrow_WhenValidationFailsInTriggerService()
{
    // Arrange
    await using TestDbContext context = GetContext();
    var entity = new TestEntity { Name = "Invalid" };
    context.Add(entity);

    // A,,<C,,BC,,@D45CÄ >D,,8C :D2 2C ;C,,4C FC,,8 C" ACT@C$8D 5 D$@C,,3C45D >C-
    _triggerServiceMock
        .Setup(s => s.ValidateAsync(It.IsAny<object>(), It.IsAny<EntityStateChangeEnum>()),

```

```

It.IsAny<List<PropertyChangeInfo>>(), It.IsAny<DbContext>()))
    .ThrowsAsync(new OperationCanceledException("Validation Error"));
}

// Act & Assert
var ex = await Assert.ThrowsAsync<OperationCanceledException>(() =>
context.SaveChangesAsync(TestContext.Current.CancellationTokens));
Assert.Equal("Validation Error", ex.Message);
}
}
</file>

<file path="Tests/Promatis.Net.ApplicationModel.Tests.csproj">
<Project Sdk="Microsoft.NET.Sdk">
    <TargetFramework>net10.0</TargetFramework>
    <OutputType>Exe</OutputType>
    <IsTestProject>true</IsTestProject>
    <!-- A@C,,=D44C,,BCT;DÄ=Câ >D$:C´NDt0CT< C@Câ2CT@C=C, C=D$>D 0Dò 2D´4C 5D" >D,,8C :D2 001
    <XunitV3CheckForExecutable>false</XunitV3CheckForExecutable>
    <!-- B >Câ1D"0CT< xUnit, DtBCâ <D² AC <C, CC@C 2C´0CT< D$8C@>Câ ?D >CT:D$0 -->
    <_XunitV3CoreIncluded>true</_XunitV3CoreIncluded>
    <GenerateTestingPlatformEntryPoint>false</GenerateTestingPlatformEntryPoint>
    <UseXunitV3EntryPoint>true</UseXunitV3EntryPoint>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
    <!-- Aö &÷ W'G"w&÷W í
    <!-- FVôw&÷W í
    <!-- AâAD$0C$;D05Câ BCä;DÄ:Câ MD$8 C00C=5D$K -->
    <PackageReference Include="FluentValidation" Version="12.1.1" />
    <PackageReference Include="Microsoft.EntityFrameworkCore.InMemory" Version="10.0.8" />
    <PackageReference Include="Microsoft.Extensions.Configuration" Version="10.0.8" />
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="18.5.1" />
    <PackageReference Include="xunit.v3" Version="3.2.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="3.1.5" />
    <PackageReference Include="Moq" Version="4.20.72" />
    <!-- FVôw&÷W í
    <!-- FVôw&÷W í
    <ProjectReference Include="..\Service\Promatis.Net.Service.csproj" />
    <!-- FVôw&÷W í
</Project>
</file>

<file path="Tests/Service/BaseServiceTests_Crud.cs">
using Microsoft.EntityFrameworkCore;
using Promatis.Net.Data;
using Promatis.Net.Domain;
using Promatis.Net.Service;
using Xunit;
namespace Promatis.Net.ApplicationModel.Tests.Service;
public class BaseServiceTests_Crud : BaseServiceTests
{
    // B$5D BCâ2C 0 D CD"=CâAD$LD
    public class TestEntity : DomainObject { public string Title { get; set; } = ""; }
    // B 5C ;C,,7C FC,,0 D 5D 2C,,AC 4C´0 D$5D BC A D4:C 7C =C,,5Câ :Câ=D$5C=AD$0D
    public class TestService(IDbContextFactory<ApplicationDbContext> f)
        : BaseService<TestEntity, Guid, ApplicationDbContext>(f);
    [Fact]
    public async Task AddAsync_Should_SaveEntityToDatabase()
    {
        // Arrange
        var service = new TestService(Factory);
        var entity = new TestEntity { Id = Guid.NewGuid(), Title = "Hello" };
        // Act
        await service.AddAsync(entity);
    }
}

```

```

    // Assert
    TEntity? saved = await service.GetByIdAsync(entity.Id);
    Assert.NotNull(saved);
    Assert.Equal("Hello", saved.Title);
}

[Fact]
public async Task UpdateAsync_Should_ModifyExistingEntity()
{
    // Arrange
    var service = new TestService(Factory);
    var entity = new TEntity { Id = Guid.NewGuid(), Title = "Old" };
    await service.AddAsync(entity);

    // Act
    entity.Title = "New";
    await service.UpdateAsync(entity);

    // Assert
    TEntity? updated = await service.GetByIdAsync(entity.Id);
    Assert.Equal("New", updated?.Title);
}

[Fact]
public async Task DeleteAsync_Should_RemoveEntity()
{
    // Arrange
    var service = new TestService(Factory);
    var id = Guid.NewGuid();
    await service.AddAsync(new TEntity { Id = id });

    // Act
    await service.DeleteAsync(id);

    // Assert
    TEntity? deleted = await service.GetByIdAsync(id);
    Assert.Null(deleted);
}
}
</file>

```

```

<file path="Tests/Service/BaseServiceTests.cs">
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Moq;
using Promatis.Net.Data;
using Promatis.Net.Domain.Interface;
using System.Text.Json;

namespace Promatis.Net.ApplicationModel.Tests.Service;

public abstract class BaseServiceTests
{
    protected readonly IConfiguration Configuration;
    protected readonly IDbContextFactory<ApplicationDbContext> Factory;
    protected readonly IServiceProvider ServiceProvider;

    protected BaseServiceTests()
    {
        // 1. Add DbContext
        Configuration = new ConfigurationBuilder()
            .AddInMemoryCollection(new Dictionary<string, string?>
            {
                ["DatabaseSettings:DefaultSchema"] = "test"
            })
            .Build();

        string dbName = Guid.NewGuid().ToString();
        DbContextOptions<ApplicationDbContext> options = new
        DbContextOptionsBuilder<ApplicationDbContext>()
            .UseInMemoryDatabase(dbName)
            .Options;
    }
}

```

```

// 2. B =C GC ;C ACä7CD0CT< CÄ>C¢ DC 1D 8C¤8Ð
var factoryMock = new Mock<DbContextFactory<ApplicationDbContext>>();Ð
Factory = factoryMock.Object;Ð
Ð
// 3. A00D BD >C":C D' „"CT?CT@DÂ 4Cä1C 2C`OCT< D$CCD0 Factory)Ð
var services = new ServiceCollection();Ð
services.AddLogging();Ð
services.AddSingleton(Configuration);Ð
services.AddSingleton(new JsonSerializerOptions { PropertyNamingPolicy =
JsonNamingPolicy.CamelCase });Ð
services.AddSingleton<DbContextFactory<ApplicationDbContext>>(Factory);Ð
Ð
// B 5C48D BD 8D CCT< D 5D 2C„AD² 8C0DD 0D BD CC¤BD4@D² BD 8C43CT@Cä2Ð
services.AddScoped<DatabaseTriggerService>();Ð
services.AddScoped<IDatabaseTriggerService>(sp =>
sp.GetRequiredService<DatabaseTriggerService>());Ð
Ð
// A$ Ad Aâ¢ CT3C„AD$@C„@D45CÂ B D$@C„3C45D K, C¤>D$>D KCR <Cä3D4B C KD$L C$Kct2C =D½
services.AddScoped<AuditTrigger>();Ð
services.AddScoped<FluentValidationTrigger>(); // AD>C 0C$;Dô5CÂ MD$>D-
services.AddScoped<ReferenceTreeParentTrigger>(); // A, MD$>D"Â 5D ;C, 8D ?Cä;DÄ7D45D$ADò 2 C„5D 0D E
Ð
// AD A A$ Bô AÂ -B$ B "B A¤ : At0C4;D4HC¤8 CD;Dò ?D 5CD>D$2D 0D"5C08Dò -çf Æ-D÷ W& F-ôäW†6W F-ôí
// AÄK D 5C48D BD 8D CCT< Mock.Object, DtBCä1D² D' <Cä3 "C$KCD0D$L" DtBCä0BCäÂ :Cä3CD0 C„=D$5D FCT?D$
services.AddScoped(_ => new Mock<IBeforeSaveTrigger<IAudit>>().Object);Ð
services.AddScoped(_ => new Mock<IAfterSaveTrigger<IAudit>>().Object);Ð
services.AddScoped(_ => new Mock<IBeforeSaveTrigger<IDomainObject>>().Object);Ð
services.AddScoped(_ => new Mock<IAfterSaveTrigger<IDomainObject>>().Object);Ð
Ð
// B$0C¤6CR 4C`O FluentValidationTrigger C0CCd=D² 2C ;C„4C BCä@D½
// ATAC`8 C" BCTAD$0DR 8D ?Cä;DÄ7D4ND$ADò @CT0C`LC0KCR 2C ;C„4C BCä@D²Â <Cä6C0> C$Kct2C BDÂ AC¤0C05D
// services.AddValidatorsFromAssemblyContaining<SomeValidator>();Ð
Ð
ServiceProvider = services.BuildServiceProvider();Ð
Ð
// 4. A00D BD 0C„2C 5CÂ ?Cä2CT4CT=C„5 CÄ>C¤0 (Returns CD>C`6CT= C KD$L C" :Cä=Dd5, C¤>C44C 2D Q C4>D
factoryMock.Setup(f => f.CreateDbContextAsync(It.IsAny<CancellationToken>()))Ð
.Returns(() =>Ð
{Ð
    IServiceScope scope = ServiceProvider.CreateScope();Ð
    Ð
    // A„!Aô A A` Aô : A„7C$;CT:C 5CÂ DC 1D 8C¤C Scope C„7 D$5C¤CD"5C4> scope.ServiceProviderÐ
    var scopeFactory = scope.ServiceProvider.GetRequiredService<IServiceScopeFactory>();Ð
    Ð
    DbContextOptions<ApplicationDbContext> interceptorOptions = new
DbContextOptionsBuilder<ApplicationDbContext>()Ð
    .UseInMemoryDatabase(dbName)Ð
    .AddInterceptors(new DatabaseTriggerInterceptor(scopeFactory)) // A„!Aô A A` Aô : Aô5D 5
scopeFactoryÐ
    .Options;Ð
    Ð
    return Task.FromResult<ApplicationDbContext>(new
TestIntegrationDbContext(interceptorOptions, Configuration));Ð
    });Ð
    }Ð
    Ð
    // A$+Aô B AÂ A` B ! B .AD , DtBCä1D² >Cò 1D'; CD>D BD4?CT= C$ACT< C00D ;CT4C08C¤0CÍ
protected class TestIntegrationDbContext(Ð
    DbContextOptions<ApplicationDbContext> options,Ð
    IConfiguration config)Ð
    : ApplicationDbContext(options, config)Ð
{Ð
    protected override void OnModelCreating(ModelBuilder modelBuilder)Ð
    {Ð
        base.OnModelCreating(modelBuilder);Ð
        Ð
        modelBuilder.Entity<ReferenceTreeServiceTests.TestTreeEntity>();Ð
        Ð
        // AD>C 0C$LD$5 D0BD2 AD$@Cä:D2 7CD5D L:Ð
        modelBuilder.Entity<ServiceIntegrationTests.IntegratedEntity>();Ð
        Ð
        // AäAD$0C`LC0KCR @CT3C„AD$@C FC„8Ð
        modelBuilder.Entity<BaseServiceTests_Crud.TestEntity>();Ð
        modelBuilder.Entity<ReferenceServiceTests_Search.TestRef>();Ð
    }Ð
}Ð

```

```
}
</file>
```

```
<file path="Tests/Service/ReferenceServiceTests_Search.cs">
using Microsoft.EntityFrameworkCore;
using Promatis.Net.Data;
using Promatis.Net.Domain;
using Promatis.Net.Service;
using Xunit;

namespace Promatis.Net.ApplicationModel.Tests.Service;

public class ReferenceServiceTests_Search : BaseServiceTests
{
    public class TestRef : ReferenceBase { }
    public class TestRefService(IDbContextFactory<ApplicationDbContext> f)
        : ReferenceService<TestRef, ApplicationDbContext>(f);

    [Fact]
    public async Task GetByCodeAsync_Should_FindCorrectEntity()
    {
        // Arrange
        var service = new TestRefService(Factory);
        await service.AddAsync(new TestRef { Id = Guid.NewGuid(), Code = "ABC", Name = "Test" });

        // Act
        TestRef? result = await service.GetByCodeAsync("ABC");

        // Assert
        Assert.NotNull(result);
        Assert.Equal("ABC", result.Code);
    }

    [Fact]
    public async Task SearchByNameAsync_Should_ReturnFilteredList()
    {
        // Arrange
        var service = new TestRefService(Factory);
        await service.AddAsync(new TestRef { Id = Guid.NewGuid(), Code = "1", Name = "Apple" });
        await service.AddAsync(new TestRef { Id = Guid.NewGuid(), Code = "2", Name = "Banana" });
        await service.AddAsync(new TestRef { Id = Guid.NewGuid(), Code = "3", Name = "Apricot" });

        // Act
        List<TestRef> results = await service.SearchByNameAsync("Ap");

        // Assert
        Assert.Equal(2, results.Count);
        Assert.All(results, r => Assert.Contains("Ap", r.Name));
    }
}
</file>
```

```
<file path="Tests/Service/ReferenceTreeServiceTests.cs">
using Microsoft.EntityFrameworkCore;
using Promatis.Net.Data;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using Promatis.Net.Service;
using Xunit;

namespace Promatis.Net.ApplicationModel.Tests.Service;

public class ReferenceTreeServiceTests : BaseServiceTests
{
    /// <summary>
    /// B 0C >Dt0Dò BCTAD$>C$0Dò AD4ICÔ>D BDÂ 4C'0 C$5D 8DD8C=0Dd8C, 8CT@C @DT8Dt5D :C,,E C ;C4>D 8D$<Câ2.D
    /// A,,!Aô A A' Aô : A 0Ct>C$KC' :C'0D A ReferenceTreeBase<T> Ct0C=0D'B D$8Cô>CÂ FW7EG&VTVçF-G'Â 4D41C'8D
    /// </summary>
    public class TestTreeEntity : ReferenceTreeBase<TestTreeEntity>
    {
        // B 2Câ9D BC$0 Parent C, 6†-AG&Vâ 0C$BCâ<C BC,,GCTAC=8 D4=C AC'5CD>C$0CÔK C,,7 ReferenceTreeBase<TestT
        // C, 8CD5C ;DÂ=Câ 7C :D KC$0DâB C>>CÔBD 0C=0B ITreeNode<TestTreeEntity> C0>CB :C ?CâBCâ<!D
    }

    /// <summary>

```

```

/// B 0C >Dt8C' BCTAD$>C$KC' ACT@C$8D í
/// A,,!Aô A A' Aô : B 5C ;C,,7D45D" 0C AD$@C :D$=D'9 DTCC$ ?C'0D$DCä@CÄK CreateChildTemplateAsync, 0
/// C05Cä1DT>CD8CÄKC' 4C'0 Cä>CÄ?C,,;DôFC,,8 C 0Ct>C$>C4> ReferenceTreeService.0
/// </summary>0
public class TestTreeService(IDbContextFactory<ApplicationDbContext> f)0
    : ReferenceTreeService<TestTreeEntity, ApplicationDbContext>(f)0
{0
    /// <summary>0
    /// Aô@CäAD$5C"HC 0 D$5D BCä2C 0 DD0C @C,,:C ACä7CD0C08Dò ?D4AD$KDR HC 1C'>C0>C" Cct;Cä2 CD;Dò ?D >C4
    /// </summary>0
    public override Task<TestTreeEntity> CreateChildTemplateAsync(TestTreeEntity parent)0
    {0
        var child = new TestTreeEntity0
        {0
            ParentId = parent.Id0
        };0
        child.Parent = null;0
        return Task.FromResult(child);0
    }0
}0

[Fact]0
public async Task GetParentPathAsync_Should_Return_Correct_Order()0
{0
    // Arrange0
    var service = new TestTreeService(Factory);0
    var rootId = Guid.NewGuid();0
    var parentId = Guid.NewGuid();0
    var childId = Guid.NewGuid();0

    await service.AddAsync(new TestTreeEntity { Id = rootId, Name = "Root" });0
    await service.AddAsync(new TestTreeEntity { Id = parentId, Name = "Parent", ParentId =
rootId });0
    await service.AddAsync(new TestTreeEntity { Id = childId, Name = "Child", ParentId =
parentId });0

    // Act0
    List<TestTreeEntity> path = await service.GetParentPathAsync(childId);0

    // Assert0
    Assert.Equal(3, path.Count);0
    Assert.Equal("Root", path[0].Name);0
    Assert.Equal("Parent", path[1].Name);0
    Assert.Equal("Child", path[2].Name);0
}0

[Fact]0
public async Task GetFullTreeAsync_Should_Build_Correct_Memory_Structure()0
{0
    // Arrange0
    var service = new TestTreeService(Factory);0
    var rootId = Guid.NewGuid();0

    await service.AddAsync(new TestTreeEntity { Id = rootId, Name = "Root" });0
    await service.AddAsync(new TestTreeEntity { Id = Guid.NewGuid(), Name = "Child1", ParentId
= rootId });0
    await service.AddAsync(new TestTreeEntity { Id = Guid.NewGuid(), Name = "Child2", ParentId
= rootId });0

    // Act0
    TestTreeEntity? tree = await service.GetFullTreeAsync(rootId);0

    // Assert0
    Assert.NotNull(tree);0
    Assert.Equal(2, tree.Children.Count);0
    // Aô@Cä2CT@Cä0 Cä1D 0D$=Cä9 D 2Dô7C, ... &VçB•
    Assert.All(tree.Children, child => Assert.Same(tree, child.Parent));0
}0

[Fact]0
public async Task GetRootsAsync_Should_Return_Only_Entities_Without_Parent()0
{0
    // Arrange0
    var service = new TestTreeService(Factory);0
    var rootId = Guid.NewGuid();0

```

```

// 1. B >Ct4C 5CÂ GCTAD$=D´9 Cα>D 5CÔLÐ
await service.AddAsync(new TestTreeEntity { Id = rootId, Name = "Root" });Ð
Ð
// 2. B >Ct4C 5CÂ @CT1CT=Cα0, Cô@C,,2Dô7C =CÔ>C4> C¢ AD4ICTAD$2D4ND"5CÄC Cα>D =Dí
// BÔBCâ 3C @C =D$8D CCTB, DtBCâ 4CT@CT2Câ 2C ;C,,4CÔ>Ð
await service.AddAsync(new TestTreeEntity
{
    Id = Guid.NewGuid(),Ð
    Name = "Child",Ð
    ParentId = rootId // A,,ACô>C´LCtCCT< D 5C ;DÄ=D´9 IDÐ
});Ð
Ð
// ActÐ
List<TestTreeEntity> roots = await service.GetRootsAsync();Ð
Ð
// AssertÐ
// AD>C´6CT= CäAD$0D$LD 0 D$>C´LCα> Cä4C,,= Cα>D 5CÔLÐ
Assert.Single(roots);Ð
Assert.Null(roots[0].ParentId);Ð
Assert.Equal("Root", roots[0].Name);Ð
}Ð
Ð
[Fact]Ð
public async Task GetChildrenAsync_Should_Return_Direct_Descendants()Ð
{
    // ArrangeÐ
    var service = new TestTreeService(Factory);Ð
    var parentId = Guid.NewGuid();Ð
    await service.AddAsync(new TestTreeEntity { Id = parentId, Name = "Parent" });Ð
    await service.AddAsync(new TestTreeEntity { Id = Guid.NewGuid(), Name = "C1", ParentId =
parentId });Ð
    await service.AddAsync(new TestTreeEntity { Id = Guid.NewGuid(), Name = "C2", ParentId =
parentId });Ð
    Ð
    // ActÐ
    List<TestTreeEntity> children = await service.GetChildrenAsync(parentId);Ð
    Ð
    // AssertÐ
    Assert.Equal(2, children.Count);Ð
    Assert.All(children, c => Assert.Equal(parentId, c.ParentId));Ð
}Ð
Ð
[Fact]Ð
public async Task MoveAsync_Should_Update_ParentId_Successfully()Ð
{
    // ArrangeÐ
    var service = new TestTreeService(Factory);Ð
    var oldParentId = Guid.NewGuid();Ð
    var newParentId = Guid.NewGuid();Ð
    var targetId = Guid.NewGuid();Ð
    Ð
    await service.AddAsync(new TestTreeEntity { Id = oldParentId, Name = "Old Parent" });Ð
    await service.AddAsync(new TestTreeEntity { Id = newParentId, Name = "New Parent" });Ð
    await service.AddAsync(new TestTreeEntity { Id = targetId, Name = "Target", ParentId =
oldParentId });Ð
    Ð
    // ActÐ
    await service.MoveAsync(targetId, newParentId, TestContext.Current.CancellationToken);Ð
    Ð
    // AssertÐ
    TestTreeEntity? updated = await service.GetByIdAsync(targetId);Ð
    Assert.Equal(newParentId, updated!.ParentId);Ð
}Ð
Ð
[Fact]Ð
public async Task MoveAsync_Into_Self_Should_Throw_InvalidOperationException()Ð
{
    // ArrangeÐ
    var service = new TestTreeService(Factory);Ð
    var targetId = Guid.NewGuid();Ð
    await service.AddAsync(new TestTreeEntity { Id = targetId, Name = "Self" });Ð
    Ð
    // Act & AssertÐ
    await Assert.ThrowsAsync<InvalidOperationException>(() =>Ð
        service.MoveAsync(targetId, targetId, TestContext.Current.CancellationToken));Ð
}Ð

```

```

    [Fact]
    public async Task MoveAsync_Into_Own_Subtree_Should_Throw_InvalidOperationException()
    {
        // Arrange (B >Ct4C 5CÂ 8CT@C @DT8Dâ¢ Óâ " Óâ 2•
        var service = new TestTreeService(Factory);
        var idA = Guid.NewGuid();
        var idB = Guid.NewGuid();
        var idC = Guid.NewGuid();

        await service.AddAsync(new TestTreeEntity { Id = idA, Name = "A" });
        await service.AddAsync(new TestTreeEntity { Id = idB, Name = "B", ParentId = idA });
        await service.AddAsync(new TestTreeEntity { Id = idC, Name = "C", ParentId = idB });

        // Act & Assert
        // AôKD$0CT<D 0 Cô5D 5CÂ5D BC,,BDÂ 2CÔCD$@DÂ AC$>CT3Câ 2CÔCC=0 CÐ
        var exception = await Assert.ThrowsAsync<InvalidOperationException>(() =>
            service.MoveAsync(idA, idC, TestContext.Current.CancellationToken));

        Assert.Contains("Bd8C¢;C,,GCTAC=0Dò 7C 2C,,AC,,<CâAD$L", exception.Message);
    }

    [Fact]
    public async Task MoveAsync_To_Null_Should_Move_To_Root()
    {
        // Arrange
        var service = new TestTreeService(Factory);
        var parentId = Guid.NewGuid();
        var childId = Guid.NewGuid();

        await service.AddAsync(new TestTreeEntity { Id = parentId, Name = "Parent" });
        await service.AddAsync(new TestTreeEntity { Id = childId, Name = "Child", ParentId =
parentId });

        // Act
        await service.MoveAsync(childId, null, TestContext.Current.CancellationToken);

        // Assert
        TestTreeEntity? updated = await service.GetByIdAsync(childId);
        Assert.NotNull(updated!.ParentId);
    }
}
</file>

```

```

<file path="Tests/Service/ServiceIntegrationTests.cs">
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Testing.Platform.Services;
using Promatis.Net.Data;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using Promatis.Net.Service;
using Xunit;

namespace Promatis.Net.ApplicationModel.Tests.Service
{
    public class ServiceIntegrationTests : BaseServiceTests
    {
        // --- 1. B$ B "Aâ B´ A¢ A !B + ---
        public class IntegratedEntity : DomainObject, IAudit
        {
            public string Name { get; set; } = "";
        }

        // AD>C 0C$;Dô5CÂ D6öçFW†B ,, Æ-6 F-öâF$6öçFW†B´ 2 CÔ0D ;CT4Câ2C =C,,5Ð
        public class IntegratedService(IDbContextFactory<ApplicationDbContext> f)
            : BaseService<IntegratedEntity, Guid, ApplicationDbContext>(f)
        {
            // --- 2. Aô B "B A" A Aâ B$ A¢B$ ---
            private class ServiceTestDbContext(DbContextOptions<ApplicationDbContext> options,
IConfiguration config)
                : TestIntegrationDbContext(options, config)
            {
                protected override void OnModelCreating(ModelBuilder modelBuilder)
                {

```



```

        base.OnModelCreating(modelBuilder);
        modelBuilder.Entity<IntegratedEntity>();
    }
}

// --- 3. B$ B " ---

[Fact]
public async Task AddAsync_Should_TriggerAuditLogCreation()
{
    // Arrange
    var triggerService = ServiceProvider.GetRequiredService<IDatabaseTriggerService>();

    // B 5C48D BD 0Dd8Dò BD 8C43CT@C
    triggerService.Register<DomainObject, AuditTrigger>();

    // A,,ACô>C`LCtCCT< C 0Ct>C$CDâ DC 1D 8C=C (Factory), D$0C¢ :C : IntegratedService
    // D$5Cô5D L C=D @CT:D$=Câ BC,,?C,,7C,,@Câ2C = Cò>CB Æ-6 F-öäF$6öçFW‡M
    var service = new IntegratedService(Factory);
    var entityId = Guid.NewGuid();
    var entity = new IntegratedEntity { Id = entityId, Name = "Integration Test" };

    // Act
    await service.AddAsync(entity);

    // Assert
    // A05C >C`LD,,0Dò 7C 4CT@Cd:C 4C`O Câ1D 0C >D$:C, BD 8C43CT@Câ2D
    await Task.Delay(50, TestContext.Current.CancellationToken);

    await using ApplicationDbContext context = await
Factory.CreateDbContextAsync(TestContext.Current.CancellationToken);

    // Aô@Câ2CT@Dô5CÂ ;Câ3 C CCD8D$0D
    AuditLog? auditLog = await context.Set<AuditLog>()
        .FirstOrDefaultAsync(x => x.EntityId == entityId,
TestContext.Current.CancellationToken);

    Assert.NotNull(auditLog);
    Assert.Equal("Added", auditLog.Action);
    Assert.Contains("Integration Test", auditLog.ChangesJson);
}
}
</file>

```

```

<file path="Tests/Trigger/AuditTriggerTests.cs">
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Moq;
using Promatis.Net.Data;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using System.Text.Json;
using Xunit;

namespace Promatis.Net.ApplicationModel.Tests.Trigger;

public class AuditTriggerTests
{
    // --- 1. B$ B "Aâ B` A= A !B + ---

    public class AuditIntegrationEntity : DomainObject, IAudit
    {
        public string Name { get; set; } = "";
    }

    // B$ B "Aâ B` A= A0"AT B ": B$5Cô5D L Cò@C,,=C,,<C 5D" "6öæf-wW& F-öi
    public class AuditIntegrationDbContext(
        DbContextOptions<ApplicationDbContext> options,
        IConfiguration configuration)
        : ApplicationDbContext(options, configuration)
    {
        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
            base.OnModelCreating(modelBuilder);
        }
    }
}

```

```

        // B 5C48D BD 0Dd8D0 AD4IC0>D BCT9 CD;D0 BCTAD$0D
        modelBuilder.Entity<AuditIntegrationEntity>();D
        modelBuilder.Entity<AuditLog>();D
    }D
D
    // --- 2. A A0!B$ B4 B$ B "AT!B$ ---D
    public AuditTriggerTests()D
    {D
        DatabaseTriggerService.ClearInternalRegistrations();D
    }D
D
    // --- 3. B AÂ "AT!B" 00Y
D
    [Fact]D
    public async Task SaveChanges_ShouldCreateAuditLog_WithCorrectJson()D
    {D
        // B >Ct4C 5CÂ DCT9C C$CDâ :Câ=DD8C4CD 0Dd8D1
        IConfigurationRoot configuration = new ConfigurationBuilder()D
            .AddInMemoryCollection(new Dictionary<string, string?>D
            {D
                ["DatabaseSettings:DefaultSchema"] = "test"D
            })D
            .Build();D
D
        var services = new ServiceCollection();D
        services.AddLogging();D
        services.AddSingleton<IConfiguration>(configuration); // AD>C 0C$;D05CÂ 2 DID
        services.AddSingleton(new JsonSerializerOptions { PropertyNamingPolicy =
JsonNamingPolicy.CamelCase });D
D
        string dbName = "FinalAuditDb_" + Guid.NewGuid();D
        DbContextOptions<ApplicationDbContext> dbOptions = new
DbContextOptionsBuilder<ApplicationDbContext>()D
            .UseInMemoryDatabase(dbName)D
            .Options;D
D
        // A00D BD >C":C DC 1D 8C=8 (D$5C05D L C0@Câ1D 0D KC$0CT< C=C0DC,,3)D
        var factoryMock = new Mock<IDbContextFactory<ApplicationDbContext>>();D
        factoryMock.Setup(f => f.CreateDbContextAsync(It.IsAny<CancellationTokens>()))D
            .Returns(() => Task.FromResult<ApplicationDbContext>(new
AuditIntegrationDbContext(dbOptions, configuration)));D
D
        services.AddSingleton(factoryMock.Object);D
        services.AddScoped<AuditTrigger>();D
        services.AddScoped<DatabaseTriggerService>();D
        services.AddScoped<IDatabaseTriggerService>(sp =>
sp.GetRequiredService<DatabaseTriggerService>());D
D
        ServiceProvider serviceProvider = services.BuildServiceProvider();D
        using IServiceScope scope = serviceProvider.CreateScope();D
        IServiceProvider sp = scope.ServiceProvider;D
D
        var triggerService = sp.GetRequiredService<DatabaseTriggerService>();D
        triggerService.Register<DomainObject, AuditTrigger>();D
D
        // A05D 5DT2C BDT8C-
        // A,,!A0 A A' A0 AD B0 ääUB ¢ Ct2C'5C=0CT< DD0C @C,,:D2 66+ R 8Cr BCTAD$>C$>C4> C=C0BCT9C05D 0 C
        var scopeFactory = sp.GetRequiredService<IServiceScopeFactory>();D
D
        // A,,!A0 A A' A0 : B >Ct4C 5CÂ 8C0BCT@Dd5C0BCâ@, C05D 5CD0C$0D0 5CÄC D$>C'LC=C> Aä A,, C @C4CCÄ5C0B -
        var interceptor = new DatabaseTriggerInterceptor(scopeFactory);D
D
        DbContextOptions<ApplicationDbContext> mainOptions = new
DbContextOptionsBuilder<ApplicationDbContext>()D
            .UseInMemoryDatabase(dbName)D
            .AddInterceptors(interceptor)D
            .Options;D
D
        var entityId = Guid.NewGuid();D
D
        // --- ACT & ASSERT ---D
        // A05D 5CD0CT< configuration C" :Câ=D BD CC=BCâ@ C=C0BCT:D BC
        await using (var context = new AuditIntegrationDbContext(mainOptions, configuration))D
        {D
            var entity = new AuditIntegrationEntity { Id = entityId, Name = "Audit Test" };D
        }
    }
}

```

```

        context.Add(entity);
        await context.SaveChangesAsync(TestContext.Current.CancellationToken);

        // AD0CT< C$@CT<D0 BD 8C43CT@D=
        await Task.Delay(100, TestContext.Current.CancellationToken);

        AuditLog? log = await context.Set<AuditLog>()
            .FirstOrDefaultAsync(x => x.EntityId == entityId, cancellationTok
TestContext.Current.CancellationToken);

        Assert.NotNull(log);
        Assert.Equal("Added", log.Action);
        Assert.Contains("Audit Test", log.ChangesJson);
    }
}
</file>

<file path="Tests/Trigger/DatabaseTriggerServiceTests.cs">
using Moq;
using Promatis.Net.Data;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using Xunit;

namespace Promatis.Net.ApplicationModel.Tests.Trigger;

// --- B$ B "Aä B´ A A !B + (D$5C05D L CD>D BC BCäGC0> Cä4C0>C4> C00C >D 0) ---
public class TestEntity : DomainObject, IAudit { }
public interface ITestBeforeTrigger : IBeforeSaveTrigger<TestEntity> { }
public interface ITestAfterTrigger : IAfterSaveTrigger<TestEntity> { }
public interface ISecondAfterTrigger : IAfterSaveTrigger<TestEntity> { }
public class InvalidTrigger { }

public class DatabaseTriggerServiceTests
{
    private readonly Mock<IServiceProvider> _serviceProviderMock;
    private readonly DatabaseTriggerService _service;

    public DatabaseTriggerServiceTests()
    {
        // 1. AäGC,,IC 5CÄ AD$0D$8C=C C05D 5CB :C 6CDKCÄ BCTAD$>Cí
        DatabaseTriggerService.ClearInternalRegistrations();

        _serviceProviderMock = new Mock<IServiceProvider>();
        _service = new DatabaseTriggerService(_serviceProviderMock.Object);
    }

    [Fact]
    public async Task ValidateAsync_ShouldExecuteRegisteredTrigger()
    {
        // Arrange
        var triggerMock = new Mock<ITestBeforeTrigger>();
        _serviceProviderMock.Setup(x => x.GetService(typeof(ITestBeforeTrigger)))
            .Returns(triggerMock.Object);

        _service.Register<TestEntity, ITestBeforeTrigger>();

        // Act
        await _service.ValidateAsync(new TestEntity(), EntityStateChangeEnum.Added, [], null!);

        // Assert
        triggerMock.Verify(x => x.HandleAsync(It.IsAny<EntityCancelEventArgs<TestEntity>>()),
Times.Once);
    }

    [Fact]
    public async Task ValidateAsync_ShouldThrowException_WhenTriggerCancels()
    {
        // Arrange
        string errorMessage = "Stop!";
        var triggerMock = new Mock<ITestBeforeTrigger>();
        triggerMock.Setup(x => x.HandleAsync(It.IsAny<EntityCancelEventArgs<TestEntity>>()))
            .Callback<EntityCancelEventArgs<TestEntity>>(args =>
        {
            throw new InvalidOperationException(errorMessage);
        })
    }
}

```

```

        args.Cancel = true;
        args.ErrorMessage = errorMessage;
    });

    _serviceProviderMock.Setup(x => x.GetService(typeof(ITestBeforeTrigger)))
        .Returns(triggerMock.Object);

    _service.Register<TestEntity, ITestBeforeTrigger>();

    // Act & Assert
    var ex = await Assert.ThrowsAsync<OperationCanceledException>( () =>
        _service.ValidateAsync(new TestEntity(), EntityStateChangeEnum.Added, [], null!));

    Assert.Equal(errorMessage, ex.Message);
}

[Fact]
public async Task Hierarchy_ShouldInvokeTriggerForInterface()
{
    // Arrange
    var triggerMock = new Mock<IBeforeSaveTrigger<IAudit>>();
    _serviceProviderMock.Setup(x => x.GetService(typeof(IBeforeSaveTrigger<IAudit>)))
        .Returns(triggerMock.Object);

    // B 5C48D BD 8D CCT< C00 C,,=D$5D DCT9D
    _service.Register<IAudit, IBeforeSaveTrigger<IAudit>>();

    // Act: TestEntity D 5C ;C,,7D45D" " VF-M
    await _service.ValidateAsync(new TestEntity(), EntityStateChangeEnum.Added, [], null!);

    // Assert
    triggerMock.Verify(x => x.HandleAsync(It.IsAny<EntityCancelEventArgs<IAudit>>()),
        Times.Once);
}

[Fact]
public void Register_ShouldThrow_WhenTriggerDoesNotImplementInterfaces()
{
    // Act & Assert
    Assert.Throws<InvalidOperationException>( () =>
        _service.Register<TestEntity, InvalidTrigger>());
}

[Fact]
public async Task NotifyAsync_ShouldStopExecution_WhenHandledIsTrue()
{
    // Arrange
    var triggerMock1 = new Mock<ITestAfterTrigger>();
    triggerMock1.Setup(x => x.HandleAsync(It.IsAny<EntityChangedEventArgs<TestEntity>>()))
        .Callback<EntityChangedEventArgs<TestEntity>>(args => args.Handled = true);

    var triggerMock2 = new Mock<ISecondAfterTrigger>();

    _serviceProviderMock.Setup(x =>
        x.GetService(typeof(ITestAfterTrigger))).Returns(triggerMock1.Object);
    _serviceProviderMock.Setup(x =>
        x.GetService(typeof(ISecondAfterTrigger))).Returns(triggerMock2.Object);

    _service.Register<TestEntity, ITestAfterTrigger>();
    _service.Register<TestEntity, ISecondAfterTrigger>();

    // Act
    await _service.NotifyAsync(new TestEntity(), EntityStateChangeEnum.Added, [], "User",
        DateTime.UtcNow);

    // Assert
    triggerMock1.Verify(x => x.HandleAsync(It.IsAny<EntityChangedEventArgs<TestEntity>>()),
        Times.Once);
    triggerMock2.Verify(x => x.HandleAsync(It.IsAny<EntityChangedEventArgs<TestEntity>>()),
        Times.Never);
}
}
</file>

<file path="Tests/Trigger/FluentValidationTriggerTests.cs">
using FluentValidation;

```

```

using Microsoft.Extensions.DependencyInjection;
using Moq;
using Promatis.Net.Data;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using Xunit;

namespace Promatis.Net.ApplicationModel.Tests.Trigger;

public class FluentValidationTriggerTests
{
    private class TestEntity : DomainObject { public string Name { get; set; } = ""; }

    private readonly Mock<IServiceProvider> _serviceProviderMock;
    private readonly Mock<IServiceScopeFactory> _scopeFactoryMock;
    private readonly Mock<IServiceScope> _serviceScopeMock;

    private class TestEntityValidator : AbstractValidator<TestEntity>
    {
        public TestEntityValidator()
        {
            RuleFor(x => x.Name).NotEmpty().WithMessage("Name is required");
        }
    }

    public FluentValidationTriggerTests()
    {
        _serviceProviderMock = new Mock<IServiceProvider>();
        _scopeFactoryMock = new Mock<IServiceScopeFactory>();
        _serviceScopeMock = new Mock<IServiceScope>();

        // A00D BD 0C,,2C 5CÂ FCT?CÄGC=C: BD0C @C,,:C ACä7CD0CTB Scope -> Scope C$>Ct2D 0D"0CTB ServiceProvide
        _serviceScopeMock
            .Setup(s => s.ServiceProvider)
            .Returns(_serviceProviderMock.Object);

        _scopeFactoryMock
            .Setup(f => f.CreateScope())
            .Returns(_serviceScopeMock.Object);
    }

    [Fact]
    public async Task HandleAsync_ShouldCancel_WhenValidationFails()
    {
        // Arrange
        var entity = new TestEntity { Name = "" };
        var validator = new TestEntityValidator();

        _serviceProviderMock
            .Setup(sp => sp.GetService(typeof(IValidator<TestEntity>)))
            .Returns(validator);

        // A$=CT4D OCT< C0@Cä2C 9CD5D =C ?D 0CÄCDâ 2 D$@C,,3C45D
        var trigger = new FluentValidationTrigger(_scopeFactoryMock.Object);
        var args = new EntityCancelEventArgs<IDomainObjectHasKey<Guid>>(
            entity, EntityStateChangeEnum.Added, [], null!);

        // Act
        await trigger.HandleAsync(args);

        // Assert
        Assert.True(args.Cancel);
        Assert.Contains("Name is required", args.ErrorMessage);
    }

    [Fact]
    public async Task HandleAsync_ShouldNotCancel_WhenValidationSucceeds()
    {
        // Arrange
        var entity = new TestEntity { Name = "Valid Name" };
        var validator = new TestEntityValidator();

        _serviceProviderMock
            .Setup(sp => sp.GetService(typeof(IValidator<TestEntity>)))
            .Returns(validator);
    }
}

```

```

        var trigger = new FluentValidationTrigger(_scopeFactoryMock.Object);
        var args = new EntityCancelEventArgs<IDomainObjectHasKey<Guid>>()
            { entity, EntityStateChangeEnum.Added, [], null! };
    }

    // Act
    await trigger.HandleAsync(args);

    // Assert
    Assert.False(args.Cancel);
    Assert.Null(args.ErrorMessage);
}

[Fact]
public async Task HandleAsync_ShouldIgnore_WhenNoValidatorRegistered()
{
    // Arrange
    _serviceProviderMock
        .Setup(sp => sp.GetService(It.IsAny<Type>()))
        .Returns(null!);

    var trigger = new FluentValidationTrigger(_scopeFactoryMock.Object);
    var args = new EntityCancelEventArgs<IDomainObjectHasKey<Guid>>()
        { new TestEntity(), EntityStateChangeEnum.Added, [], null! };

    // Act
    await trigger.HandleAsync(args);

    // Assert
    Assert.False(args.Cancel);
}
}
</file>

```

```

<file path="Tests/Trigger/FullTriggerChainTests.cs">
using FluentValidation;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Moq;
using Promatis.Net.Data;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using System.Text.Json;
using Xunit;

namespace Promatis.Net.ApplicationModel.Tests.Trigger;

// --- 1. B$ B "Aä B´ A A !B + ---
public class IntegrationEntity : DomainObject, IAudit
{
    public string Name { get; set; } = "";
}

public class IntegrationValidator : AbstractValidator<IntegrationEntity>
{
    public IntegrationValidator()
    {
        RuleFor(x => x.Name).NotEmpty().WithMessage("NameRequired");
    }
}

public class IntegrationDbContext(
    DbContextOptions<ApplicationDbContext> options,
    IConfiguration configuration)
    : ApplicationDbContext(options, configuration)
{
    // 1. A,,!Aô A A´ Aô AS¢ CT;C 5CÂ BCTAD$>C$KC´ CCT5C² ?Cä;CÔ>Dd5CÔ=D´< C,,7Cä;C,,@Cä2C =CÔKCÂ 4CT@CT2Cä<
    private class TestNode : ReferenceTreeBase<TestNode>
    {
    }
}

protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    // 1. At0CôCD :C 5CÂ 1C 7Cä2D´9 C 2D$>CÄ0D$8Dt5D :C,,9 D :C =CT@ Promatis
    base.OnModelCreating(modelBuilder);
}

```

```

    }
    // 2. A00D BD >C":C 4C'O InMemory C0@Cä2C 9CD5D 0D
    if (Database.ProviderName == "Microsoft.EntityFrameworkCore.InMemory")
    {
        modelBuilder.Entity<TestNode>(builder =>
        {
            // At0CD0CT< D02C0>CR 8CÄO D$0C ;C,,FD²Â GD$>C K InMemory C,,7Cä;C,,@Cä2C ; CTQD
            builder.ToTable("FullTriggerChain_TestNodes");

            // A,,!A0 A A' A0 AS¢ C AD$@C 8C$0CT< D 2D07DÄ AD$@Cä3Cä =C BC,,?CR FW7DæöFRÄ
            // C,,AC0>C'LCtCD0 AD$@Cä3Cä BC,,?C,,7C,,@Cä2C =C0KCR æ ÖVöb >D" AC <Cä3Cä :C'0D AC BCTAD"0=Cä4D
            builder.HasOne(x => x.Parent)
                .WithMany(x => x.Children)
                .HasForeignKey(x => x.ParentId)
                .OnDelete(DeleteBehavior.Restrict);
        });

        // B$0C=6CR @CT3C,,AD$@C,,@D45CÄ AC <D2 BCTAD$>C$CDä 8C0BCT3D 0Dd8Cä=C0CDä AD4IC0>D BDÍ
        modelBuilder.Entity<IntegrationEntity>();
    }
}

public class FullTriggerChainTests
{
    [Fact]
    public async Task SaveChanges_ShouldExecuteFullChain_AndCancelOnValidationError()
    {
        // --- ARRANGE ---

        // 1. B >Ct4C 5CÄ DCT9C= >C$CDä :Cä=DD8C4CD 0Dd8Dí
        IConfigurationRoot configuration = new ConfigurationBuilder()
            .AddInMemoryCollection(new Dictionary<string, string>()
            {
                ["DatabaseSettings:DefaultSchema"] = "test"
            })
            .Build();

        var services = new ServiceCollection();
        services.AddLogging();
        services.AddSingleton<IConfiguration>(configuration); // AD>C 0C$;D05CÄ :Cä=DD8C2 2 DÍD

        // 2. A,,=DD@C AD$@D4:D$CD =D'5 C00D BD >C":C•
        services.AddSingleton(new JsonSerializerOptions { PropertyNamingPolicy =
        JsonNamingPolicy.CamelCase });
        // AÄ>C¢ DC 1D 8C=8 D$5C05D L C$>Ct2D 0D"0CTB C= >C0BCT:D B D :Cä=DD8C4>CÍ
        var factoryMock = new Mock<IDbContextFactory<ApplicationDbContext>>();
        services.AddSingleton(factoryMock.Object);

        // 3. B 5C48D BD 0Dd8D0 ACT@C$8D >C" BD 8C43CT@Cä2D
        services.AddScoped<DatabaseTriggerService>();
        services.AddScoped<IDatabaseTriggerService>(sp =>
        sp.GetRequiredService<DatabaseTriggerService>());

        // B 0CÄ8 D$@C,,3C45D K0
        services.AddScoped<FluentValidationTrigger>();
        services.AddScoped<ReferenceTreeParentTrigger>();
        services.AddScoped<AuditTrigger>();

        // 4. At0C4;D4HC=8 CD;D0 8C0BCT@DD5C"ACä2 (DtBCä1D² FCT?CäGC=0 C05 D 2C ;C ADÄÄ 5D ;C, BD 8C43CT@ D$0
        services.AddScoped(_ => new Mock<IBeforeSaveTrigger<IAudit>>().Object);
        services.AddScoped(_ => new Mock<IBeforeSaveTrigger<IDomainObject>>().Object);

        // 5. B 5C48D BD 0Dd8D0 2C ;C,,4C BCä@C
        services.AddTransient<IValidator<IntegrationEntity>, IntegrationValidator>();

        ServiceProvider serviceProvider = services.BuildServiceProvider();

        using IServiceScope scope = serviceProvider.CreateScope();
        IServiceProvider sp = scope.ServiceProvider;
        var triggerService = sp.GetRequiredService<DatabaseTriggerService>();

        // A0@C,,2D07D'2C 5CÄ f¢VV¢Ef Æ-F F-öâG&-vvW" :Cä 2D 5CÄ AD4IC0>D BD0< D wV-B0:C'NDt>CÍ
        triggerService.Register<IDomainObjectHasKey<Guid>, FluentValidationTrigger>();

        // A0@C,,2D07D'2C 5CÄ f¢VV¢Ef Æ-F F-öâG&-vvW" :Cä 2D 5CÄ AD4IC0>D BD0< D wV-B0:C'NDt>CÍ
    }
}

```

```

        triggerService.Register<IDomainObjectHasKey<Guid>, FluentValidationTrigger>();
    }

    // A,,!Aô A A' Aô AD Bô ääUB ¢ Ct2C'5C¤0CT< DD0C @C,,:D2 66÷ R 8Cr BCTAD$>C$>C4> Cô@Cä2C 9CD5D 0D
    var scopeFactory = sp.GetRequiredService<IServiceScopeFactory>();
}

    // Aô0D BD >C":C >CôFC,,9 A D 8D ?D 0C$;CT=CôKCÂ ?CT@CTEC$0D$GC,,:Cä< (Cô5D 5CD0CT< D$>C'LC¤> DD0C
    DbContextOptions<ApplicationDbContext> options = new
DbContextOptionsBuilder<ApplicationDbContext>()
    .UseInMemoryDatabase(Guid.NewGuid().ToString())
    .AddInterceptors(new DatabaseTriggerInterceptor(scopeFactory)) // A,,!Aô A A' Aô : Aä4C,,= C @C4CC
    .Options;

    // B >Ct4C 5CÂ :Cä=D$5C¤AD"Â ?CT@CT4C 2C 0 Cä?Dd8C, 8 C¤>CôDC,,3D
    await using var context = new IntegrationDbContext(options, configuration);
    await context.Database.EnsureCreatedAsync(TestContext.Current.CancellationToken);

    // B >Ct4C 5CÂ >C JCT:D"Â :CäBCä@D'9 Aô Cô@Cä9CD5D" 2C ;C,,4C FC,,N (Name CöCD BCä9)D
    var entity = new IntegrationEntity { Name = "" };
    context.Add(entity);

    // --- ACT & ASSERT ---

    // Aä6C,,4C 5CÂÂ GD$> C,,=D$5D FCT?D$>D 2D'7Cä2CTB D$@C,,3C45D Â BCäB C$Kct>C$5D" 2C ;C,,4C BCä@, C, 1D4
    var exception = await Assert.ThrowsAsync<OperationCanceledException>(async () =>
    {
        await context.SaveChangesAsync(TestContext.Current.CancellationToken);
    });

    // Aô@Cä2CT@Dô5CÂÂ GD$> CD>D,,;C, 8CÄ5CÔ=Cä 4Cä =C HCT3Cä ACä>C ICT=C,,O C,,7 IntegrationValidatorD
    Assert.Equal("NameRequired", exception.Message);
}
}
</file>

```

```

<file path="Tests/Trigger/ReferenceTreeParentTriggerTests.cs">
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Promatis.Net.Data;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using Xunit;
}
namespace Promatis.Net.ApplicationModel.Tests.Trigger;
{
    public class ReferenceTreeParentTriggerTests
    {
        // --- 1. B$ B "Aä B' A¤ A !B + ---
        private class TestNode : ReferenceTreeBase<TestNode> { }

        // A¤>CôBCT:D B D$5Cô5D L Cö@C,,=C,,<C 5D" "6öæf-wW& F-öí
        private class TestDbContext(
            DbContextOptions<ApplicationDbContext> options,
            IConfiguration configuration)
            : ApplicationDbContext(options, configuration)
        {
            protected override void OnModelCreating(ModelBuilder modelBuilder)
            {
                base.OnModelCreating(modelBuilder);
                modelBuilder.Entity<TestNode>();
            }
        }

        // --- 2. Aô AD Aä"Aä A¤ Aä B #Ad Aô Bô òòÝ
        private readonly IConfiguration _configuration;

        public ReferenceTreeParentTriggerTests()
        {
            // B >Ct4C 5CÂ DCT9C¤>C$KC' :Cä=DD8C2 4C'0 D$5D BCä2D
            _configuration = new ConfigurationBuilder()
                .AddInMemoryCollection(new Dictionary<string, string?>()
                {
                    ["DatabaseSettings:DefaultSchema"] = "test"
                })
                .Build();
        }
    }
}

```



```

    }
}

private async Task<TestDbContext> GetContextAsync()
{
    DbContextOptions<ApplicationDbContext> options = new
DbContextOptionsBuilder<ApplicationDbContext>()
        .UseInMemoryDatabase(Guid.NewGuid().ToString())
        .Options;

    var context = new TestDbContext(options, _configuration);
    await context.Database.EnsureCreatedAsync();
    return context;
}

// --- 3. B$ B "B² 00Ý

[Fact]
public async Task Trigger_ShouldCancel_WhenObjectIsItsOwnParent()
{
    // Arrange
    await using TestDbContext context = await GetContextAsync();
    var trigger = new ReferenceTreeParentTrigger();

    var nodeId = Guid.NewGuid();
    // Id C, &VçD-B ACâ2C00CD0DäB
    var node = new TestNode { Id = nodeId, ParentId = nodeId, Name = "Self Parent" };

    var args = new EntityCancelEventArgs<IDomainObjectHasKey<Guid>>(
        node, EntityStateChangeEnum.Added, [], context);

    // Act
    await trigger.HandleAsync(args);

    // Assert
    Assert.True(args.Cancel);
    Assert.Equal("Aä1D±5C±B C05 CÄ>Cd5D" 1D´BDÂ @Cä4C,,BCT;CT< D 0CÄ>CÄC D 5C 5.", args.ErrorMessage);
}

[Fact]
public async Task Trigger_ShouldCancel_WhenParentDoesNotExistInDatabase()
{
    // Arrange
    await using TestDbContext context = await GetContextAsync();
    var trigger = new ReferenceTreeParentTrigger();

    var missingParentId = Guid.NewGuid();
    var node = new TestNode { Name = "Orphan Node", ParentId = missingParentId };

    var args = new EntityCancelEventArgs<IDomainObjectHasKey<Guid>>(
        node, EntityStateChangeEnum.Added, [], context);

    // Act
    await trigger.HandleAsync(args);

    // Assert
    Assert.True(args.Cancel);
    Assert.Contains("C05 C00C"4CT=", args.ErrorMessage);
}

[Fact]
public async Task Trigger_ShouldNotCancel_WhenParentExistsInDatabase()
{
    // Arrange
    await using TestDbContext context = await GetContextAsync();
    var trigger = new ReferenceTreeParentTrigger();

    var parent = new TestNode { Name = "Valid Parent" };
    context.Add(parent);
    await context.SaveChangesAsync(TestContext.Current.CancellationTokens);

    var child = new TestNode { Name = "Child Node", ParentId = parent.Id };
    var args = new EntityCancelEventArgs<IDomainObjectHasKey<Guid>>(
        child, EntityStateChangeEnum.Added, [], context);

    // Act
    await trigger.HandleAsync(args);
}

```

```

    }
    // Assert
    Assert.False(args.Cancel);
    Assert.Null(args.ErrorMessage);
}

[Fact]
public async Task Trigger_ShouldCancel_WhenParentIsSoftDeleted()
{
    // Arrange
    await using TestDbContext context = await GetContextAsync();
    var trigger = new ReferenceTreeParentTrigger();

    var parent = new TestNode
    {
        Id = Guid.NewGuid(),
        Name = "Deleted Parent",
        DeletedAt = DateTime.UtcNow,
        DeletedBy = "Admin"
    };
    context.Add(parent);
    await context.SaveChangesAsync(TestContext.Current.CancellationToken);

    var child = new TestNode { Name = "Child of Dead", ParentId = parent.Id };
    var args = new EntityCancelEventArgs<IDomainObjectHasKey<Guid>>(
        child, EntityStateChangeEnum.Added, [], context);

    // Act
    await trigger.HandleAsync(args);

    // Assert
    Assert.True(args.Cancel);
    Assert.Equal("A05C'Lct0 C00Ct=C GC,,BDÂ @Cä4C,,BCT;CT< D44C ;CT=C0KC' >C JCT:D"â"Â &w2äW'&÷$ÖW76 vR"½
}

[Fact]
public async Task Trigger_ShouldCancel_WhenDirectCyclicDependencyDetected()
{
    // Arrange
    await using TestDbContext context = await GetContextAsync();
    var trigger = new ReferenceTreeParentTrigger();

    // 1. B >Ct4C 5CÂ 8 D >DT@C =D05CÂ @Cä4C,,BCT;DÄAC=8C' MC'5CÄ5C0B A
    var nodeA = new TestNode { Id = Guid.NewGuid(), Name = "Node A" };
    context.Add(nodeA);
    await context.SaveChangesAsync(TestContext.Current.CancellationToken);

    // 2. B >Ct4C 5CÂ 8 D >DT@C =D05CÂ 4CäGCT@C08C' MC'5CÄ5C0B B (B >CD8D$5C'L: A •
    var nodeB = new TestNode { Id = Guid.NewGuid(), Name = "Node B", ParentId = nodeA.Id };
    context.Add(nodeB);
    await context.SaveChangesAsync(TestContext.Current.CancellationToken);

    // 3. A,,<C,,BC,,@D45CÂ ?Cä?D'BC=C D 4CT;C BDÂ Cct5C² " @Cä4C,,BCT;CT< CD;D0 Cct;C ,, CTBC'0: A -> B ->
    nodeA.ParentId = nodeB.Id;

    var args = new EntityCancelEventArgs<IDomainObjectHasKey<Guid>>(
        nodeA, EntityStateChangeEnum.Modified, [], context);

    // Act
    await trigger.HandleAsync(args);

    // Assert
    Assert.True(args.Cancel);
    Assert.Equal("Bd8C=C,,GCTAC=0D0 7C 2C,,AC,,<CäAD$L: C05C'Lct0 C05D 5CÄ5D BC,,BDÂ @Cä4C,,BCT;DÄAC=8C' Cct5
}

[Fact]
public async Task Trigger_ShouldCancel_WhenDeepCyclicDependencyDetected()
{
    // Arrange
    await using TestDbContext context = await GetContextAsync();
    var trigger = new ReferenceTreeParentTrigger();

    // A0>D BD >C,,< Dd5C0>Dt:D3¢ &ö÷B Óâ 6†-ÆB Óâ 7V$6†-ÆM
    var root = new TestNode { Id = Guid.NewGuid(), Name = "Root" };
    context.Add(root);

```

```

    var child = new TestNode { Id = Guid.NewGuid(), Name = "Child", ParentId = root.Id };
    context.Add(child);

    var subChild = new TestNode { Id = Guid.NewGuid(), Name = "SubChild", ParentId = child.Id };
    context.Add(subChild);

    await context.SaveChangesAsync(TestContext.Current.CancellationToken);

    // A,,<C,,BC,,@D45CÂ ?Că?D´BC=C C05D 5CĂ5D BC,,BDÂ AC <D´9 C$5D EC08C´ CCT5C² %&ö÷B" 2C0CD$@DÂ 5C4> C4;D4
    // Bd5C0>Dt:C ?D >C$5D :C, ?Că9CD5D" 2C$5D E: SubChild (C" AB´ Ôâ 6†-ÆB „2 A ) -> Root (C05D 5CĂ5D
    root.ParentId = subChild.Id;

    var args = new EntityCancelEventArgs<IDomainObjectHasKey<Guid>>(&
        root, EntityStateChangeEnum.Modified, [], context);

    // Act
    await trigger.HandleAsync(args);

    // Assert
    Assert.True(args.Cancel);
    Assert.Equal("Bd8C;C,,GCTAC=0D0 7C 2C,,AC,,<CăAD$L: C05C´Lct0 C05D 5CĂ5D BC,,BDÂ @Că4C,,BCT;DĂAC=8C´ CCT5
    }
}
</file>

<file path="Tests/Trigger/TreeTriggerChainTests.cs">
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Moq;
using Promatis.Net.Data;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using System.Text.Json;
using Xunit;

namespace Promatis.Net.ApplicationModel.Tests.Trigger;

// --- B4 A,, A BĂ B´ B #B" Aă!B$ AD B0 At A´/Bd A, "AT!B$ A" 00Ý
public class SoftDeleteNode : ReferenceTreeBase<SoftDeleteNode> { }
public class SelfParentNode : ReferenceTreeBase<SelfParentNode> { }

// Aă1D"8C´ :Că=D$5C=AD" 4C´O C,,=D$5C4@C FC,,>C0=D´E D$5D BCă2 (D$5C05D L D "6öæf-wW& F-öâ•
public class TreeTestDbContext(
    DbContextOptions<ApplicationDbContext> options,
    IConfiguration configuration)
    : ApplicationDbContext(options, configuration)
{
    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);
        modelBuilder.Entity<SoftDeleteNode>();
        modelBuilder.Entity<SelfParentNode>();
    }
}

public class TreeTriggerChainTests
{
    private (ServiceProvider Provider, IConfiguration Config) CreateProviderAndConfig()
    {
        // B >Ct4C 5CÂ DCT9C=C$CDâ :Că=DD8C4CD 0Dd8Dí
        IConfigurationRoot configuration = new ConfigurationBuilder()
            .AddInMemoryCollection(new Dictionary<string, string?>
            {
                ["DatabaseSettings:DefaultSchema"] = "test"
            })
            .Build();

        var services = new ServiceCollection();
        services.AddLogging();
        services.AddSingleton<IConfiguration>(configuration); // AD>C 0C$;D05CĂ :Că=DD8C=

        // 1. AăAC0>C$=D´5 D 5D 2C,,AD%
        services.AddScoped<DatabaseTriggerService>();
        services.AddScoped<IDatabaseTriggerService>(sp =>
            sp.GetRequiredService<DatabaseTriggerService>());
    }
}

```

```

    // 2. B A4 B "B B #AT A$!AR "B A4 AT B½
    services.AddScoped<FluentValidationTrigger>();
    services.AddScoped<AuditTrigger>();
    services.AddScoped<ReferenceTreeParentTrigger>();

    // 3. At0C$8D 8CÄ>D BC, BD 8C43CT@Cä2D
    services.AddSingleton(new JsonSerializerOptions { PropertyNamingPolicy =
JsonNamingPolicy.CamelCase });

    var factoryMock = new Mock<IDbContextFactory<ApplicationDbContext>>();
    services.AddSingleton(factoryMock.Object);

    // 4. At0C4;D4HC8 CD;Dð 8CÔBCT@DD5C"ACä2D
    services.AddScoped(_ => new Mock<IBeforeSaveTrigger<IAudit>>().Object);

    return (services.BuildServiceProvider(), configuration);
}

[Fact]
public async Task SaveChanges_ShouldCancel_WhenParentIsSoftDeleted_ThroughChain()
{
    // --- ARRANGE ---
    (ServiceProvider serviceProvider, IConfiguration configuration) = CreateProviderAndConfig();
    using IServiceScope scope = serviceProvider.CreateScope();
    IServiceProvider sp = scope.ServiceProvider;

    var triggerService = sp.GetRequiredService<DatabaseTriggerService>();
    triggerService.Register<IDomainObjectHasKey<Guid>, ReferenceTreeParentTrigger>();

    // A,,!Aô A A´ Aô AD Bò ääUB ¢ Ct2C´5C00CT< DD0C @C,,:D2 66÷ R 8Cr BCTAD$>C$>C4> Cò@Cä2C 9CD5D 0D
    var scopeFactory = sp.GetRequiredService<IServiceScopeFactory>();

    // A,,!Aô A A´ Aô : Aô5D 5CD0CT< C" 8CÔBCT@Dd5CöBCä@ D$>C´LC0> Cä4C,,= C @C4CCÄ5CÔB – scopeFactoryD
    DbContextOptions<ApplicationDbContext> options = new
DbContextOptionsBuilder<ApplicationDbContext>()
    .UseInMemoryDatabase(Guid.NewGuid().ToString())
    .AddInterceptors(new DatabaseTriggerInterceptor(scopeFactory))
    .Options;

    // Aô5D 5CD0CT< Cä?Dd8C, 8 C0>CÔDC,,3D
    await using var context = new TreeTestDbContext(options, configuration);

    // 1. B >Ct4C 5CÄ $CCD0C´5CÔ=Cä3Cä" @Cä4C,,BCT;Dý
    var deadParent = new SoftDeleteNode
    {
        Id = Guid.NewGuid(),
        Name = "Dead Parent",
        DeletedAt = DateTime.UtcNow
    };
    context.Add(deadParent);
    await context.SaveChangesAsync(TestContext.Current.CancellationTokens);

    // 2. B 5C 5CÔ>C-
    var child = new SoftDeleteNode
    {
        Name = "Child",
        ParentId = deadParent.Id
    };
    context.Add(child);

    // --- ACT & ASSERT ---
    await Assert.ThrowsAsync<OperationCanceledException>(async () =>
    {
        await context.SaveChangesAsync(TestContext.Current.CancellationTokens);
    });
}

[Fact]
public async Task SaveChanges_ShouldCancel_WhenSelfParenting_ThroughChain()
{
    // --- ARRANGE ---
    (ServiceProvider serviceProvider, IConfiguration configuration) = CreateProviderAndConfig();
    using IServiceScope scope = serviceProvider.CreateScope();
    IServiceProvider sp = scope.ServiceProvider;

```

```

var triggerService = sp.GetRequiredService<DatabaseTriggerService>();
triggerService.Register<IDomainObjectHasKey<Guid>, ReferenceTreeParentTrigger>();

// A,,!Aô A A´ Aô AD Bô ääUB ¢ Ct2C´5C¤0CT< DD0C @C,,:D2 66÷ R 8Cr BCTAD$>C$>C4> Cô@Cä2C 9CD5D 0D
var scopeFactory = sp.GetRequiredService<IServiceScopeFactory>();

// A,,!Aô A A´ Aô : Aô5D 5CD0CT< C" 8C0BCT@Dd5C0BCä@ D$>C´LC¤> Cä4C,,= C @C4CCÄ5C0B – scopeFactoryD
DbContextOptions<ApplicationDbContext> options = new
DbContextOptionsBuilder<ApplicationDbContext>()
    .UseInMemoryDatabase(Guid.NewGuid().ToString())
    .AddInterceptors(new DatabaseTriggerInterceptor(scopeFactory))
    .Options;

await using var context = new TreeTestDbContext(options, configuration);

var node = new SelfParentNode { Id = Guid.NewGuid(), Name = "Self" };
node.ParentId = node.Id;
context.Add(node);

// --- ACT & ASSERT ---
var exception = await Assert.ThrowsAsync<OperationCanceledException>(async () =>
    await context.SaveChangesAsync(TestContext.Current.CancellationToken));

Assert.Equal("Aä1D¤5C¤B C05 CÄ>Cd5D" 1D´BDÂ @Cä4C,,BCT;CT< D 0CÄ>CÄC D 5C 5.", exception.Message);
}
</file>

<file path="Tests/Validator/DomainObjectValidatorTests.cs">
using FluentValidation.TestHelper;
using Promatis.Net.Domain;
using Xunit;

namespace Promatis.Net.ApplicationModel.Tests.Validator;

public class DomainObjectValidatorTests
{
    // 1. B >Ct4C 5CÂ BCTAD$>C$K¢´ >C JCT:D" 8 CT3Cä 2C ;C,,4C BCä@D
    private class TestEntity : DomainObject { }

    private class TestEntityValidator : DomainObjectValidator<TestEntity> { }

    private readonly TestEntityValidator _validator = new();

    [Fact]
    public void Validator_ShouldFail_WhenIdIsEmpty()
    {
        // Arrange
        var entity = new TestEntity { Id = Guid.Empty };

        // Act
        TestValidationResult<TestEntity>? result = _validator.TestValidate(entity);

        // Assert
        result.ShouldHaveValidationErrorFor(x => x.Id)
            .WithErrorMessage("A,,4CT=D$8DD8C¤0D$>D >C JCT:D$0 C05 CÄ>Cd5D" 1D´BDÂ ?D4AD$KCÄâ""½);
    }

    [Fact]
    public void Validator_ShouldFail_WhenCreatedAtIsInFuture()
    {
        // Arrange
        var entity = new TestEntity
        {
            CreatedAt = DateTime.UtcNow.AddMinutes(5)
        };

        // Act
        TestValidationResult<TestEntity>? result = _validator.TestValidate(entity);

        // Assert
        result.ShouldHaveValidationErrorFor(x => x.CreatedAt)
            .WithErrorMessage("AD0D$0 D >Ct4C =C,,0 C05 CÄ>Cd5D" 1D´BDÂ 2 C CCDCD"5CÄâ""½);
    }

    [Fact]

```

```

public void Validator_ShouldPass_WhenObjectIsValid()
{
    // Arrange
    var entity = new TestEntity(); // Id C, 7&V FVD B 7C ?Cä;CÖDäBD O C" :Cä=D BD CC=BCä@CR 1C 7Cä2D'E C
}
// Act
TestValidationResult<TestEntity>? result = _validator.TestValidate(entity);
// Assert
result.ShouldNotHaveAnyValidationErrors();
}
}
</file>

<file path="Tests/Validator/ReferenceBaseValidatorTests.cs">
using FluentValidation.TestHelper;
using Promatis.Net.Domain;
using Xunit;
namespace Promatis.Net.ApplicationModel.Tests.Validator;
public class ReferenceBaseValidatorTests
{
    // B$5D BCä2C O D CD"=CäAD$L C, 5E 2C ;C,,4C BCä@
    private class TestReference : ReferenceBase { }
    private class TestReferenceValidator : ReferenceBaseValidator<TestReference> { }

    private readonly TestReferenceValidator _validator = new();

    [Theory]
    [InlineData("")]
    [InlineData(" ")]
    [InlineData("A")] // AÄ5CÖLD,,5 2 D 8CÄ2Cä;Cä2D
    public void Name_ShouldHaveErrors_WhenInvalid(string? invalidName)
    {
        var model = new TestReference { Name = invalidName! };
        TestValidationResult<TestReference>? result = _validator.TestValidate(model);

        result.ShouldHaveValidationErrorFor(x => x.Name);
    }

    [Fact]
    public void Name_ShouldHaveError_WhenTooLong()
    {
        var model = new TestReference { Name = new string('A', 251) };
        TestValidationResult<TestReference>? result = _validator.TestValidate(model);

        result.ShouldHaveValidationErrorFor(x => x.Name)
            .WithErrorMessage("Aö@CT2D'HCT=C <C :D 8CÄ0C'LCÖ0Dö 4C'8CÖ0 CÖ0C,,<CT=Cä2C =C,,O (250 D 8CÄ2Cä;C
    }

    [Theory]
    [InlineData("VALID_CODE_123")]
    [InlineData("REF1")]
    [InlineData("")] // B$5Cö5D L DÖBCä ?D >C"4CTB D4ACö5D,,=Cí
    [InlineData(null)] // A, MD$> D$>Cd5D
    public void Code_ShouldBeValid_WhenCorrectFormat(string? code)
    {
        var model = new TestReference { Code = code };
        TestValidationResult<TestReference>? result = _validator.TestValidate(model);

        result.ShouldNotHaveValidationErrorFor(x => x.Code);
    }

    [Theory]
    [InlineData("C>CEö=C ö@D4AD :Cä<")]
    [InlineData("lower_case")]
    [InlineData("CODE-1")] // B$8D 5 CtöCö@CTICT=Cí
    public void Code_ShouldHaveError_WhenInvalidFormat(string invalidCode)
    {
        var model = new TestReference { Code = invalidCode };
        TestValidationResult<TestReference>? result = _validator.TestValidate(model);

        result.ShouldHaveValidationErrorFor(x => x.Code);
    }
}

```

```

[Fact]
public async Task ValidateValue_ShouldReturnErrors_OnlyForSpecificProperty()
{
    // Arrange
    var model = new TestReference
    {
        Name = "", // AÄHC,,1C=0
        Code = "INVALID-CODE" // AÄHC,,1C=0
    };

    // Act
    // Aô@Cä2CT@Dô5CÂ @C 1CäBD2 2C HCT3Cä 4CT;CT3C BC f Æ-F FUF ÇVR 4C´O Cö>C´O Name
    IEnumerable<string> nameErrors = await _validator.ValidateValue(model,
nameof(TestReference.Name));
    IEnumerable<string> codeErrors = await _validator.ValidateValue(model,
nameof(TestReference.Code));

    // Assert
    Assert.Contains(nameErrors, e => e.Contains("A00C,,<CT=Cä2C =C,,5 Cä1Dô7C BCT;DÄ=Câ''''½
    Assert.Contains(codeErrors, e => e.Contains("A>CB <Cä6CTB D >CD5D 6C BDÄ BCä;DÄ:Câ ;C BC,,=C,,FD2''''½
}

[Theory]
[InlineData(" Name")]
[InlineData("Name ")]
public void Name_ShouldHaveError_WhenHasLeadingOrTrailingSpaces(string invalidName)
{
    var model = new TestReference { Name = invalidName };
    TestValidationResult<TestReference>? result = _validator.TestValidate(model);

    result.ShouldHaveValidationErrorFor(x => x.Name)
        .WithErrorMessage("A00C,,<CT=Cä2C =C,,5 CÔ5 CÄ>Cd5D" =C GC,,=C BDÄADò 8C´8 Ct0C=0C0GC,,2C BDÄADò ?D >
}

[Fact]
public async Task ValidateValue_ShouldIgnoreOtherPropertiesErrors()
{
    // Arrange: Cä1C ?Cä;Dò =CT2C ;C,,4CÔK
    var model = new TestReference { Name = "A", Code = "low_case" };

    // Act: C$0C´8CD8D CCT< B$ A´,A= Name
    IEnumerable<string> nameErrors = await _validator.ValidateValue(model,
nameof(TestReference.Name));

    // Assert
    Assert.Single(nameErrors); // B$>C´LC= CäHC,,1C=0 CD;C,,=D² 8CÄ5C08
    Assert.DoesNotContain(nameErrors, e => e.Contains("A>CB''''² òò D,,8C >C¢ :Cä4C 1D´BDÄ =CR 4Cä;Cd=Ci
}
}
</file>

```

```

<file path="Tests/Validator/ReferenceTreeBaseValidatorTests.cs">
using FluentValidation.TestHelper;
using Promatis.Net.Domain;
using Xunit;

namespace Promatis.Net.ApplicationModel.Tests.Validator;

public class ReferenceTreeBaseValidatorTests
{
    // B$5D BCä2C 0 D CD"=CäAD$L CD;Dò ?D >C$5D :C, 0C AD$@C :D$=Cä3Cä :C´0D AC
    private class TestNode : ReferenceTreeBase<TestNode> { }

    // B$5D BCä2D´9 C$0C´8CD0D$>D
    private class TestNodeValidator : ReferenceTreeBaseValidator<TestNode> { }

    private readonly TestNodeValidator _validator = new();

    [Fact]
    public void Validator_ShouldFail_WhenParentIdIsEqualToId()
    {
        // Arrange
        var nodeId = Guid.NewGuid();
        var node = new TestNode
        {
            Id = nodeId,

```

```

        ParentId = nodeId, // AäHC,,1C□0: ID D >C$?C 4C 5D" A ParentId□
        Name = "Self-referencing node"□
    };□
}
// Act□
TestValidationResult<TestNode>? result = _validator.TestValidate(node);□
}
// Assert□
result.ShouldHaveValidationErrorFor(x => x.ParentId)□
    .WithErrorMessage("Aä1D□5C□B C05 CÄ>Cd5D" 1D´BDÂ @Cä4C,,BCT;CT< D 0CÄ>CÄC D 5C 5");□
}
[Fact]□
public void Validator_ShouldFail_WhenParentIdIsEmptyGuid()□
{□
    // Arrange□
    var node = new TestNode□
    {□
        ParentId = Guid.Empty, // AäHC,,1C□0 C0> C$BCä@Cä<D2 ?D 0C$8C´C□
        Name = "Empty ParentId"□
    };□
}
// Act□
TestValidationResult<TestNode>? result = _validator.TestValidate(node);□
}
// Assert□
result.ShouldHaveValidationErrorFor(x => x.ParentId)□
    .WithErrorMessage("B >CD8D$5C´LD :C,,9 C,,4CT=D$8DD8C□0D$>D =CR <Cä6CTB C KD$L C0CD BD´< GUID");□
}
[Fact]□
public void Validator_ShouldPass_WhenParentIdIsDifferent()□
{□
    // Arrange□
    var node = new TestNode□
    {□
        Id = Guid.NewGuid(),□
        ParentId = Guid.NewGuid(),□
        Name = "Valid child node"□
    };□
}
// Act□
TestValidationResult<TestNode>? result = _validator.TestValidate(node);□
}
// Assert□
result.ShouldNotHaveValidationErrorFor(x => x.ParentId);□
}
[Fact]□
public void Validator_ShouldPass_WhenParentIdIsNull()□
{□
    // Arrange (A□>D 5C0L CD5D 5C$0)□
    var node = new TestNode□
    {□
        Id = Guid.NewGuid(),□
        ParentId = null,□
        Name = "Root node"□
    };□
}
// Act□
TestValidationResult<TestNode>? result = _validator.TestValidate(node);□
}
// Assert□
result.ShouldNotHaveValidationErrorFor(x => x.ParentId);□
}
}
</file>

</files>

```